
사내 AI 비서

환각부터 평가까지, RAG의 모든 것

FastAPI + LangChain + ChromaDB로 만드는 사내 AI 비서. 환각 체험에서 시작해 검색 품질 평가까지, 하나의 프로젝트로 RAG 전체 여정을 경험합니다.

RAG 실전 가이드

목차

들어가며	7
한눈에 보기	8
이 책이 맞는 분	8
권장 사양	9
Ch.1: “ChatGPT에 물어봤더니...” — 환각과 RAG의 첫 만남 (ex01)	10
1.1 입사 3일 차, 첫 번째 임무	10
1.2 “당연히 알고 있죠” — 자신감 넘치는 거짓말	10
1.3 “입사 첫날부터 회사 내부를 아는 직원은 없다”	11
1.4 문서를 직접 넣어주면 되지 않을까?	11
1.5 RAG — “오픈북 시험”으로 바꾸기	12
2.1 용어 정리	13
2.2 이번 챕터 파일 구조	13
2.3 실습 환경 구축	14
2.4 실습 순서	14
2.5 실습 1 — step1_fail.py: LLM에게 직접 물어보기	15
2.6 실습 2 — step2_context.py: 문서를 직접 넣어보기	16
2.7 실습 3 — step3_rag.py: RAG 파이프라인 구성	17
2.8 실습 4 — step3_rag_no_chunking.py: 청킹이 왜 필요한가	19
2.9 실습 5 — step4_rag.py: 추론이 필요한 질문	21
2.10 이것만은 기억하세요	23
Ch.2: “일단 사내 시스템부터” — 사내 시스템 소개 (ex02)	24
1.1 “연차 몇 개 남았어?” — AI가 대답 못 하는 질문	24
1.2 식당에 비유해보자	24
1.3 메뉴판이 필요하다 — CRUD 네 가지	25
1.4 세 개의 테이블	26
1.5 이름을 붙이자	26
2.1 용어 정리	27
2.2 파일 구조	27
2.3 실습 환경 준비	27
2.4 실습 순서	28
2.5 API 엔드포인트 목록	30
2.6 더 알아보기	31
2.7 이것만은 기억하세요	31

Ch.3: “어떤 문서를 넣을까” — 사내 문서 수집 전략과 표준 (ex03)	32
1.1 “이걸 다 넣어야 해?”	32
1.2 도서관에서 배우는 문서 정리	32
1.3 첫 번째 원칙: 넣을 문서와 뺄 문서	33
1.4 두 번째 원칙: 먼저 같은 말로 번역해야 한다	33
1.5 세 번째 원칙: 라벨이 없으면 검색할 수 없다	34
1.6 네 번째 원칙: 잘라야 찾는다 — 청킹 전략	34
1.7 다섯 번째 원칙: 문서는 계속 바뀐다 — 재인덱싱 전략	35
1.8 우리 프로젝트의 문서 구조	35
2.1 용어 정리	35
2.2 문서 표준 규칙 (템플릿)	36
2.3 더 알아보기	37
2.4 이것만은 기억하세요	37
Ch.4: “문서를 ‘지식’으로 바꾸다” — VectorDB 구축 (ex04)	38
1.1 “정리는 했는데, 어떻게 넣지?”	38
1.2 요리를 떠올려보겠습니다	38
1.3 1단계: 손질 — 문서에서 텍스트 꺼내기 (파싱)	39
1.4 2단계: 다지기 — 적당한 크기로 자르기 (청킹)	41
1.5 3단계: 양념 — 의미를 숫자로 바꾸기 (임베딩)	41
1.6 4단계: 냉장고 정리 — 벡터 DB에 저장	42
2.1 용어 정리	43
2.2 실습 환경 구축	43
2.3 파일 계층 구조	44
2.4 실습 순서	45
2.5 실습 1: 파싱부터 확인합니다 (main.py -step 1)	45
2.6 실습 2: 전체 파이프라인 실행 (main.py)	46
2.7 실습 3: 벡터 검색 CLI 도구 (cli_search.py)	47
2.8 [설명] extractor.py — 형식별 텍스트 추출	48
2.9 [설명] chunker.py — 청킹 알고리즘	50
2.10 [설명] store.py — 임베딩 + ChromaDB 저장	52
2.11 더 알아보기	53
2.12 이것만은 기억하세요	53
Ch.5: “드디어 답해준다” — RAG Q&A 엔진 (ex05)	55
1.1 “그냥 답을 알려줘”	55
1.2 도서관 사서가 필요하다	56
1.3 파이프라인이라는 이름의 레시피	56
1.4 “출처를 대”	56
1.5 사서도 기억력이 있다	57
1.6 이번 버전에서 뭘 만드나	57
2.1 용어 정리	58
2.2 실습 환경 구축	59

2.3 파일 계층 구조	60
2.4 실습 순서	61
2.5 [설명] chat_api.py — POST /api/chat	61
2.6 실습 1: LCEL 파이프라인 (rag_chain.py)	62
2.7 [설명] response_parser.py — 답변 정제	64
2.8 실습 2: 멀티턴 대화 (conversation.py)	65
2.9 실행 결과	67
2.10 더 알아보기	68
2.11 이것만은 기억하세요	69
Ch.6: “연차 몇 개, 규정은?” 동시에” — 통합 에이전트 설계 (ex06)	70
1.1 RAG가 답 못 하는 질문	70
1.2 두 종류의 정보	70
1.3 안내데스크를 떠올려봅시다	71
1.4 안내데스크: QueryRouter	71
1.5 담당자들: 도구(Tool)	72
1.6 생각하고, 행동하고, 확인한다: ReAct	73
1.7 이제 복합 질문에 답할 수 있다	74
1.8 이번 버전에서 뭘 만드나	74
2.1 용어 정리	75
2.2 파일 구조	75
2.3 실습 환경 구축	76
2.4 실습 순서	78
2.5 실습 1: QueryRouter — 3단계 질문 분류기 (router.py)	78
2.6 실습 2: @tool 데코레이터 — 에이전트용 도구 만들기 (mcp_tools.py)	82
2.7 실습 3: ReAct 에이전트 조립하기 (agent.py)	85
2.8 실행 결과	89
2.9 더 알아보기	92
2.10 이것만은 기억하세요	92
Ch.7: “실제로 써보니” — 운영을 위한 안정화 (ex07)	93
1.1 “왜 이렇게 느려?”	93
1.2 사서에게 메모장을 하나 더 줬다	93
1.3 업무 일지를 쓰기 시작했다	94
1.4 재시도도 배웠다	94
1.5 이번 버전에서 뭘 만드나	94
2.1 용어 정리	95
2.2 파일 계층 구조	95
2.3 실습 환경 구축	96
2.4 실습 순서	97
2.5 실습 1: ConnectHRAgent — 캐시/모니터링/재시도 통합 (agent_config.py)	97
2.6 실습 2: ResponseCache + EmbeddingCache (cache.py)	100
2.7 실습 3: TokenTracker + JSON 로깅 (monitoring.py)	103

2.8 [참고] 도구 모듈화	105
2.9 실행 결과	106
2.10 더 알아보기	108
2.11 이것만은 기억하세요	108
Ch.8: “엉뚱한 문서를 가져온다” — 검색 품질 튜닝 (ex08)	109
1.1 “병가 규정을 물어봤는데 출장 규정이 나왔다”	109
1.2 사서의 검색 실력	109
1.3 이번 버전에서 뭘 바꾸나	110
2.1 용어 정리	111
2.2 파일 계층 구조	111
2.3 실습 환경 구축	112
2.4 실습 순서	112
2.5 실습 1: 청킹 전략 실험 (tuning/step1_chunk_experiment/)	113
2.6 실습 2: Cross-Encoder 리랭킹 (tuning/step2_reranker/)	118
2.7 실습 3: 하이브리드 검색 (tuning/step3_hybrid_search/)	118
2.8 더 알아보기	119
2.9 이것만은 기억하세요	120
Ch.9: “질문을 제대로 이해 못한다” — 질의+결과 고급 튜닝 (ex09)	121
1.1 “휴가”라고 물었는데 못 찾는다	121
1.2 사서의 언어 능력이 자라다	121
1.3 CH08과 CH09의 차이	122
2.1 용어 정리	123
2.2 파일 계층 구조	123
2.3 실습 1: 고급 Retriever (tuning/step1_advanced_retriever/)	124
2.4 실습 2: 질문을 바꿔서 검색한다 (tuning/step2_query_rewrite/)	127
2.5 더 알아보기	130
2.6 CH08과 CH09의 조합	131
2.7 이것만은 기억하세요	132
Ch.10: “PDF 이미지까지 잡아라” — 고급 문서 처리와 평가 (ex10)	133
1.1 표가 깨졌다	133
1.2 사서에게 눈을 달아주자	133
1.3 매번 골라야 해?	134
1.4 그런데, 진짜 좋아진 건 맞아?	134
1.5 성적표를 받았으니, 진짜 서비스를 만들자	135
1.6 이번 버전에서 뭘 만드나	135
1.7 전체 프로젝트 회고	135
2.1 용어 정리	136
2.2 실습 환경 구축	137
2.3 파일 계층 구조	138
2.4 실습 순서	138
2.5 실습 1: 문서 파싱 전략 비교 (tuning/step1_document_parser/)	138

2.6 실습 2: 하이브리드 이미지 처리 (tuning/step2_hybrid_parser/)	142
2.7 실습 3: RAG 평가 프레임워크 (tuning/step3_eval_framework/)	145
2.8 실습 4: 문서 캡처와 근거 표시 (src/capture.py, src/evidence.py)	148
2.9 더 알아보기	152
2.10 이것만은 기억하세요.	153
에필로그: 당신만의 AI 비서를	154
부록	155
A. 환경 설정	155
Python 설치	155
가상환경 설정	155
자주 만나는 오류	155
LLM 설정	156
비전 LLM 설정 (CH10)	157
.env 파일 예시	157
Docker (PostgreSQL)	158
핵심 패키지 요약	158
C. 더 알고 싶다면	158

들어가며

팀장: “AI로 사내 문서 검색 시스템 만들어봐.”

입사 첫 주였습니다. 사수도 없었습니다. **1일차.** 일단 ChatGPT에 물어봤습니다. “우리 회사 연차 규정이 어떻게 되나요?” AI는 자신 있게 답했습니다. 연차는 15일이고 3일 전까지 신청하면 된다고요. 우리 회사 규정을 열어보니 연차는 입사일 기준으로 달랐고 신청 기한도 일주일 전이었습니다. AI가 그럴듯하게 거짓말을 한 겁니다. 그날 하나를 깨달았습니다. **AI는 우리 회사 문서를 읽은 적이 없습니다.** 그래서 직접 넣어주기로 했습니다. 문서를 통째로 프롬프트에 복사해봤습니다. 작은 문서 하나는 됐습니다. 규정집 200페이지를 복사하니 토큰 한도를 넘겼습니다. 그때 RAG라는 걸 알게 됐습니다. “오픈북 시험”이라고 생각하면 됩니다. AI가 사내 문서 전체를 외울 필요 없이 질문할 때마다 관련 문서만 찾아서 보여주면 됩니다.

2일차. 문서를 넣기 전에 AI 비서가 조회할 사내 시스템부터 만들어야 했습니다. “팀원 연차 몇 개 남았어?”는 문서가 아니라 데이터베이스에서 답이 나오는 질문이니까요. 직원 정보와 연차, 매출 테이블을 만들고 API를 열었습니다. 식당에 비유하면 웨이터를 고용한 셈입니다. 손님이 주문하면 주방에서 요리를 가져다주는 것처럼요.

3일차. 사내 시스템은 됐습니다. 이제 문서를 넣을 차례입니다. 공유 드라이브를 열었는데 한숨이 나왔습니다. PDF와 DOCX, XLSX, 심지어 HWP까지. 최신 문서와 3년 전 폐기된 문서가 한 폴더에 섞여 있었습니다. (이걸 전부 넣으면?) 쓰레기를 넣으면 쓰레기가 나옵니다. AI도 마찬가지입니다. 도서관을 떠올렸습니다. 사서가 새 책을 받으면 분류하고 라벨을 붙이고 서가에 꽂습니다. 같은 원리입니다. 넣을 문서를 골라내고 형식을 통일하고 메타데이터를 붙이는 것. “신입사원에게 쥐도 되는 문서인가?”를 필터링 기준으로 삼았습니다.

4일차. 문서를 골랐으면 검색 가능한 지식으로 바꿔야 합니다. 요리에 비유하면 네 단계입니다. 재료를 손질하고(파싱) 먹기 좋게 다지고(청킹) 양념을 입히고(임베딩) 냉장고에 넣습니다(벡터DB). “연차”와 “유급 휴가”가 같은 뜻이라는 걸 숫자 배열로 바꿔야 기계가 알아듣습니다.

5일차. 서가가 완성됐습니다. 동료는 서가를 뒤지고 싶어 하지 않았습니다. “검색 결과 다섯 개를 던져주지 말고, 그냥 답을 알려줘.” 사서가 책을 찾아와서 읽어서 정리해서 답해주는 게 필요했습니다. 검색은 재료이고 답변은 요리입니다. 레시피를 만들었습니다. 검색, 질문 조합, AI가 읽고 답변, 정리. 이 네 단계를 하나의 흐름으로 연결했습니다. 출처 없는 답변은 근거 없는 주장이니까요. “어떤 문서에서 이 답을 찾았는지” 반드시 보여주게 만들었습니다. 한 번 물어보고 끝이 아니라 대화를 이어갈 수 있는 기억력도 붙여줬습니다. 드디어 질문하면 답해주는 AI 비서가 된 겁니다.

6일차. 동료가 이런 질문을 했습니다. “A 사원 연차 몇 개? 그리고 연차 신청 절차는?” 한 문장에 두 가지가 섞여 있었습니다. “연차 몇 개”는 데이터베이스를 조회해야 하고 “신청 절차”는 문서를 검색해야 합니다. RAG만으로는 절반밖에 답할 수 없었습니다. AI 비서를 “안내데스크”로 바꿔야 했습니다. 질문이 들어오면 유형을 분류하고 맞는 담당자를 호출하는 구조입니다. DB를 조회하는 담당자, 문서를 검색하는 담당자, 둘 다 호출해서 합치는 담당자. 이렇게 해서 **ConnectHR** 이 태어났습니다. 정형 데이터와 비정형 문서를 한번에 다루는 사내 AI 비서입니다.

7일차. ConnectHR을 동료에게 풀어줬습니다. 처음에는 신기해했습니다. 같은 질문을 할 때마다 20초씩 기다려야 했습니다. “아까도 물어봤는데 또 기다려?” 사서에게 기억력을 줬습니다. 한 번 답한 내용은 메모해두고 같은 질문이 오면 메모를 보여줍니다. 업무 일지도 쥐여줬습니다. 하루에 토큰을 얼마나 쓰는지, 비용은 얼마인지, 느린 질문은 어떤 건지.

8일차. 운영은 안정됐습니다. 진짜 문제가 터졌습니다. “병가 규정을 물어봤는데 출장 규정이 나왔다.” 검색이 엉뚱한 문서를 가져오고 있었습니다. LLM 탓이 아니었습니다. 사서가 서가에서 잘못된 책을 꺼내온 겁니다. 사서의 검색 실력을 키워야 했습니다. 가위로 일정하게 자르던 걸 의미 단위로 바꾸고 검색 결과를 심사위원에게 다시 줄 세우게 하고 키워드 검색과 의미 검색을 합쳤습니다. 같은 질문인데 답이 달라졌습니다.

9일차. 검색이 좋아지니 이번엔 질문이 문제였습니다. 동료가 “WFH 정책 알려줘”라고 물었는데 ConnectHR이 멈췄습니다. 문서에는 “재택근무”라고 적혀 있었으니까요. “휴가 규정 알려줘”도 마찬가지였습니다. 문서에는 “연차유급휴가”라고 돼 있었습니다. 사서에게 언어 능력을 줬습니다. 약어를 풀어쓰고 동의어를 확장하고 한 질문을 여러 표현으로 바꿔서 각각 검색하게 했습니다. 답변에 근거도 붙였습니다. “이 문서의 3페이지에서 찾았습니다”라고 원본 이미지와 함께.

10일차. 마지막 관문이 남았습니다. 조직도 PDF를 넣었는데 텍스트만 나왔습니다. 박스와 화살표가 사라지고 “대표이사경영지원본부 기술개발본부”가 한 줄로 붙었습니다. 사서에게 눈을 달아줘야 했습니다. 이미지를 보고 의미를 이해하는 능력을 준 겁니다. 성적표도 만 들었습니다. “찾아온 문서 중 몇 개가 정답이었나”, “정답 문서를 몇 개나 찾았나”, “AI가 지어낸 답변은 없었나”. 이 지표로 ConnectHR의 실력을 숫자로 측정했습니다. 튜닝 전과 후의 차이가 눈에 보였습니다. 어디를 더 개선해야 하는지도 보이기 시작했습니다.

한눈에 보기

챕터	핵심 질문	끝나면 뭐가 보이냐
CH01	AI는 왜 거짓말을 할까?	환각 vs RAG 응답 비교
CH02	AI가 조회할 DB는 어떻게 만들까?	CRUD API 동작
CH03	어떤 문서를, 어떤 기준으로 넣을까?	수집 규칙 확립 (개념)
CH04	문서를 어떻게 검색 가능한 형태로 바꿀까?	벡터 검색 + 유사도
CH05	검색 결과를 어떻게 답변으로 만들까?	질문 → 답변 + 출처
CH06	DB 조회와 문서 검색을 한 번에 하려면?	ConnectHR 탄생
CH07	캐시, 비용 추적, 예러 처리는?	운영 안정화
CH08	왜 엉뚱한 문서를 가져올까?	튜닝 전후 비교
CH09	같은 뜻인데 왜 못 찾을까?	쿼리 변환 + 근거
CH10	PDF 이미지는? 품질은 어떻게 측정할까?	성적표 + v1.0 완성

세 가지 전환점이 이야기를 이끅니다. 1. **CH01** — LLM 환각을 보고 RAG가 필요하다는 걸 깨닫습니다. 2. **CH06** — RAG만으로는 부족하고 DB와 문서를 합쳐야 한다는 걸 깨닫습니다. 3. **CH08** — 기본 RAG로는 한계가 있고 튜닝이 필요하다는 걸 깨닫습니다.

모든 챕터는 **실패에서 시작** 합니다. 에러를 만나고 원인을 찾고 해결하는 과정이 곧 학습입니다.

각 챕터는 “이야기 파트”와 “기술 파트”로 나뉩니다. 이야기 파트에서 왜 이 기술이 필요한지를 상황과 비유로 먼저 보여주고 기술 파트에서 코드로 구현합니다. 이야기 파트만 읽어도 전체 흐름을 이해할 수 있습니다.

이 책이 맞는 분

- Python 기초 문법은 아는 분 (함수, 클래스, 패키지 설치)

- LLM이나 RAG는 처음이거나 이론만 읽어본 분
- “조각난 예제 말고 하나의 프로젝트를 처음부터 끝까지 만들어보고 싶은” 분

권장 사양

이 책의 예제를 따라하려면 아래 환경이 필요합니다.

항목	권장 사양	비고
Python	3.10 ~ 3.12	3.13 이상은 일부 호환성 문제 가능
RAM	16GB 이상	임베딩 모델 로딩 시 8GB 이상 사용
디스크	10GB 여유	모델 파일 + Docker 이미지
Docker	Docker Desktop 설치	PostgreSQL 실행에 필요
LLM	Ollama + llama3.1:8b	로컬에서 무료로 실행
Vision LLM	Ollama + qwen2.5vl:7b	CH10 이미지 파싱에 필요

CH06 이후 에이전트 기능(Tool Calling)을 사용하려면 Ollama 기준 **llama3.1:8b** 이상 모델이 필요합니다. CH10에서 PDF 이미지를 분석하려면 비전 모델 **qwen2.5vl:7b** 이 필요합니다.

로컬 환경이 어려운 경우 OpenAI API(gpt-4o-mini)를 대안으로 쓸 수 있습니다. 각 예제의 `.env` 파일에서 **LLM_PROVIDER=openai** 로 변경하고 API 키만 설정하면 동일하게 동작합니다. Ollama 설치나 GPU 없이도 모든 챕터를 진행할 수 있습니다.

자세한 환경 설정 방법은 **부록 A. 환경 설정** 을 참고하세요.

Ch.1: “ChatGPT에 물어봤더니...” — 환각과 RAG의 첫 만남 (ex01)

한 줄 요약: LLM은 우리 회사 문서를 읽은 적이 없다. 문서를 직접 넣어줘야 한다.

핵심 개념: LLM 환각, Context Injection, RAG

1.1 입사 3일 차, 첫 번째 임무



입사 3일 차, 첫 번째 미션

챗터 오픈ING

커넥트에 입사한 지 3일 차. 아직 사내 Wi-Fi 비밀번호를 포스트잇에 적어 모니터에 붙여놓던 시절입니다. 오전 10시, 팀장이 저를 바라보며 다가옵니다.

팀장: “AI로 사내 문서 검색 시스템 만들어봐. 직원들이 규정이나 정책 찾는 게 번거롭다고 해서. 채팅창에 물어보면 바로 답해주는 거.”

AI 비서. 사내 문서. 대화식 검색. 나 혼자서?

나: “언제까지요?”

팀장: “급하진 않아. 2주 내로 간단한 프로토타입만.”

노트북을 열고 ChatGPT를 실행했습니다.

ChatGPT도 뭐든 대답하잖아. LLM에게 직접 물어보면 되는 거 아니야? 코드부터 짰습니다.

1.2 “당연히 알고 있죠” — 자신감 넘치는 거짓말

연차 규정을 예시로 물어봤습니다.

나: 우리 회사(커넥트)의 신입사원 연차 발생 규정이 어떻게 돼?

LLM이 답했습니다.

LLM: 커넥트 사의 신입사원 연차 규정은 근로기준법에 따라, 입사 후 1년 미만 기간에는 1개월 개근 시 1일의 유급휴가가 발생합니다. 1년 이상 근무 시에는 15일의 연차가 발생하며, 3년 이상 근무한 경우 1년마다 1일씩 추가됩니다...

그렇듯했습니다. 공식적인 느낌도 나고요. 입사할 때 받은 규정집을 꺼내 비교해봤습니다. 커넥트의 실제 규정은 이랬습니다.

신입사원은 입사 후 3년 동안은 연차가 없다. 대신 매월 1회 ‘리프레시 데이’를 유급으로 제공한다. 3년 근속 시 30일의 연차가 일시에 발생한다.

잠깐, 뭐라고?

다시 읽었습니다. 완전히 다른 내용이었습니다. LLM이 방금 그럴듯한 거짓말을 한 겁니다.

1.3 “입사 첫날부터 회사 내부를 아는 직원은 없다”

여기서 의문이 생깁니다. LLM은 왜 자신 있게 틀린 대답을 했을까요? LLM을 이렇게 생각해 보겠습니다. 입사 면접을 보러 온 외부인 이라고요. 이 외부인은 세상에 공개된 거의 모든 자료를 읽었습니다. 인터넷, 뉴스, 책, 논문까지. 공개된 텍스트라면 뭐든 섭렵했습니다. 그래서 근로기준법은 완벽하게 알고 일반적인 회사 연차 제도도 줄줄 외웁니다. 그런데 커넥트의 내부 규정집은 공개된 적이 없습니다. 이 외부인이 읽을 방법이 없었어요.

문제는 이 외부인이 “모른다”고 솔직히 말하지 못한다는 점입니다. 질문을 받으면 자기가 아는 것 중에서 가장 비슷해 보이는 걸 자신감 있게 말합니다. “아마 일반적인 회사라면 이렇겠지”라는 추측인데, 마치 확실히 아는 것처럼 들립니다. 이게 **LLM 환각(Hallucination)**입니다.

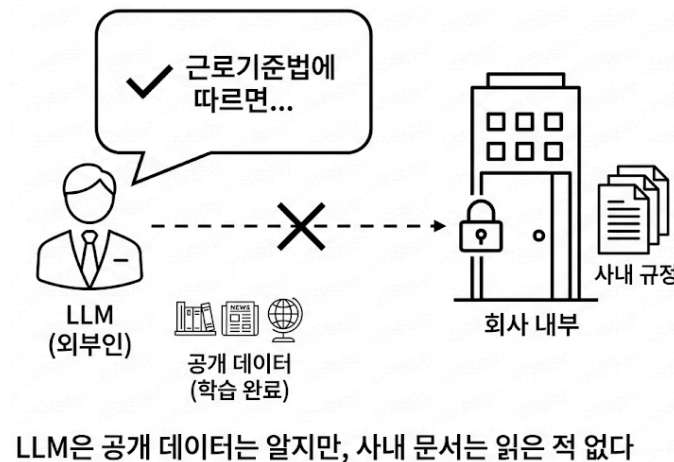


그림 1-1: LLM은 세상의 공개 데이터는 학습했지만, 우리 회사 내부 문서는 읽은 적이 없다.

GPT든 Claude든 Gemini든 마찬가지입니다. 학습 데이터에 없는 정보는 알 방법이 없어요. 그런데 솔직히 “모른다”고 하지 않고 그럴듯하게 지어냅니다. 일반적인 내용과 비슷한 맥락일수록 더 자연스럽게 지어내고요. 커넥트의 연차 규정은 공개된 인터넷 어디에도 없습니다. LLM이 알 리가 없죠. 근로기준법 기반으로 그럴듯한 답을 만들어낸 것뿐입니다.

1.4 문서를 직접 넣어주면 되지 않을까?

생각해보면 해결책은 단순합니다. LLM이 모른다면 직접 알려주면 되지 않을까요? 규정 내용을 통째로 프롬프트에 붙여서 다시 물어봤습니다.

나: 아래 [커넥트 취업규칙]을 참고해서 신입사원 연차 규정을 알려줘.

이번엔 달랐습니다. 커넥트의 실제 규정을 정확히 설명해줬습니다.

(오, 이거면 되는 거 아니야?)

그런데 사내 문서가 규정집 하나가 아닙니다. 복지 정책, 보안 지침, 업무 가이드, 회의록, 프로젝트 문서까지 파일만 수십 개입니다. 매번 전부 복사해서 프롬프트에 붙이면 어떻게 될까요? LLM에는 한 번에 처리할 수 있는 텍스트 길이 한도가 있습니다. 문서가 쌓일수록 한도를 넘기기 쉽고요. 무엇보다 연차 규정을 물어보는데 보안 지침이나 복지 정책까지 다 넣어서 보내는 건 비효율적입니다. 관련 없는 내용이 섞일수록 LLM이 정작 필요한 부분을 놓치기 쉬워집니다.

문서를 통째로 넣는 방식은 임시방편이었습니다.

[이미지: 컨텍스트 넘침] 그림 1-2: 문서를 통째로 넣는 방식의 한계. 문서가 늘어나면 프롬프트 창이 넘친다.

1.5 RAG — “오픈북 시험”으로 바꾸기

더 나은 방법이 있습니다. LLM이 모든 사내 문서를 외울 필요가 있을까요? 사람도 비슷한 문제를 해결한 방식이 있습니다. 시험에서 모든 내용을 통째로 외우는 대신 오픈북을 허용하면 됩니다. 시험지가 나오면 그 문제와 관련된 페이지를 찾아서 보면서 답하는 거죠.

LLM도 마찬가지입니다. 사내 문서 전체를 외울 필요가 없어요. 질문이 들어왔을 때 **그 질문과 관련된 문서 조각만 찾아서 LLM에게 건네주면 됩니다.**

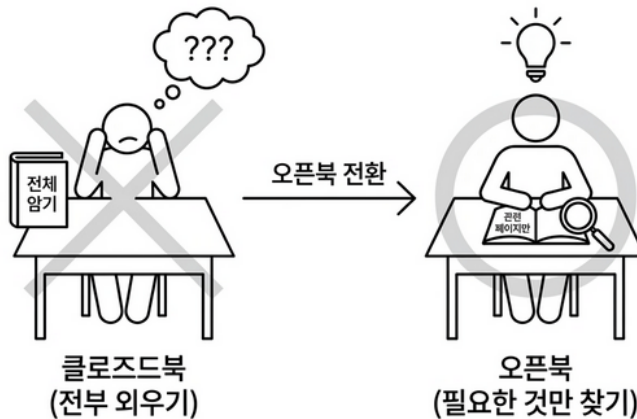
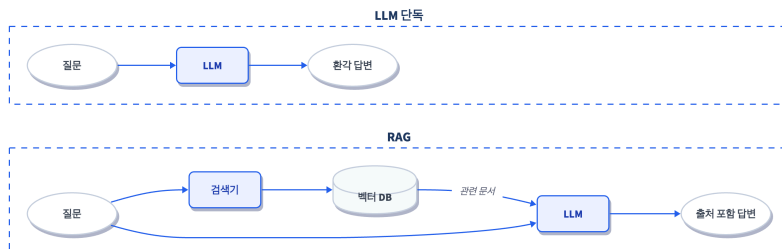


그림 1-3: 클로즈드북 vs 오픈북. RAG는 LLM에게 오픈북 시험을 치르게 하는 것이다.

이것이 **RAG** — Retrieval-Augmented Generation, 검색 증강 생성입니다. 흐름을 보면 이렇게 됩니다.



LLM 단독 호출과 RAG의 차이

그림 1-4: LLM 단독 호출과 RAG의 차이. RAG는 질문마다 관련 문서를 찾아서 LLM에 건네준다.

1. 사내 문서들을 미리 **벡터 DB**에 조각으로 나눠 저장해 놓습니다 (오픈북 준비)
2. 질문이 들어오면, 그 질문과 의미가 비슷한 문서 조각을 벡터 DB에서 찾습니다 (관련 페이지 찾기)
3. 찾은 문서 조각 + 질문을 LLM에게 함께 넘깁니다
4. LLM이 그 문서를 보면서 답합니다 (오픈북으로 시험 보기)

이제 LLM이 우리 회사 규정을 외울 필요가 없습니다. 질문할 때마다 관련 규정을 찾아서 보여주면 되니까요. 어느 문서를 참고했는지도 함께 돌려줄 수 있고요. 이번 챕터의 목표는 이 흐름을 직접 만들어보는 겁니다. 더미 문서 3개짜리 간단한 버전으로 시작하겠습니다. 실제 PDF 파싱이나 한국어 임베딩 모델 적용, DB 연동은 뒤 챕터에서 차례로 붙입니다.

이제 실습으로 LLM 환각을 직접 확인하고, RAG로 해결해보겠습니다.

2.1 용어 정리

이야기 속 표현	진짜 용어	정식 정의
“자신감 넘치는 거짓말”	LLM 환각 (Hallucination)	LLM이 학습 데이터에 없는 정보를 그럴듯하게 만들어내는 현상
“문서를 직접 붙여 넣기”	Context Injection	관련 정보를 프롬프트에 직접 넣어서 LLM에 제공하는 방법
“오픈북 시험”	RAG (Retrieval-Augmented Generation)	외부 지식 저장소에서 관련 문서를 검색해 LLM 생성에 활용하는 방식
“오픈북 준비”	임베딩 + 벡터 DB 저장	문서를 수치 벡터로 변환해 ChromaDB에 인덱싱하는 과정
“관련 페이지 찾기”	벡터 유사도 검색	질문 벡터와 문서 벡터 간 코사인 유사도를 계산해 가장 관련 있는 문서를 반환

2.2 이번 챕터 파일 구조

```
ex01/
├── step1_fail.py          [실습] LLM 단독 호출 → 환각 체험
├── step2_context.py      [실습] 컨텍스트 직접 주입 → 임시 해결
├── step3_rag.py          [실습] RAG 기본 파이프라인 구성
├── step3_rag_no_chunking.py [실습] 청킹 없이 비교 → 차이 체감
└── step4_rag.py          [실습] 추론 심화 (Chain-of-Thought)
```

이 챕터는 더미 문서(3개)로 동작을 확인하는 맛보기 버전입니다. 실제 PDF/DOCX 파싱과 영속 저장은 CH04(VectorDB 구축)에서 다룹니다.

2.3 실습 환경 구축

기본 환경(Python 3.12, Ollama)이 아직 없다면 [부록\(환경 설정\)](#)을 먼저 참고하세요.

```
cd ex01
python3.12 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Ollama 모델이 아직 없다면 다운로드합니다.

```
ollama pull deepseek-r1:8b
ollama pull nomic-embed-text
```

팁: LLM 선택 기본값은 Ollama + **deepseek-r1:8b**입니다(16GB RAM 이상 권장). RAM이 부족하거나 응답이 너무 느리면 `.env`에서 `LLM_PROVIDER=openai`로 바꿔서 GPT-4o-mini를 쓸 수도 있습니다. 단, API 비용이 발생합니다. `.env` 파일에 `OPENAI_API_KEY=sk-xxxxxx` 형태로 키를 등록하세요. 상세 안내는 [프롤로그](#)의 “시작하기 전에”를 참고하세요.

이번 챕터에서는 **LangChain**이라는 프레임워크를 사용합니다. LLM 호출, 벡터 검색, 체인 조립처럼 RAG에 필요한 부품을 제공하는 도구입니다. 여기서는 맛보기로만 쓰고 CH05에서 본격적으로 다룹니다.

패키지	역할
<code>langchain-ollama</code>	Ollama LLM/임베딩 연동
<code>langchain-chroma</code>	ChromaDB 벡터 저장소
<code>langchain-classic</code>	RetrievalQA 체인 (CH05에서 LCEL로 전환)
<code>chromadb</code>	벡터 DB

팁: 지금은 **개념만 잡으세요** LangChain, 임베딩, 벡터 DB 같은 용어가 한꺼번에 나와서 부담스러울 수 있습니다. 지금은 “문서를 넣어주면 LLM이 정확하게 답한다”는 **RAG의 개념**만 잡으면 충분합니다. 각 기술의 동작 원리는 CH04~CH05에서 차근차근 다룹니다.

2.4 실습 순서

1. `step1_fail.py` — LLM 단독 질문
2. `step2_context.py` — 문서 직접 전달
3. `step3_rag.py` — RAG 파이프라인
4. `step3_rag_no_chunking.py` — 청킹 없이 비교

5. step4_rag.py — 추론 심화

환각을 직접 체험하고(step1), 문서를 넣으면 달라지는 걸 확인한 뒤(step2), RAG로 조립합니다(step3). 그다음 청킹 없이 돌려서 차이를 체험하고(step3_no_chunking), 추론이 필요한 질문까지 던져봅니다(step4). **step1부터 순서대로 실행하세요.**

2.5 실습 1 — step1_fail.py: LLM에게 직접 물어보기

아래 코드를 ex01/step1_fail.py에 작성합니다.

```
from langchain_ollama import ChatOllama

# 로컬 LLM 연결
llm = ChatOllama(model="deepseek-r1:8b", temperature=0)

# 질문: 모델이 학습했을 리 없는 가상의 회사 규정
question = "우리 회사(커넥트)의 신입사원 연차 발생 규정이 어떻게 돼?"

print(f"질문: {question}\n")
response = llm.invoke(question)
print(f"답변: \n{response.content}")
```

ChatOllama는 LangChain이 Ollama LLM을 호출할 때 쓰는 래퍼입니다. `temperature=0`은 LLM이 창의적 변형 없이 가장 확률 높은 답변을 내놓게 하는 설정이에요. 실행하면 그럴듯하게 들리지만 커넥트의 실제 규정과는 다른 답변이 나옵니다.

```
# 실행
python step1_fail.py
```

```
bash - ex01.step1_fail

질문: 우리 회사 (커넥트)의 신입사원 연차 발생 규정이 어떻게 돼?

답변:
우리 회사 (커넥트)의 신입사원 연차 (연가) 발생 규정은 **회사의 정책 문서**에 명시되어 있습니다. 일반적으로 신입사원의 연차는 근무 기간과 직무, 회사 정책에 따라 결정되며, 다음과 같은 일반적인 패턴을 따릅니다:

1. **초기 근무 기간**: 신입사원 (예: 입사 1년 이내)은 연차가 적게 부여되거나, 연차가 **월별로 일정 비율로 발생**하는 경우가 많습니다.
2. **연차 발생 방식**:
   - **1일 당 ~ 0.1일~0.2일** (1개월당 1~2시간)로 발생하는 경우가 많습니다.
   - 또는 **연차가 매년 1일씩 증가**하는 방식으로 시작되기도 합니다.

**예시**:
- 입사 1년차: 연차 1일
- 입사 2년차: 연차 2일
- 입사 3년차: 연차 3일
...以此类推 ...

**참고**:
- 정확한 규정은 **인사부문 또는 근로자 협의회**에서 확인 가능합니다.
- 연차 사용 시 **선출시 (사전 승인)**가 필요한 경우가 많으니, 사전에 요청하세요.

**주의**:
회사 정책은 변경될 수 있으므로, **공식 문서**를 반드시 확인해 주세요!
```

그림 1-5: step1_fail.py 실행 결과. 자신감 있게 답하지만 실제 커넥트 규정과 다르다.

2.6 실습 2 — step2_context.py: 문서를 직접 넣어보기

step1에서 LLM이 거짓말하는 걸 봤습니다. 이번에는 **규정 내용을 프롬프트에 직접 포함** 시켜 봅니다. 아래 코드를 **ex01/step2_context.py**에 작성합니다.

```
from langchain_ollama import ChatOllama

llm = ChatOllama(model="deepseek-r1:8b", temperature=0)

# 1. 정보를 변수에 담습니다 (아직 DB 안 씀)
context_data = """
[커넥트 취업규칙]
1. 신입사원은 입사 후 3년 동안은 연차가 없다. (파격적인 규정)
2. 대신 매월 1회 '리프레시 데이'를 유급으로 제공한다.
3. 3년 근속 시 30일의 연차가 일시에 발생한다.
"""

question = "우리 회사(커넥트)의 신입사원 연차 발생 규정이 어떻게 돼?"

# 2. 프롬프트에 정보를 포함시킵니다
prompt = f"""
아래 [참고 정보]를 보고 질문에 답해줘.
[참고 정보]
{context_data}

질문: {question}
"""

print(f"질문: {question}\n")
response = llm.invoke(prompt)
print(f"답변:\n{response.content}")
```

step1과 달라진 부분은 **context_data**를 프롬프트에 직접 넣었다는 것뿐입니다. 이제 정확한 답변이 나옵니다. 하지만 한계도 바로 보여요. 문서 하나면 괜찮지만 수십 개를 매번 통째로 붙이면 프롬프트가 엄청나게 길어집니다. LLM이 처리할 수 있는 텍스트 길이에는 한도(컨텍스트 윈도우)가 있으니까요.

```
# 실행
python step2_context.py
```

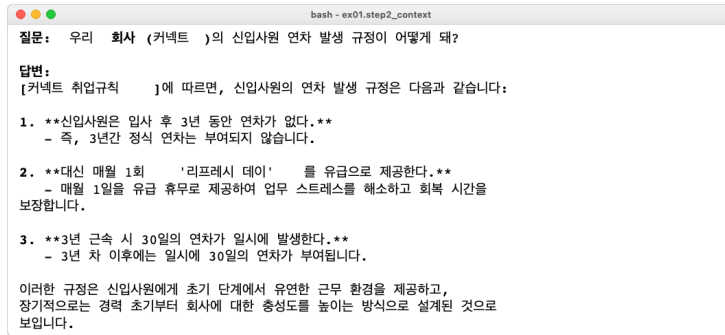


그림 1-6: step2_context.py 실행 결과. 문서를 직접 넣으니 정확하게 답한다.

2.7 실습 3 — step3_rag.py: RAG 파이프라인 구성

step2에서는 문서를 수동으로 넣었습니다. 이번에는 벡터 DB에 문서를 저장하고 질문에 맞는 문서를 자동으로 찾아오는 RAG 파이프라인을 만들어 봅시다. 아래 코드를 ex01/step3_rag.py에 작성합니다.

```
from langchain_classic.chains import RetrievalQA
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings, ChatOllama
from langchain_core.documents import Document
from langchain_core.prompts import PromptTemplate

# 1. 더미 데이터 준비 - 문서 3개를 Document 객체로 만듭니다
docs = [
    Document(
        page_content="[인사규정] 신입사원 휴가 및 연차: 신입사원은 입사 후 처음 3년 동안은 "
        "법정 연차가 발생하지 않습니다. 대신 매월 1회의 유급 '리프레시 데이'를 "
        "휴가로 사용할 수 있습니다.",
        metadata={"source": "인사규정"},
    ),
    Document(
        page_content="[보안규정] 업무 보안: 모든 임직원은 회사에서 지급한 승인된 보안 USB만 "
        "사용해야 하며, 개인 USB나 외부 저장 매체 사용은 엄격히 금지됩니다.",
        metadata={"source": "보안규정"},
    ),
    Document(
        page_content="[복지규정] 식대 지원: 점심 식사는 무제한 법인카드로 지원하며, "
        "저녁 식사는 오후 9시 이후 야근 시에만 사용이 가능합니다.",
        metadata={"source": "복지규정"},
    ),
]

# 2. VectorDB 생성 - 문서를 임베딩하여 ChromaDB에 저장
```

```

print("문서를 학습(임베딩) 중입니다...")
embeddings = OllamaEmbeddings(model="nomic-embed-text")
vectorstore = Chroma.from_documents(documents=docs, embedding=embeddings)

# 3. 검색기 + LLM 체인 연결
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})

template = """당신은 회사의 규정에 대해 설명해주는 AI 비서입니다.
아래의 참고 정보를 바탕으로 질문에 답하세요. 반드시 한국어로 답변해야 합니다.

참고 정보: {context}

질문: {question}
답변: """

llm = ChatOllama(model="deepseek-r1:8b", temperature=0)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True,
    chain_type_kwargs={
        "prompt": PromptTemplate(template=template, input_variables=["context", "question"])
    },
)

# 4. 질문하고 출처 확인
question = "신입사원 휴가 규정에 대해 알려줘."
print(f"\n질문: {question}")
print("-" * 30)

result = qa_chain.invoke({"query": question})

print("\n--- 검색된 문서(근거) ---")
for doc in result["source_documents"]:
    print(f"[{doc.metadata['source']}]: {doc.page_content}")

print("\n--- AI 답변 ---")
print(result["result"])

```

코드가 길어 보이지만 흐름은 네 단계입니다. 1. **문서 준비** — `Document` 객체 3개를 만듭니다. 인사규정, 보안규정, 복지규정이네요. 2. **벡터 DB 저장** — `OllamaEmbeddings`가 각 문서를 수백 차원의 숫자 배열(벡터)로 변환합니다. `Chroma.from_documents()`가 이 벡터를 ChromaDB에 저장하고요. 3. **검색기 + LLM 연결** — `k=3`은 “질문과 가장 비슷한 문서 3개를 가져오라”는 설정입니다. `RetrievalQA`가 검색기와 LLM을 체인으로 연결합니다. 4. **질문 + 출처 확인** — `return_source_documents=True` 덕분에 어떤 문서를 참고했는지 도 함께 돌아옵니다.

이 챕터의 임베딩 모델: `nomic-embed-text`를 사용합니다. CH04에서 한국어에 최적화된 `ko-sroberta-multitask`로 교체합니다.

```
# 실행
python step3_rag.py
```

```
bash - ex01.step3_rag

문서를 학습 (임베딩 ) 중입니다 ...

질문: 신입사원 휴가 규정에 대해 알려줘.
-----

--- 검색된 문서 (근거 ) ---
[복지규정 ]: [복지규정 ] 식대 지원: 점심 식사는 무제한 법인으로 지원하며, 저녁
식사는 오후 9시 이후 야근 시에만 사용이 가능합니다.
[보안규정 ]: [보안규정 ] 업무 보안: 모든 임직원은 회사에서 지급한 승인된 보안
USB만 사용해야 하며, 개인 USB나 외부 저장 매체 사용은 엄격히 금지됩니다.
[인사규정 ]: [인사규정 ] 신입사원 휴가 및 연차: 신입사원은 입사 후 처음 3년 동안은
법정 연차가 발생하지 않습니다. 대신 매월 1회의 유급 '리프레시 데이' 를 휴가로
사용할 수 있습니다.

--- AI 답변 ---
신입사원 휴가 규정에 대해 알려드리겠습니다.

신입사원은 입사 후 처음 3년 동안 법정 연차가 발생하지 않습니다. 대신 매월 1회의
유급 '리프레시 데이' 를 휴가로 사용할 수 있습니다. 이는 회사에서 제공하는 특별
휴가로, 사용 시에는 일반 연차와 마찬가지로 업무 일정 조정이 필요합니다.

추가적인 휴가 사용에 관한 세부 사항은 인사부문과 상담하시는 것이 좋습니다.
```

그림 1-7: `step3_rag.py` 실행 결과. [인사규정] 문서를 찾아서 답변하고, 어디서 가져왔는지 출처까지 보여준다.

이제 답변과 함께 어느 문서를 참고했는지 나옵니다. `step2`에서는 문서를 수동으로 넣어줬지만 이번에는 질문에 맞는 문서를 자동으로 찾아왔습니다. 환각이 사라지고 출처가 생겼습니다.

2.8 실습 4 — `step3_rag_no_chunking.py`: 청킹이 왜 필요한가

`step3`에서 문서 3개를 각각 따로 벡터 DB에 저장했습니다. 이번에는 반대로, 모든 문서를 하나의 덩어리로 합쳐서 저장하면 어떻게 되는지 비교해 봅시다. 아래 코드를 `ex01/step3_rag_no_chunking.py`에 작성합니다.

```
from langchain_classic.chains import RetrievalQA
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings, ChatOllama
from langchain_core.documents import Document
from langchain_core.prompts import PromptTemplate

# 1. 청킹 미적용: 모든 텍스트를 하나의 문자열로 합침 (통짜 데이터)
context_all = """
[인사규정] 신입사원 휴가 및 연차: 신입사원은 입사 후 처음 3년 동안은 법정 연차가 발생하지 않습니다. 대신 매월 1회의 유급 '리프레시 데이'를 휴가로 사용할 수 있습니다.
[보안규정] 업무 보안: 모든 임직원은 회사에서 지급한 승인된 보안 USB만 사용해야 하며, 개인 USB나 외부 저장 매체 사용은 엄격히 금지됩니다.
[복지규정] 식대 지원: 점심 식사는 무제한 법인으로 지원하며, 저녁 식사는 오후 9시 이후 야근 시에만 사용이 가능합니다.
"""
```

```
# 하나의 거대한 문서로 만들 -> 검색이 비효율적임
docs_bad = [Document(page_content=context_all, metadata={"source": "전체규정"})]

# 2. VectorDB 생성
print("문서를 학습(임베딩) 중입니다... (칭킹 미적용)")
embeddings = OllamaEmbeddings(model="nomic-embed-text")
vectorstore = Chroma.from_documents(documents=docs_bad, embedding=embeddings)

# 3. 검색기 및 프롬프트 설정 (통째로 하나뿐이므로 k=1로 검색해도 전체가 다 나옴)
retriever = vectorstore.as_retriever(search_kwargs={"k": 1})

template = """당신은 회사의 규정에 대해 설명해주는 AI 비서입니다.
아래의 참고 정보를 바탕으로 질문에 답하세요. 반드시 한국어로 답변해야 합니다.

참고 정보: {context}

질문: {question}
답변:"""
PROMPT = PromptTemplate(template=template, input_variables=["context", "question"])

# 4. RAG 체인 실행
llm = ChatOllama(model="deepseek-r1:8b", temperature=0)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    chain_type_kwargs={"prompt": PROMPT},
    return_source_documents=True,
)

question = "신입사원 휴가 규정에 대해 알려줘."
print(f"\n질문: {question}")
print("-" * 30)

result = qa_chain.invoke({"query": question})
print(f"\nAI 답변:\n{result['result']}")
```

step3과 비교하면 달라진 부분은 딱 하나입니다. 문서 3개를 하나의 문자열 (`context_all`)로 합쳐서 `Document` 1개로 만들었어요. 벡터 DB에 저장되는 문서가 하나뿐이므로 `k=1`로도 전체가 다 나옵니다. 두 파일을 직접 실행해서 결과를 비교해 보세요.

```
# 실행
python step3_rag_no_chunking.py
```

[이미지: 칭킹 비교 결과] 그림 1-8: 칭킹 여부에 따른 검색 결과 비교. 조각으로 나누면 관련 문서만 정확히 찾는다.

step3에서는 인사규정만 깔끔하게 찾아왔지만 여기서는 인사규정 + 보안규정 + 복지규정이 통째로 들어옵니다. 관련 없는 내용이 섞이면 LLM이 정작 필요한 부분을 놓치기 쉽습니다. 문서를 조각으로 나누는 것, 즉 **청킹(Chunking)** 이 왜 필요한지 체감되실 겁니다. 청킹 전략의 상세 비교는 CH08(검색 품질 튜닝)에서 다룹니다.

2.9 실습 5 — step4_rag.py: 추론이 필요한 질문

step3까지의 질문은 “규정이 뭐야?” 같은 단순 검색이었습니다. 이번에는 **규정을 찾아서 읽고 계산까지 해야 하는 질문** 을 던져봅니다. 아래 코드를 `ex01/step4_rag.py`에 작성합니다.

```
from langchain_classic.chains import RetrievalQA
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings, ChatOllama
from langchain_core.documents import Document
from langchain_core.prompts import PromptTemplate

# 1. 더미 데이터 준비
docs = [
    Document(page_content="[인사규정] 신입사원 휴가 및 연차: 신입사원은 입사 후 처음 3년 동안은 법정 연차가 발생하지 않습니다. 대  
신 매월 1회의 유급 '리프레시 데이'를 추가로 사용할 수 있습니다.", metadata={"source": "인사규정"}),
    Document(page_content="[보안규정] 업무 보안: 모든 임직원은 회사에서 지급한 승인된 보안 USB만 사용해야 하며, 개인 USB나 외부  
저장 매체 사용은 엄격히 금지됩니다.", metadata={"source": "보안규정"}),
    Document(page_content="[복지규정] 식대 지원: 점심 식사는 무제한 법인카드로 지원하며, 저녁 식사는 오후 9시 이후 야근 시에만 사  
용이 가능합니다.", metadata={"source": "복지규정"}),
]

# 2. VectorDB 생성
print("문서를 학습(임베딩) 중입니다...")
embeddings = OllamaEmbeddings(model="nomic-embed-text")
vectorstore = Chroma.from_documents(documents=docs, embedding=embeddings)

# 3. 검색기(Retriever) 설정
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})

# 4. 프롬프트 템플릿
template = """당신은 회사의 규정에 대해 설명해주는 AI 비서입니다.  
아래의 참고 정보를 바탕으로 질문에 답하세요. 반드시 한국어로 답변해야 합니다.  
  
참고 정보: {context}  
  
질문: {question}  
답변: """

PROMPT = PromptTemplate(
    template=template, input_variables=["context", "question"]
)
```

```
# 5. RAG 체인 연결

llm = ChatOllama(model="deepseek-r1:8b", temperature=0)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True,
    chain_type_kwargs={"prompt": PROMPT},
)

# 6. 질문하기 - 추론이 필요한 복잡한 질문
question = "입사 6개월차 신입인데 리프레시 데이 2번 썼어. 몇 번 남았는지 규정 기반으로 계산해줘."
print(f"\n질문: {question}")
print(f"--- * 30")

result = qa_chain.invoke({"query": question})

print("\n--- 검색된 문서(근거) ---")
for doc in result["source_documents"]:
    print(f"[{doc.metadata['source']}] : {doc.page_content}")

print("\n--- AI 답변 ---")
print(result["result"])
```

코드 구조는 step3과 거의 같습니다. 달라진 건 **질문** 뿐이에요. “매월 1회 제공” → “6개월이면 6번” → “2번 썼으면 4번 남음”까지, 규정을 읽고 계산해야 하는 질문입니다.

```
# 실행
python step4_rag.py
```

```
bash - ex01.step4_rag

문서를 학습 (임베딩) 중입니다 ...

질문: 입사 6개월차 신입인데 리프레시 데이 2번 썼어. 몇 번 남았는지 규정 기반으로
계산해줘.
-----

--- 검색된 문서 (근거) ---
[보안규정 ]: [보안규정 ] 업무 보안: 모든 임직원은 회사에서 지급한 승인된 보안
USB만 사용해야 하며, 개인 USB나 외부 저장 매체 사용은 엄격히 금지됩니다.
[복지규정 ]: [복지규정 ] 식대 지원: 점심 식사는 무제한 법인카드로 지원하며, 저녁
식사는 오후 9시 이후 야근 시에만 사용이 가능합니다.
[인사규정 ]: [인사규정 ] 신입사원 휴가 및 연차: 신입사원은 입사 후 처음 3년 동안은
법정 연차가 발생하지 않습니다. 대신 매월 1회의 유급 '리프레시 데이' 를 휴가로
사용할 수 있습니다.

--- AI 답변 ---
입사 6개월차 신입사원으로서 총 6개월 동안 매월 1회씩 리프레시 데이를 사용할 수
있습니다.
이미 2회 사용했으므로, 남은 리프레시 데이는 ** 6 - 2 = 4회**입니다.

**참고** : 신입사원은 입사 후 3년 동안 매월 1회씩 리프레시 데이를 사용할 수
있습니다.
```

실행 결과 1-9: step4_rag.py 실행 결과. 규정을 바탕으로 연차를 스스로 계산하고 추론해 낸 모습이다.

DeepSeek R1은 `<think>` 태그 안에서 단계별로 생각하는 **Chain-of-Thought** 추론을 합니다. 실행하면 검색된 문서(근거)와 함께 계산 과정이 포함된 답변이 나옵니다.

2.10 이것만은 기억하세요

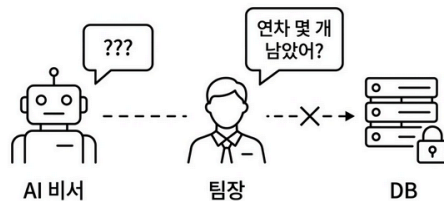
- **LLM은 우리 회사 문서를 읽은 적이 없습니다.** 아무리 자신감 있게 답해도 사내 정보는 우리가 직접 넣어줘야 합니다.
- **RAG는 오픈북 시험입니다.** LLM이 모든 걸 외울 필요 없이 질문마다 관련 문서를 찾아보면서 답합니다.
- 이 챕터의 **RetrievalQA**는 구버전 API입니다. CH05에서 LCEL 파이프라인으로 교체하고, CH04에서 ChromaDB를 디스크에 영구 저장하는 방식으로 바꿉니다.
- 다음 챕터에서는 AI 비서가 조회할 실제 사내 시스템(직원, 연차, 매출 DB)을 FastAPI로 만들어 봅니다.

Ch.2: “일단 사내 시스템부터” — 사내 시스템 소개 (ex02)

한 줄 요약: AI 비서가 조회할 사내 시스템을 실행해보고 구조를 파악한다. API는 웨이터처럼 요청을 받아 DB에서 데이터를 가져다준다.

핵심 개념: REST API, CRUD 패턴

1.1 “연차 몇 개 남았어?” — AI가 대답 못 하는 질문



AI 비서에게 물어봤지만, 조회할 시스템이 없다

챕터 오프닝

지난 챕터에서 RAG의 기본 개념을 알았습니다. 사내 문서를 벡터 DB에 넣어두면 질문할 때 관련 문서를 찾아서 답할 수 있다는 것. 좋습니다. 그런데 팀장이 저를 바라보며 말을 꺼냅니다.

팀장: “문서 검색만 되면 안 되지.”

“‘팀원 연차 몇 개 남았어?’, ‘이번 달 개발팀 매출 얼마야?’ 이런 것도 답해줘야지.”

잠깐. 연차 잔여일은 문서에 적혀있는 게 아닌데?

직원 데이터베이스에서 실시간으로 조회해야 하는 데이터입니다. 매출도 마찬가지고요. 문서 검색으로는 절대 답할 수 없습니다.

AI 비서가 진짜 업무를 도우려면 사내 데이터를 조회할 수 있는 시스템이 먼저 있어야 합니다. AI가 “팀원 연차 잔여일을 알려줘”라고 부탁할 대상. 그게 없었던 겁니다.

그래서 이번 챕터에서는 AI 비서보다 먼저 **사내 시스템**을 실행해봅니다. 코드를 하나하나 뜯어보진 않을 겁니다. 완성된 시스템을 띄워 보고 “이런 데이터를 이렇게 조회할 수 있구나”를 확인하는 게 목표입니다.

1.2 식당에 비유해보자

API가 뭔지 어렵게 생각할 필요 없습니다. 식당을 떠올려보세요.

손님(프론트엔드)이 식당 문을 열고 들어섭니다. “원장찌개 하나요.” 이 주문을 받아 적는 사람이 **웨이터(API)**입니다. 웨이터는 주문서를 들고 **주방(데이터베이스)**으로 향합니다. 잠시 후 요리가 완성되면 웨이터가 손님 테이블로 가져다주죠.

여기서 중요한 건 하나입니다. 손님은 주방에 직접 들어가지 않습니다. 반드시 웨이터를 통해야 해요. 주방 레시피도, 냉장고에 뭐가 있는지도 모릅니다. “원장찌개 주세요.” 그 한마디면 됩니다. 웨이터가 알아서 주방과 소통하니까요.



그림 2-1: API는 식당의 웨이터다. 손님(프론트엔드)과 주방(DB)을 연결한다.

우리가 실행해볼 사내 시스템도 똑같습니다. 나중에 AI 비서가 손님 역할을 맡게 돼요. “팀원 연차 몇 개?”라고 물으면 API(웨이터)가 DB(주방)에서 찾아다 줍니다.

참고: AI는 어떻게 API를 호출할까? 사람이 UI에서 버튼을 누르듯 AI 비서도 API를 호출합니다. CH06에서 **MCP(Model Context Protocol)** 라는 도구를 통해 AI가 직접 API를 호출하는 법을 다룹니다. 지금은 “AI가 쓸 시스템을 먼저 확인해두는 것”에 집중하겠습니다.

1.3 메뉴판이 필요하다 — CRUD 네 가지

식당에 메뉴판이 있듯 API에도 할 수 있는 일의 목록이 있습니다. 사내 시스템에서 데이터를 다루는 기본 동작은 딱 네 가지예요.

식당 비유	데이터 동작	CRUD
새 메뉴 등록	직원 등록	Create
메뉴판 보기	직원 목록 조회	Read
메뉴 가격 변경	직원 정보 수정	Update
메뉴 삭제	직원 삭제	Delete

이 네 가지면 거의 모든 데이터를 관리할 수 있습니다. 직원 정보든 연차 잔여량이든 매출 기록이든. 결국 등록하고 조회하고 수정하고 삭제하는 겁니다.



이 네 가지로 모든 데이터를 관리한다

그림 2-2: API의 메뉴판. 네 가지 동작이면 거의 모든 데이터를 다룰 수 있다.

1.4 세 개의 테이블

우리 사내 시스템이 관리할 데이터는 세 종류예요.

직원(Employee) — 사번, 이름, 부서, 직급, 입사일. “EMP001 김민수 개발팀 대리.”

연차(LeaveBalance) — 누가, 몇 년도에, 총 연차가 며칠이고, 사용한 게 며칠인지. “김민수의 2025년: 총 15일, 사용 3일, 잔여 12일.”

매출(Sale) — 어느 부서가, 언제, 얼마를, 뭘 팔았는지. “개발팀 2025-03-01 5,000,000원 SI프로젝트.”

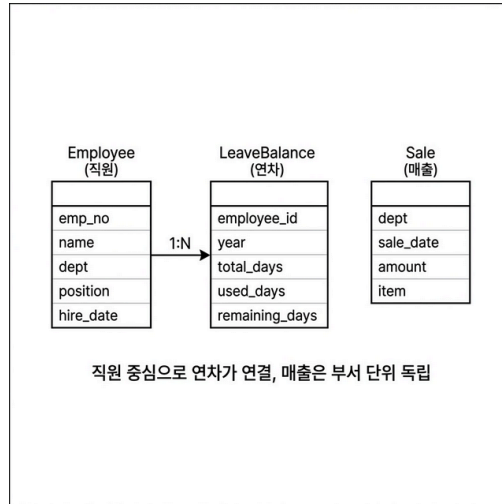


그림 2-3: 사내 시스템의 세 테이블. 직원을 중심으로 연차가 연결되고, 매출은 부서 단위로 독립 관리된다.

이 세 테이블의 데이터를 API로 관리하는 시스템. 그게 이번 챕터에서 확인할 내용이에요.

1.5 이름을 붙이자

시스템 구조를 정리하고 나니 팀장이 한마디 던집니다.

팀장: “이 AI 비서, 이름이 뭐야?”

이름이요? 그냥 ‘AI 비서’라고 부르고 있었는데...

팀장: “프로젝트에 이름이 없으면 회의할 때 불편해. 우리 회사가 커넥트잖아. HR 데이터 다루는 AI 비서니까... **ConnectHR** 어때?” 커넥트의 HR 비서. 짧고 뭘 하는지 바로 알 수 있습니다.

나: “좋네요. ConnectHR.”

이름이 붙으니 프로젝트가 진짜 시작된 느낌입니다. 지금은 사내 시스템만 있는 빈 껍데기지만 앞으로 챕터를 거듭하면서 **ConnectHR** 이 한 단계씩 성장해요. 문서를 읽고 질문에 답하고 DB도 조회하고, 결국 진짜 사내 비서가 되는 여정입니다.

이제 실습으로 사내 시스템을 직접 실행해보겠습니다.

2.1 용어 정리

이야기 속 표현	진짜 용어	정식 정의
“식당 웨이터”	REST API	HTTP 메서드(GET/POST/PATCH/DELETE)로 자원을 조작하는 인터페이스
“메뉴판의 네 동작”	CRUD	Create, Read, Update, Delete — 데이터의 기본 4가지 조작
“주방”	PostgreSQL	관계형 데이터베이스. 테이블 형태로 데이터를 저장하고 SQL로 조회
“주문서 양식”	Pydantic	요청/응답 데이터의 구조와 검증 규칙을 정의하는 Python 라이브러리

2.2 파일 구조

```

ex02/
├── run.py           [참고] 서버 플로우 실행
├── docker-compose.yml [참고] PostgreSQL 컨테이너
├── requirements.txt [참고] 의존성 목록
├── app/
│   ├── main.py     [참고] FastAPI 진입점
│   ├── api.py      [참고] REST API 엔드포인트
│   ├── crud.py     [참고] DB CRUD 로직
│   ├── database.py [참고] PostgreSQL 연결
│   ├── schemas.py  [참고] Pydantic 데이터 검증
│   └── views.py    [참고] 관리자 웹 라우터
├── data/
│   └── schema.sql  [참고] 기본 테이블 및 샘플 데이터
├── templates/      [참고] 웹 UI HTML
└── static/         [참고] 웹 CSS/JS

```

2.3 실습 환경 준비

기본 환경(Python 3.12, Docker)이 없다면 **부록(환경 설정)** 을 먼저 참고하세요.

```

cd ex02
cp .env.example .env
python3.12 -m venv .venv

```

```
source .venv/bin/activate # Windows: .venv\Scripts\activate
docker compose up -d
pip install -r requirements.txt
```

패키지	역할
fastapi	웹 API 서버
uvicorn	ASGI 서버
jinja2	HTML 템플릿 엔진
psycopg2-binary	PostgreSQL 드라이버
pydantic	요청/응답 데이터 검증
python-dotenv	환경 변수 관리

2.4 실습 순서

1. **python run.py** — 서버 시작
2. **/docs** — Swagger UI 확인
3. CRUD 테스트 — POST, GET, PATCH, DELETE
4. **/admin/** — 웹 UI 확인

서버를 실행하면 두 가지 인터페이스를 확인할 수 있습니다. **Swagger UI** (**/docs**)에서 API를 직접 호출해보고 **웹 UI** (**/admin/**)에서 일반 사용자 화면도 확인해 보세요.

```
# 실행
python run.py
```

브라우저에서 <http://localhost:8000/docs>를 열면 **Swagger UI**가 뜹니다. FastAPI가 코드에서 자동으로 만들어주는 API 문서예요.

사내 AI 비서 — CRUD 시스템 1.2.2 OAS 3.1

v0.2 FastAPI + PostgreSQL, CRUD 시스템

default	
GET	/admin/dashboard View Dashboard
GET	/admin/ View Root
GET	/admin/employees View Employees
POST	/admin/employees/create Create Employee View
POST	/admin/employees/update Update Employee View
POST	/admin/employees/delete Delete Employee View
GET	/admin/leaves View Leaves
POST	/admin/leaves/usage Record Leave Usage
POST	/admin/leaves/update Update Leave View
GET	/admin/sales View Sales
POST	/admin/sales/create Create Sale View
API	
GET	/api/employees 직원 목록 조회
POST	/api/insert/insert insert insert

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/api/employees' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "test",
    "dept": "테스터",
    "position": "개발팀",
    "hire_date": "2026-03-06"
  }'
```

Request URL

http://localhost:8000/api/employees

Server response

Code **Details**

201

Response body

```
{
  "id": 8,
  "emp_no": "EMP008",
  "name": "test",
  "dept": "테스터",
  "position": "개발원",
  "hire_date": "2026-03-06"
}
```

 **Download**

Response headers

```
content-length: 107
content-type: application/json
date: Fri, 06 Mar 2026 02:29:19 GMT
server: uvicorn
```

직원, 연차, 매출 — 세 영역의 API가 보입니다. 직접 눌러보세요.

POST로 직원을 등록하고 GET으로 조회하면 방금 등록한 데이터가 돌아옵니다. 수정(PATCH)이나 삭제(DELETE)도 됩니다. 이야기 파트에서 말한 CRUD 네 가지가 전부 동작하는 거예요. Swagger UI는 개발자용입니다. 하지만 이 시스템에는 일반 사용자를 위한 웹 UI도 있어요. 브라우저에서 <http://localhost:8000/admin/>을 열어보세요.

메타코딩

URL

데atabord

회원 관리

로그 관리

대용 관리

데atabord

DB URL: PgvectorSQL (metacoding_db)

회원 수

5

회원 개수 수

5

대용 개수 수

10

총 대용

63,950,000원

오류관리

로그 관리

대용 관리

최근 대용

일	부서	금액	일차	일차
10	영업팀	10,000,000원	2025-09-20	물류팀 구축 GH
9	인사팀	800,000원	2025-09-01	교육 프로그램 운영
8	영업팀	4,000,000원	2025-04-08	재택근무 키트
	영업팀	3,000,000원	2025-04-01	재택근무 키트

그림 2-6: Jinja2 템플릿으로 만든 관리자 대시보드. 직원, 연차, 매출 현황을 한눈에 볼 수 있다.

직원 관리 메뉴에서 사번과 이름, 부서, 직급, 입사일을 입력하고 등록하면 아래 목록에 바로 나타납니다. 기존 직원 5명에서 홍길동 사원이 추가된 걸 확인할 수 있어요.

그림 2-7: 웹 UI에서 직원을 등록하면 목록에 바로 반영된다. API를 몰라도 CRUD가 된다.

API는 뒷단의 배관이고 웹 UI는 수도꼭지입니다. 사용자는 수도꼭지만 틀면 되고 물이 어떤 배관을 타고 오는지 몰라도 돼요. 나중에 AI 비서도 같은 배관(API)을 사용합니다. 다만 수도꼭지 대신 코드로 틀 뿐이에요.

Ctrl + C를 눌러 서버를 종료합니다. Docker 컨테이너도 **docker compose down**으로 정리합니다.

2.5 API 엔드포인트 목록

이 시스템이 제공하는 API 전체 목록입니다. CH06에서 AI 비서가 MCP로 이 API를 호출하게 됩니다.

메서드	경로	설명
GET	<code>/api/employees</code>	직원 목록 조회 (이름/부서 필터)
POST	<code>/api/employees</code>	직원 등록
GET	<code>/api/employees/{id}</code>	직원 상세 조회
PATCH	<code>/api/employees/{id}</code>	직원 정보 수정
DELETE	<code>/api/employees/{id}</code>	직원 삭제
GET	<code>/api/leaves</code>	연차 목록 조회 (직원/연도 필터)
POST	<code>/api/leaves</code>	연차 등록
GET	<code>/api/sales</code>	매출 목록 조회 (부서/기간 필터)
POST	<code>/api/sales</code>	매출 등록
GET	<code>/api/sales/dept-summary</code>	부서별 매출 합계

팁: 코드가 궁금하다면 `code/ex02/app/` 폴더에 전체 소스가 있습니다. FastAPI + psycpg2 + Pydantic 조합으로 만들어져 있어요. 이 책의 주제가 아니므로 코드 설명은 생략하지만 관심 있으면 직접 읽어봐도 좋습니다.

2.6 더 알아보기

Swagger UI — FastAPI는 코드에서 API 문서를 자동 생성합니다. Pydantic 스키마에 적어둔 필드 설명과 타입이 그대로 문서에 나와요. `/docs`는 Swagger UI, `/redoc`은 ReDoc 스타일로 볼 수 있습니다.

DeptSummary — `GET /api/sales/dept-summary`는 부서별 매출 합계를 반환합니다. CH06에서 AI 비서의 `sales_sum` 도구가 이 엔드포인트를 호출해서 “개발팀 매출 얼마야?”에 답하게 됩니다.

2.7 이것만은 기억하세요

- **AI 비서가 조회할 사내 시스템이 준비됐습니다.** API는 식당 웨이터처럼 요청을 받아 DB에서 데이터를 가져다줍니다.
- **CRUD 네 가지면 거의 모든 데이터를 관리할 수 있습니다.** 등록하고 조회하고 수정하고 삭제하기.
- 다음 챕터에서는 AI 비서에게 먹일 사내 문서를 어떻게 수집하고 정리할지 설계합니다.

Ch.3: “어떤 문서를 넣을까” — 사내 문서 수집 전략과 표준 (ex03)

한 줄 요약: AI에게 좋은 답을 원하면 좋은 문서를 넣어야 한다. 도서관처럼 분류하고 라벨을 붙이자.

핵심 개념: 문서 품질, 메타데이터, 청킹 전략 사전 설계, 재인덱싱 전략



공유 드라이브의 문서 더미

1.1 “이걸 다 넣어야 해?”

CH02에서 사내 시스템은 만들었습니다. 직원, 연차, 매출 데이터는 API로 조회할 수 있고요. 이제 AI 비서의 나머지 절반인 **문서 검색**을 본격적으로 준비할 차례입니다. CH01에서 더미 문서 3개로 RAG를 돌려봤습니다. 잘 됐죠. 그런데 이제 실제 사내 문서를 넣어야 합니다. 공유 드라이브를 열었습니다.

취업규칙.pdf, 보안지침_v3_최종_진짜최종.docx, 2024년_복지정책.xlsx, 회의록_0301.hwp, 프로젝트_보고서.pptx...

한숨이 납니다. 형식이 제각각이에요. PDF도 있고 워드도 있고 엑셀도 있고, 심지어 한글 파일까지 있습니다. 어떤 건 최신이고 어떤 건 2년 전 문서입니다. 이걸 통째로 다 벡터 DB에 밀어 넣으면 될까요?

1.2 도서관에서 배우는 문서 정리

도서관을 생각해보겠습니다. 새 책이 기증되면 사서가 바로 서가에 꽂지 않습니다.

1. **먼저 분류합니다** — 이 책이 어느 분야인지, 대여 가능한지, 최신판인지 확인합니다.
2. **라벨을 붙입니다** — 청구기호(위치), 저자, 출판년도, 키워드를 기록합니다.
3. **서가에 꽂습니다** — 분류에 맞는 위치에 넣습니다.

라벨 없이 마구잡이로 꽂아놓으면 어떻게 될까요? 나중에 찾을 수가 없습니다. “경영학 개론이 어딴지?” 하면서 서가 전체를 뒤집게 됩니다. 사내 문서도 마찬가지예요. 벡터 DB에 넣기 전에 **분류하고 라벨을 붙이고 정리하는 단계**가 필요합니다. 이 단계를 건너뛰면 검색 품질이 엉망이 됩니다.

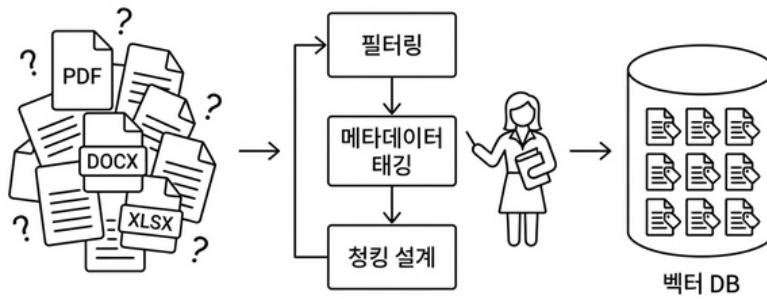


그림 3-1: 사내 문서를 벡터 DB에 넣기까지. 정리 없이 넣으면 검색 품질이 떨어진다.

1.3 첫 번째 원칙: 넣을 문서와 뺄 문서

모든 문서를 넣을 필요는 없습니다. 오히려 필요 없는 문서가 섞여 들어가면 검색 품질이 떨어져요. 연차 규정을 물어봤는데 2년 전 폐기된 규정이 나오면 곤란하겠죠.

넣어야 할 것: 현재 유효한 규정, 정책, 가이드. 자주 질문받는 내용이 담긴 문서.

빼야 할 것: 폐기된 문서, 개인 메모, 초안, 중복 문서(같은 내용의 v1/v2/v3).

간단한 기준 하나면 충분합니다. “이 문서를 신입사원에게 줘도 되나?” 된다면 넣고, 아니라면 뺍니다.

1.4 두 번째 원칙: 먼저 같은 말로 번역해야 한다

PDF, DOCX, XLSX — 형식이 제각각입니다. 이걸 그대로 벡터 DB에 넣을 수는 없어요. 벡터 DB는 **텍스트** 만 이해하니까요.

해외 지사에서 보고서가 도착했다고 상상해보겠습니다. 미국 지사는 영어로, 일본 지사는 일본어로, 프랑스 지사는 프랑스어로 보냈습니다. 이 보고서를 우리 팀원 누구나 검색하고 읽으려면? 먼저 **한국어로 번역** 해서 하나의 언어로 통일해야 합니다. PDF/DOCX/XLSX도 같은 문제예요. 형식이 제각각이면 벡터 DB가 읽지 못합니다. 먼저 **하나의 텍스트 형태** 로 바꿔야 합니다.

이 책에서는 **Markdown** 으로 통일합니다. 왜 Markdown일까요?

- LLM이 가장 잘 이해하는 포맷입니다. 훈련 데이터에 Markdown이 대량 포함되어 있어서 # 제목이나 - 목록 같은 구조를 자연스럽게 인식해요.
- 제목이 보존됩니다. PDF의 큰 글씨, DOCX의 “제목 1” 스타일이 # 제목으로 바뀝니다.
- 표도 보존됩니다. 엑셀 시트가 | 열1 | 열2 | 형태로 바뀌고요.
- 사람도 읽을 수 있습니다. 변환 결과가 제대로인지 눈으로 바로 확인할 수 있죠.

Tip: 반드시 Markdown이어야 하는 건 아닙니다. 일반 텍스트(plain text)나 JSON으로 변환해도 벡터 DB에 넣을 수 있습니다. 다만 Markdown은 제목·표·목록 같은 **문서 구조를 보존하면서도 가볍다**는 점에서 RAG 파이프라인에 가장 널리 쓰입니다.

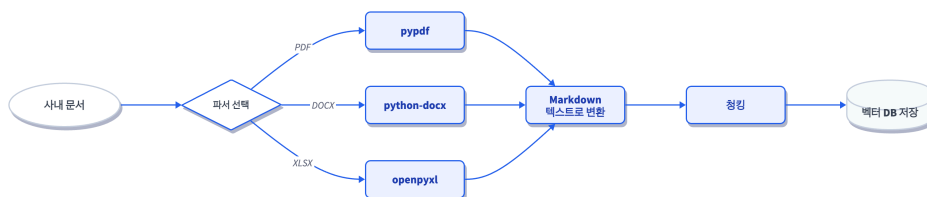


그림 3-2: 파서가 다양한 형식을 Markdown 텍스트로 통일한다. 그래야 청킹하고 검색할 수 있다.

1.5 세 번째 원칙: 라벨이 없으면 검색할 수 없다

도서관에서 책에 붙이는 정보가 있죠. 청구기호, 저자, 분야, 출판년도. 문서 세계에서는 이것 **메타데이터** 라고 부릅니다. 메타데이터가 왜 중요할까요? AI 비서에게 “보안 관련 규정 알려줘”라고 물었을 때 메타데이터에 **file_name: SEC_보안규정**이 있으면 보안 문서를 바로 식별합니다. 없으면? 모든 문서를 처음부터 끝까지 뒤져야 합니다.

어떤 메타데이터를 붙일지는 프로젝트마다 다릅니다. 작성자, 부서, 보안등급, 유효기간 등 필요한 정보가 다양하죠. 우리는 최소한의 것만 쓰기로 했습니다. **파서가 파일명과 경로에서 자동으로 추출할 수 있는 것**만 사용합니다.

메타데이터	추출 방식	예시
파일명 (file_name)	파일에서 자동	HR_취업규칙_v1.0.pdf
파일 형식 (file_type)	확장자에서 자동	pdf
원본 경로 (source_path)	폴더 구조에서 자동	docs/hr/HR_취업규칙_v1.0.pdf
문서 ID (doc_id)	파일명에서 자동 생성	hr_취업규칙_v1_0
페이지 (page)	파싱 시 자동	1

사람이 직접 입력하는 항목이 하나도 없습니다. 대신 **파일명에 정보를 담는 게** 중요해요. HR_취업규칙_v1.0.pdf처럼 분류(HR)와 버전(v1.0)을 파일명에 넣으면 파서가 알아서 메타데이터로 만들어줍니다.

1.6 네 번째 원칙: 잘라야 찾는다 — 청킹 전략

CH01에서 이미 경험했습니다. 문서를 통째로 넣으면 검색 정확도가 떨어져요. 적절한 크기로 **조각(chunk)** 내야 합니다. 조각 크기를 어떻게 정할까요? 너무 작으면 문맥이 잘립니다. “신입사원은 3년 동안 연차가 없다” 다음 줄에 “대신 리프레시 데이를 제공한다”가 있는데 잘못 자르면 “연차가 없다”만 나와요.

너무 크면 CH01의 노청킹 실험처럼 관련 없는 내용까지 달려옵니다.

지금은 설계만 해두겠습니다. 실제 구현은 CH04(VectorDB 구축)에서 하고요.

전략	크기	오버랩	적합한 경우
고정 크기	500자	100자	규정, 매뉴얼 (구조화된 문서)
문단 기준	문단 단위	—	보고서, 회의록 (자연스러운 구분)
의미 기준	가변	—	긴 문서, 주제 전환이 잦은 문서

이 책에서는 CH04에서 **고정 크기(500자 + 100자 오버랩)**으로 시작하고, CH08(검색 품질 튜닝)에서 의미 기준 청킹과 비교 실험을 합니다.

1.7 다섯 번째 원칙: 문서는 계속 바뀐다 — 재인덱싱 전략

사내 문서는 살아있습니다. 취업규칙이 개정되고 새 보안지침이 나오고 복지정책이 바뀌죠. 벡터 DB에 한 번 넣어놓고 끝이 아닙니다. 재인덱싱을 안 하면 어떻게 될까요? AI 비서가 폐기된 규정을 근거로 답변합니다. “연차 15일입니다”라고 답하는데 실제로는 규정이 바뀌어서 20일인 상황이에요. 환각보다 더 위험합니다. 출처까지 달려있으니 그걸 신뢰하게 되거든요.

재인덱싱에는 두 가지 방식이 있습니다. **전체 재인덱싱** — 모든 문서를 지우고 처음부터 다시 넣습니다. 간단하지만 시간이 오래 걸려요. **증분 재인덱싱** — 변경된 문서만 업데이트합니다. 빠르지만 “어떤 문서가 변경됐는지” 추적해야 합니다.

이 책에서는 문서 수가 적으므로 **전체 재인덱싱**으로 충분합니다. 문서가 수천 개 이상이면 증분 방식을 고려합니다.

1.8 우리 프로젝트의 문서 구조

정리해보겠습니다. 우리 사내 AI 비서에 넣을 문서는 이런 구조로 관리합니다.

```
data/docs/
├── hr/                                ← 분류가 폴더명
│   ├── HR_취업규칙_v1.0.pdf          ← 분류_제목_버전이 파일명
│   └── HR_정보보안서약서.pdf
├── security/
│   └── SEC_보안규정_v1.0.docx
├── finance/
│   ├── FIN_2025_상반기_매출현황.xlsx
│   └── FIN_부서별_예산기안서.xlsx
└── ops/
    └── OPS_신규서비스_런칭전략.pdf
```

별도의 메타데이터 파일을 만들 필요가 없습니다. **폴더 구조와 파일명이 곧 메타데이터**니까요. 다음 챕터에서 파서가 이 정보를 자동으로 추출해 벡터 DB에 저장합니다.

이제 실습으로 문서 수집 규칙을 코드로 적용해보겠습니다.

2.1 용어 정리

이야기 속 표현	진짜 용어	정식 정의
“같은 말로 번역”	파싱 (Parsing)	다양한 형식(PDF/DOCX/XLSX)에서 텍스트를 추출해 통일된 형태로 변환하는 과정
“도서관 분류 라벨”	메타데이터 (Metadata)	문서 자체 내용이 아닌 문서에 대한 정보 (파일명, 형식, 경로 등)
“문서 조각내기”	청킹 (Chunking)	긴 문서를 벡터 DB에 저장할 수 있는 크기로 분할하는 과정

이야기 속 표현	진짜 용어	정식 정의
“조각이 겹치는 부분”	오버랩 (Overlap)	청크 경계에서 문맥이 잘리지 않도록 앞뒤를 겹치게 자르는 기법
“문서 다시 넣기”	재인덱싱 (Re-indexing)	변경되거나 추가된 문서를 벡터 DB에 반영하는 과정
“쓰레기 넣으면 쓰레기”	GIGO (Garbage In, Garbage Out)	입력 데이터 품질이 출력 품질을 결정한다는 원칙

2.2 문서 표준 규칙 (템플릿)

실제 프로젝트에서 사내 문서를 관리할 때 참고할 규칙입니다.

1. 파일 형식 제한

허용 형식	파서	비고
PDF	pypdf	텍스트 기반. 이미지 PDF는 CH10에서 OCR 처리
DOCX	python-docx	표, 목록 포함 가능
XLSX	openpyxl	표 형태 데이터 (규정 비교표 등)
TXT/MD	기본 읽기	가장 깔끔

HWP, PPT는 이 책에서 다루지 않습니다. 가능하면 PDF로 변환 후 사용하세요.

2. 메타데이터 — 파일명 규칙

별도의 JSON 파일은 만들지 않습니다. 파서가 파일명과 경로에서 자동 추출하기 때문에 **파일명 규칙**이 중요합니다.

[분류]_[제목]_[버전]·확장자

예시:

HR_취업규칙_v1.0.pdf → file_name: HR_취업규칙_v1.0.pdf

SEC_보안규정_v1.0.docx → file_type: docx

FIN_2025_상반기_매출현황.xlsx → source_path: data/docs/finance/...

3. 청킹 설계 가이드

항목	권장값	이유
기본 크기	500자	한국어 기준 2~3문단. 의미 단위와 대략 일치
오버랩	100자	청크 경계에서 문맥 유지

항목	권장값	이유
최소 크기	100자	너무 짧은 청크는 의미 없음 → 이전 청크에 병합

4. 재인덱싱 운영 가이드

시점	방식	실행
규정 개정 시	전체 재인덱싱	수동 트리거
주기적	전체 재인덱싱	월 1회 (문서 수 적을 때)
문서 추가 시	해당 문서만 추가	기존 인덱스 유지

2.3 더 알아보기

문서 품질 체크리스트 — 벡터 DB에 넣기 전에 확인할 항목입니다. - (1) 현재 유효한 문서인가? - (2) 중복 문서가 없는가? - (3) 텍스트 추출이 가능한가(이미지만 있는 PDF 아닌가)? - (4) 메타데이터가 기록되어 있는가?

한국어 청킹의 특수성 — 영어는 단어 사이에 공백이 있어서 토큰 수 기반 청킹이 자연스럽습니다. 한국어는 띄어쓰기 단위가 영어보다 크고 조사가 붙기 때문에 같은 500자라도 정보 밀도가 다를 수 있어요. CH08에서 의미 기반 청킹(Semantic Chunking)과 비교 실험을 해보겠습니다.

메타데이터 필터링 — 이 프로젝트에서 쓰는 ChromaDB는 저장할 때 메타데이터를 함께 넣을 수 있고 검색할 때 `where={"file_type": "pdf"}`처럼 필터를 걸 수 있습니다. 지금은 메타데이터를 검색 결과의 출처 표시용으로 활용하지만 문서가 많아지면 필터링으로 검색 범위를 좁히는 것도 가능합니다.

Tip: 메타데이터 필터링은 벡터 DB마다 문법이 다릅니다. ChromaDB는 `where={"key": "value"}` 딕셔너리 방식이고 Pinecone은 `filter={"key": {"$eq": "value"}}` 처럼 MongoDB 스타일 연산자를 씁니다. PostgreSQL 기반 pgvector는 아예 SQL `WHERE` 절로 필터링하고요. 문법만 다를 뿐 “메타데이터로 검색 범위를 좁힌다”는 개념은 동일합니다.

2.4 이것만은 기억하세요

- **AI에게 좋은 답을 원하면 좋은 문서를 넣어야 합니다.** 쓰레기를 넣으면 쓰레기가 나옵니다(Garbage In, Garbage Out).
- **PDF/DOCX/XLSX는 먼저 Markdown 텍스트로 통일해야 합니다.** 형식이 다르면 청킹도 검색도 안 돼요.
- **폴더 구조와 파일명이 곧 메타데이터입니다.** 파서가 자동으로 추출하므로 파일명 규칙을 지켜주세요.
- 다음 챗에서는 이 문서를 실제로 파싱하고 청킹해서 벡터 DB에 저장합니다.

Ch.4: “문서를 ‘지식’으로 바꾸다” — VectorDB 구축 (ex04)

한 줄 요약: 문서는 조각내야 찾는다. 손질(파싱), 다지기(청킹), 양념(임베딩), 냉장고(벡터DB).

핵심 개념: 문서 파싱, 청킹, 임베딩, 벡터 저장/검색, 임베딩 모델 선택 기준



요리 준비 — 문서를 지식으로

1.1 “정리는 했는데, 어떻게 넣지?”

CH03에서 사내 문서를 정리했습니다. 분류도 했고 메타데이터 라벨도 붙였죠. 이제 이 문서들을 벡터 DB에 넣어서 AI 비서가 검색할 수 있게 만들 차례입니다.

그런데 문서를 그냥 통째로 넣으면 될까요? CH01에서 이미 경험했습니다. 더미 문서 3개는 괜찮았어요. 하지만 실제 사내 문서는 다릅니다. 취업규칙만 해도 수십 페이지인데, 이것 통째로 넣으면 “연차 몇 일이야?”라고 물었을 때 보안규정이랑 매출 현황까지 달려옵니다. 문서를 **검색 가능한 지식**으로 바꾸려면 몇 단계를 거쳐야 합니다.

1.2 요리를 떠올려보겠습니다

냉장고에 식재료를 넣는 걸 생각해보겠습니다. 마트에서 사온 재료를 봉지째로 냉장고에 던져 넣으면 어떻게 될까요? 나중에 찾을 수가 없습니다. 양파가 어디 있는지, 고기는 아직 쓸 수 있는지. 뒤져봐야 알아요. 제대로 하려면 이런 과정을 거칩니다.

1. **손질한다** — 흠을 씻고, 껍질을 벗기고, 뼈를 발라낸다. 먹을 수 없는 부분을 제거한다.
2. **다진다** — 요리에 맞게 적당한 크기로 자른다. 너무 크면 익지 않고, 너무 작으면 형체가 없어진다.
3. **양념한다** — 소금에 절이거나 밑간을 한다. 나중에 바로 쓸 수 있게 맛을 입힌다.
4. **냉장고에 정리한다** — 라벨을 붙이고, 구분해서 넣는다. “닭가슴살 — 3월 5일 — 볶음용”

사내 문서도 똑같습니다.

요리 과정	문서 처리	무슨 일이 벌어지나
손질	파싱 (Parsing)	PDF/DOCX/XLSX에서 텍스트를 꺼낸다

요리 과정	문서 처리	무슨 일이 벌어지나
다지기	청킹 (Chunking)	텍스트를 적당한 크기로 조각낸다
양념	임베딩 (Embedding)	텍스트 조각을 숫자 벡터로 변환한다
냉장고 정리	벡터 DB 저장	벡터를 ChromaDB에 넣고 검색 가능하게 한다

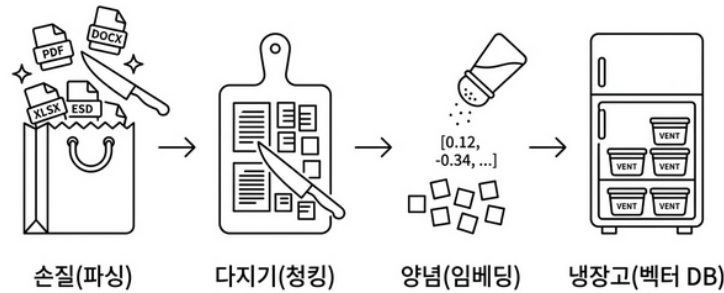


그림 4-1: 문서를 벡터 DB에 넣는 과정은 요리의 손질 → 다지기 → 양념 → 냉장고 정리와 같다.

1.3 1단계: 손질 — 문서에서 텍스트 꺼내기 (파싱)

취업규칙 (다단 편집형) 문서번호: HR-2025-001 버전: v2.0 (Draft) 대외비: (Confidential) 4. 휴가 및 리프레시 (Leave & Refresh) 4.1 스마트 휴가 승인 (Smart Approval) 메타포는 구성원의 자율성을 존중하며, 휴가 사용에 있어 불필요한 절차를 최소화합니다. 이를 위해 'Ask Less, Trust More' 원칙을 기반으로 휴가 제도를 운영하고 있습니다. 연차, 병차, 및 병가는 별도의 관리자 대면 승인 절차를 거치지 않습니다. 구성원은 언제 어디서나 사내 메신저(Slack/Teams)에 등록된 AI HR 봇(Bot)을 통해 평행이로 간편하게 신청할 수 있습니다. 시스템은 달력 간의 일정 충돌 여부를 자동으로 확인한 후, 특별한 경우상 크리터럴 이슈가 있다면 선형 즉시 승인(Instant Approval) 처리됩니다. 승인된 내역은 구글 캘린더 및 팀 공유 캘린더에 자동으로 동기화되어 동료들에게 실시간으로 공유됩니다. 단, 5일 이상의 장기 휴가나 프로젝트 마감 압박 시기(Red Zone)에는 팀 리더에게 사전 알림(Notification)이 발송될 수 있습니다. 이는 승인을 위한 절제 과정이 아닌, 업무 누락 방지와 리소스 조율을 위한 정보 공유 목적입니다. 4.2 병가 및 건강 관리 본인 또는 가족의 질병으로 인해 요양이 필요한 경우, 연간 최대 60일의 유급 병가를 사용할 수 있습니다. 3일 미만: 증명 서류 없이 신청 가능	4.3 워케이션 지원 (Workation) 회사는 직원들이 창의적인 환경에서 업무에 몰입하고 재충전할 수 있도록 다음과 같은 워케이션 프로그램들을 적극 지원합니다. 사무실을 벗어나 새로운 환경에서 일하는 것은 커리어뿐만 아니라 창의력 사고에도 큰 도움이 됩니다. 프로그램 종류 글로벌 노마드 (Global Nomad): 근무 1년 이상 직원은 연 1회 신청 가능하며, 최대 4주간 해외 원하는 도시에서 재유하며 원격으로 근무할 수 있습니다. 사자 비용 및 현지 체류 생계를 위해 해당 기간 동안은 코어 타임(Core Time) 중추 의무가 면제됩니다. 거점 오피스 (Local Hub): 제주, 강원, 부산 등 회사가 제공하는 국내 주요 휴양지의 공유 오피스(WeWork, Local Strich 등)를 연중 첫수 제한 없이 무료로 이용할 수 있습니다. 지원 사항 워케이션 기간 동안 발생하는 숙박비의 50%(연간 최대 100만원 한도)를 회사에서 마우스 생활비 지원합니다. 또한, 현지에서의 원활한 업무 환경 생업을 위해 포터블 모니터, 노트북, 연결용 헤드셋, 5G 와이파이 등 원격 근무 키트(Remote Kit)를 무상으로 대여해 드립니다.
---	--

메타인인 1300 10-partition 10 | 본 문서는 사내 보안 규정의 범위 외부 공유를 금지합니다. | Page 3 of 3

취업규칙 PDF 원본 확인

위 예제로 들어가 있는 취업규칙 PDF 파일을 확인해봅니다. 사람 눈에는 글자가 보이지만 컴퓨터 입장에서는 그냥 바이너리 데이터입니다. “취업규칙 제1조”라는 텍스트를 꺼내려면 **파서(Parser)**가 필요합니다.

문제는 형식마다 파서가 다르다는 겁니다. PDF는 PDF 파서가, DOCX는 DOCX 파서가, XLSX는 XLSX 파서가 필요해요. CH03에서 허용한 형식을 기억하시나요?

형식	파서 라이브러리	특징
PDF	pypdf	텍스트 기반 PDF에서 페이지별 텍스트 추출
DOCX	python-docx	단락(Paragraph)과 표(Table) 추출, 제목을 마크다운으로 변환
XLSX	openpyxl	시트별 셀 데이터를 행 단위로 읽기

그런데 모든 PDF가 잘 읽히는 건 아닙니다. **이미지로 된 PDF** (스캔한 문서나 캡처 화면)는 텍스트가 아예 추출되지 않아요. 이 문제는 CH10에서 OCR과 Vision LLM으로 해결합니다. 지금은 텍스트 기반 문서만 다룹니다.

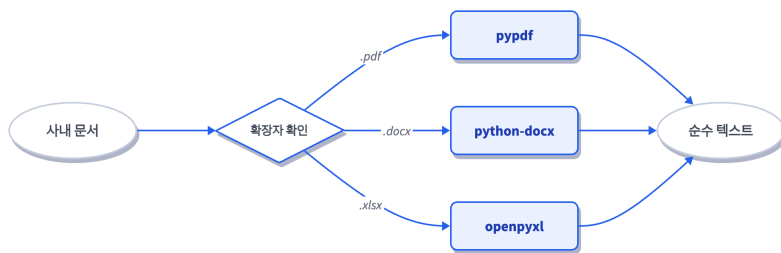


그림 4-2: 파일 확장자에 따라 적절한 파서를 선택한다. 통합 함수 하나로 자동 분기.

실제 우리 프로젝트의 **data/docs/** 폴더. 6개 문서가 있습니다.

```

data/docs/
├── hr/
│   ├── HR_취업규칙_v1.0.pdf
│   └── HR_정보보안서약서.pdf
├── security/
│   └── SEC_보안규정_v1.0.docx
├── finance/
│   ├── FIN_2025_상반기_매출현황.xlsx
│   └── FIN_부서별_예산기안서.xlsx
└── ops/
    └── OPS_신규서비스_런칭전략.pdf
  
```

CH03에서 설계한 분류 구조 그대로입니다. 이 6개 문서를 파싱하면, 순수 텍스트가 추출됩니다.

1.4 2단계: 다지기 — 적당한 크기로 자르기 (칭킹)

텍스트를 꺼냈습니다. 그런데 취업규칙 전문이 한 덩어리로 들어가면 안 됩니다. CH01에서 겪은 그 문제죠 — 문서가 너무 크면 관련 없는 내용까지 딸려옵니다.

요리에서 재료를 다지듯이 텍스트를 **적당한 크기의 조각(chunk)**으로 잘라야 합니다.

CH03에서 설계한 대로 **고정 크기 500자 + 100자 오버랩**으로 갑니다.

왜 오버랩이 필요할까요? “신입사원은 첫 3년간 연차가 없다” 다음에 “대신 리프레시 데이를 제공한다”가 이어지는데, 정확히 500자에서 잘리면 “연차가 없다”만 남고 뒤의 대안 정보가 사라져요. 오버랩 100자가 이 문제를 줄여줍니다.

원본 텍스트: [—————]

청크 1: [————— 500자 —————]

청크 2: [————— 500자 —————]

청크 3: [————— 500자 —————]

↑ 100자 겹침 ↑ ↑ 100자 겹침 ↑

각 조각에는 **메타데이터**도 함께 붙입니다. 출처 파일명, 페이지 번호, 분류 — CH03에서 설계한 라벨이 여기서 쓰이는 거예요. 나중에 AI 비서가 “이 답변의 출처는 취업규칙 3페이지입니다”라고 말할 수 있게 됩니다.

1.5 3단계: 양념 — 의미를 숫자로 바꾸기 (임베딩)

여기가 마법이 일어나는 곳입니다. “연차 사용 규정”이라는 텍스트를 컴퓨터가 이해할 수 있을까요? 컴퓨터는 글자를 모릅니다. 숫자만 알아요. **임베딩(Embedding)**은 텍스트의 **의미**를 숫자 벡터(768개의 숫자 리스트)로 바꾸는 과정입니다. 여기서 핵심은 단순히 글자를 숫자로 바꾸는 게 아니라 **의미가 비슷한 텍스트는 비슷한 숫자**가 된다는 점입니다.

“연차 사용 규정” → [0.12, -0.34, 0.87, ...] “휴가 관련 정책” → [0.11, -0.33, 0.85, ...] ← 비슷! “매출 현황 보고서” → [-0.45, 0.22, -0.11, ...] ← 완전 다름

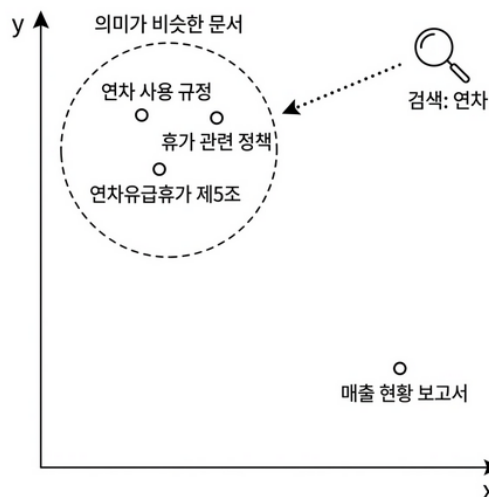


그림 4-3: 임베딩은 의미가 비슷한 텍스트를 가까운 좌표에 배치한다. “연차”를 검색하면 의미상 가까운 문서들이 먼저 발견된다.

벡터 검색의 핵심이 바로 이겁니다. “연차 사용 규정”을 검색하면 숫자가 비슷한 “휴가 관련 정책” 청크를 찾아올 수 있습니다. 키워드가 정확히 일치하지 않아도 의미가 가까우면 찾아내요. 그런데 “숫자가 비슷하다”는 걸 어떻게 판단할까요? 768개나 되는 숫자를 하나하나 비교할 수는 없습니다. 여기서 **코사인 유사도(Cosine Similarity)** 라는 방법을 씁니다. 벡터를 좌표 위의 **화살표** 라고 생각해 보세요. 두 화살표가 같은 방향을 가리키면 의미가 비슷한 거고, 반대 방향이면 의미가 다른 겁니다. 코사인 유사도는 이 **두 화살표 사이의 각도**를 측정합니다.

- 같은 방향(각도 0°) → 유사도 **1** (완전히 같은 의미)
- 직각(90°) → 유사도 **0** (관련 없음)
- 반대 방향(180°) → 유사도 **-1** (반대 의미)



그림 4-4: 코사인 유사도의 각도에 따른 값의 변화

“연차 사용 규정”과 “휴가 관련 정책”은 거의 같은 방향을 가리킵니다. “매출 현황 보고서”는 완전히 다른 방향이고요. 벡터 DB는 이 각도를 계산해서 가장 비슷한 방향의 문서부터 순서대로 가져옵니다.

왜 ko-sroberta-multitask인가?

임베딩 모델은 여러 가지가 있습니다. OpenAI의 text-embedding-ada-002도 있고 다국어 모델도 있어요. 우리는 **ko-sroberta-multitask** 를 선택했습니다. 이유는 간단합니다.

1. **한국어에 특화됐다** — 한국어 문장 유사도 태스크로 파인튜닝된 모델입니다. “연차”와 “휴가”가 의미상 가깝다는 걸 잘 잡아냅니다.
2. **로컬에서 돌릴 수 있다** — OpenAI 임베딩은 API 호출마다 비용이 듭니다. ko-sroberta는 한 번 다운로드하면 로컬에서 무료로 쓸 수 있어요.
3. **사내 문서에 적합하다** — 사내 정보를 외부 API에 보내지 않아도 됩니다. 보안 관점에서도 안전하고요.

CH08(검색 품질 튜닝)에서 다른 임베딩 모델과 비교 실험을 해봅니다. 지금은 ko-sroberta로 시작하고, 나중에 더 나은 선택지가 있는지 확인합니다.

1.6 4단계: 냉장고 정리 — 벡터 DB에 저장

양념까지 끝난 재료를 냉장고에 정리합니다. 라벨 붙이고 구분해서 나중에 바로 꺼낼 수 있게.

ChromaDB 가 우리의 냉장고입니다. CH01에서 이미 써봤지만 그때는 `Chroma.from_documents()` 한 줄로 끝냈죠. 이번에는 직접 넣습니다. ChromaDB에 저장하는 항목은 네 가지입니다.

저장 항목	내용	예시
id	청크 고유 ID	hr_취업규칙_v1_0_text_p001_c0003

저장 항목	내용	예시
document	청크 텍스트 원문	“제5조 연차유급휴가는…”
embedding	768차원 벡터	[0.12, -0.34, …]
metadata	출처 정보	{file_name: “HR_취업규칙_v1.0.pdf”, page: 3}

저장할 때 **upsert** 를 사용합니다. 같은 ID가 이미 있으면 덮어쓰고 없으면 새로 추가해요. 파이프라인을 여러 번 실행해도 데이터가 중복되지 않습니다. CH03에서 설계한 **전체 재인덱싱** 전략과 맞닿는 부분이죠.

손질(파싱) → 다지기(청킹) → 양념(임베딩) → 냉장고 정리(벡터 DB 저장). 네 단계를 알았습니다. 기술 파트에서 직접 파이프라인을 돌려보고 “연차 사용 규정”이 정말 검색되는지 확인하겠습니다.

이제 실습으로 문서를 벡터 DB에 저장하는 파이프라인을 만들어보겠습니다.

2.1 용어 정리

이야기 속 표현	진짜 용어	정식 정의
“손질”	파싱 (Parsing)	파일 형식(PDF/DOCX/XLSX)에서 순수 텍스트를 추출하는 과정
“다지기”	청킹 (Chunking)	긴 텍스트를 벡터 DB에 적합한 크기로 분할하는 과정. 고정 크기 500자 + 100자 오버랩
“양념”	임베딩 (Embedding)	텍스트의 의미를 768차원 숫자 벡터로 변환하는 과정
“냉장고”	벡터 DB (Vector Database)	벡터를 저장하고, 유사한 벡터를 빠르게 검색하는 특수 데이터베이스
“의미가 비슷한 숫자”	코사인 유사도 (Cosine Similarity)	두 벡터 사이의 각도로 유사도를 측정. 1에 가까울수록 유사
“덮어쓰기”	업서트 (Upsert)	같은 ID가 있으면 업데이트, 없으면 삽입하는 연산

2.2 실습 환경 구축

기본 환경(Python 3.12)이 없다면 **부록(환경 설정)** 을 먼저 참고하세요.

```
cd ex04
cp .env.example .env
python3.12 -m venv .venv
```

```
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

패키지	버전	역할
pypdf	4.3.1	PDF 텍스트 추출
python-docx	1.1.2	DOCX 단락/표 추출
openpyxl	3.1.5	XLSX 셀 데이터 추출
sentence-transformers	3.3.1	ko-sroberta 임베딩 모델
chromadb	1.5.1	벡터 DB (로컬 영속)

ko-sroberta-multitask 모델은 최초 실행 시 HuggingFace에서 자동 다운로드됩니다 (약 400MB). 이후에는 로컬 캐시를 사용합니다.

2.3 파일 계층 구조

```
ex04/
├── requirements.txt
├── data/
│   ├── docs/                ← 사내 문서 원본
│   │   ├── hr/              (PDF 2개)
│   │   ├── security/       (DOCX 1개)
│   │   ├── finance/        (XLSX 2개)
│   │   └── ops/             (PDF 1개)
│   ├── markdown/           ← 파싱 결과 (마크다운 변환)
│   └── chroma_db/           ← ChromaDB 영속 저장소
├── src/
│   ├── main.py              [실습] 파이프라인 실행 진입점 ◀ 먼저 돌려보세요!
│   ├── cli_search.py        [실습] 벡터 검색 CLI 도구 ◀ 검색 결과 확인!
│   ├── extractor.py         [설명] 형식별 텍스트 추출 통합 모듈
│   ├── chunker.py           [설명] Fixed-size 청킹 알고리즘 ◀ CH04 핵심
│   ├── store.py             [설명] ko-sroberta 임베딩 + ChromaDB 저장/검색
│   ├── extract_pdf.py       [참고] PDF 파싱 → Markdown 변환
│   ├── extract_docx.py      [참고] DOCX 파싱 → Markdown 변환
│   └── extract_xlsx.py      [참고] XLSX 파싱 → Markdown 변환
```

2.4 실습 순서

1. `main.py --step 1` — 파싱 + 마크다운 저장
2. `data/markdown/` — 결과 확인
3. `main.py` — 전체 파이프라인 실행
4. `cli_search.py` — 검색 테스트

파싱 결과가 마크다운으로 저장되면 눈으로 확인하고(step1 → markdown), 전체 파이프라인을 돌린 뒤(main.py), 검색이 되는지 직접 테스트합니다(cli_search.py). **파싱 결과 확인을 건너뛰지 마세요.**

2.5 실습 1: 파싱부터 확인합니다 (main.py --step 1)

`main.py` 는 이야기 파트에서 설명한 손질(파싱) → 다지기(청킹) → 양념(임베딩) → 냉장고 정리(저장) 네 단계를 한 번에 실행하는 진입점입니다. 하지만 **전체를 한 번에 돌리기 전에 파싱만 먼저** 확인합니다.

```
# 실행
python src/main.py --step 1
```

`--step 1` 은 손질(파싱)과 마크다운 변환을 실행합니다. 파싱이 끝나면 `data/markdown/` 폴더에 마크다운 파일이 생겨요. 이 파일을 열어보는 것이 핵심입니다. **파싱이 잘 됐는지 눈으로 확인하는 과정** — 이 단계를 건너뛰면 안 됩니다.

`data/markdown/HR_취업규칙_v1.0.md` 를 열어보겠습니다.

```
# HR_취업규칙_v1.0.pdf

- 파일 형식: pdf
- 추출 글자 수: 1916자

---

## 페이지 1

취업규칙 ( 다 단 편 집 형 )문서번호 : HR-2026-001
버전: v2.0 (Draft)
대외비 (Confidential)
4. 휴가 및 리 프 레 시 (Leave & Refresh)
4.1 스마트 휴 가 승 인 (Smart Approval)
메타코딩는 구성원의 자 율 성을 존 중 하 며 , 휴 가 사 용에
있어 불필요한 절 차 를 최 소화합니 다 . 이 를 위해 'Ask-
Less, Trust-More' 원 칙 을 기 반 으 로 휴 가 제도를
운영하고 있습니 다 .

...
```

파일 정보(파일명, 파싱 형식, 글자 수)가 상단에 요약되어 있고 **## 페이지 1** 처럼 페이지 단위로 텍스트가 잘 추출된 걸 볼 수 있습니다. 이번에는 XLSX 파일을 확인해 보겠습니다. `data/markdown/FIN_부서별_예산기안서.md` 를 열어봅니다.

```
# FIN_부서별_예산기안서.xlsx
```

- 파일 형식: `xlsx`
- 추출 글자 수: 833자

```
---
```

```
[시트: 예산기안서_최종]
```

```
| 2025년도 주요사업부 예산 집행 기안서 | | | | | | | |
| --- | --- | --- | --- | --- | --- | --- | --- |
| 문서번호: | FIN-BDG-2025-001 | 결
```

```
재 | 기안 | 검토 | 승인 | | |
| 기안부서: | 재무전략실 | | | | | | |
| 작성일자: | 2025. 02. 20 | | | | | |
| # | 본부명 | 부서명 | 예산 계정 | Q1 배정액(원) | Q2 배정액(원) | 상반기 총액 | 집행 사유 (비고) |
| 1 | 경영지원본부 | 인사총무팀 | 복리후생비 | 25000000 | 30000000 | 전사 총계 워크샵 비용 포함 |
| 2 | 경영지원본부 | 재무전략실 | 지급수수료 | 12000000 |
...
```

마크다운 표 형식을 확인할 수 있습니다. 표의 생김새가 일반 표보다 비뚤뻔하고 다소 끊어진 것처럼 보일 수 있어요. 하지만 **벡터 DB(임베딩 모델) 입장에서는 오히려 이 방식이 엑셀 원본보다 훨씬 이해하기 좋습니다**. 헤더 행과 구분선(---)이 존재하고 행/열의 문맥이 텍스트 흐름(파이프 | 기호 등)으로 유지되기 때문에 “경영지원본부 예산”을 검색하면 그 표의 데이터가 정확하게 검색됩니다. > 파싱은 RAG의 첫 단추입니다. 여기서 텍스트가 깨지면 뒤에서 아무리 고쳐도 소용이 없어요. RAG에서는 쓰레기가 들어가면 쓰레기가 나온다는 **Garbage In, Garbage Out** 격언이 있을 정도입니다. `data/markdown/` 파일을 반드시 열어보고 원본 문서와 비교해 보세요.

이 책에서는 `pypdf`, `python-docx`, `openpyxl` 라이브러리를 사용하므로 파서 코드를 직접 따라 치지는 않습니다. 하지만 실무에서는 파싱이 까다로운 경우가 훨씬 많아요. 2단 열 레이아웃, 투명 표, 표 안의 표, 헤더가 페이지를 넘어가며 끊기는 표, 한 페이지에 표가 여러 개인 경우 등 온갖 상황이 있습니다. 이런 복잡한 레이아웃을 처리하는 전용 라이브러리(`pdfplumber`, `camelot`, `unstructured` 등)도 있고 이미지를 인식하는 Vision LLM을 활용하는 방법도 있습니다(CH10에서 다룹니다). 여러분의 사내 문서로 직접 파싱해 보고 결과를 눈으로 확인하는 습관을 들이세요.

2.6 실습 2: 전체 파이프라인 실행 (main.py)

파싱 결과가 깨끗한 걸 확인했으면 이제 **전체 파이프라인을 돌립니다**.

```
# 실행
python src/main.py

# 체크 크기를 바꿔보고 싶다면
python src/main.py --chunk-size 300 --overlap 50
```

`--chunk-size` 옵션으로 청크 크기를 바꿀 수 있습니다. 아무 옵션 없이 실행하면 기본값(500자, 오버랩 100자)으로 동작해요. 내부적으로는 두 단계를 순서대로 호출합니다.

1. **Step 1: 손질(파싱) + 마크다운 변환** — `data/docs/` 의 6개 문서에서 텍스트를 추출하고 마크다운으로 변환하여 `data/markdown/` 에 저장합니다
2. **Step 2: 다지기 + 양념 + 저장** — 마크다운 텍스트를 청크로 자르고 임베딩한 뒤 ChromaDB에 넣습니다



그림 4-5: 6개 사내 문서가 마크다운으로 변환된 뒤, 18개 청크로 잘려 벡터 DB에 저장됐다.

2.7 실습 3: 벡터 검색 CLI 도구 (cli_search.py)

파이프라인을 돌렸으면 이제 **검색**이 되는지 직접 확인합니다.

```
# 실행: 단일 쿼리
python src/cli_search.py --query "휴가 규정" --top-k 3
```

```
# 실행: 대화형 모드 (반복 검색)
python src/cli_search.py
```

검색 결과의 유사도는 어떻게 계산될까요? ChromaDB는 코사인 **거리** (0~2)를 반환합니다. 직관적이지 않으므로 **유사도** (0~100%)로 변환합니다.

```
# cli_search.py - 유사도 변환

def format_distance_as_similarity(distance: float) -> float:
    """코사인 거리를 백분율 유사도로 변환합니다."""
    return max(0.0, (1.0 - distance / 2.0)) * 100
```

거리가 0이면 유사도 100%(완전 일치), 거리가 2이면 유사도 0%(완전 반대)입니다. “휴가 규정”을 검색했을 때 유사도 71%가 나왔다면 코사인 거리로는 약 0.58이라는 뜻이에요.

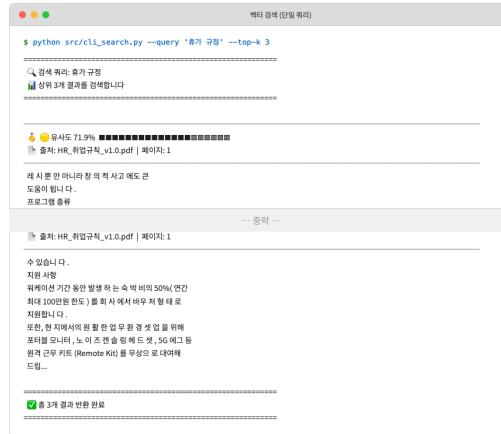


그림 4-6: “휴가 규정”을 검색하자 취업규칙에서 글로벌 노마드, 스마트 휴가 승인 등의 관련 내용을 찾아냈다.

1~3위 모두 취업규칙에서 휴가(글로벌 노마드, 스마트 휴가, 워케이션) 관련 내용을 찾아왔습니다. 모델이 문맥을 잘 파악해서 정답을 찾아낸 거예요. 다만 유사도가 71%대로 아주 높지는 않고 **--top-k 5**처럼 검색수를 늘리면 전혀 관계없는 문서가 억지로 달려올 수도 있습니다. 벡터 DB는 키워드 매칭이 아니라 ‘그나마 의미가 가까운 순서’대로 k개를 무조건 채워서 가져오기 때문입니다. 검색 품질을 높이는 심화 기법은 CH08에서 집중적으로 다룹니다.

참고: 검색 결과는 다를 수 있습니다 임베딩 모델의 버전이나 PC 환경 등에 따라 검색된 청크의 순서나 유사도 수치(%)는 책의 예시와 조금 다르게 나올 수 있습니다. 최상위(Top 1~3) 검색 결과에 취업규칙의 휴가/연차 관련 내용이 잘 나왔다면 정상적으로 벡터 DB가 구축된 것입니다.

2.8 [설명] extractor.py — 형식별 텍스트 추출

파일 확장자를 보고 적절한 파서를 자동 선택하는 통합 모듈입니다. 핵심은 `extract_text()` 함수입니다. 확장자를 보고 파서 함수를 고르는 매핑 딕셔너리를 씁니다.

```
# extractor.py — 핵심 구조

def extract_text(file_path: str | Path) -> dict:
    """파일 형식을 자동 감지하여 텍스트를 추출하는 통합 함수."""
    file_path = Path(file_path)
    suffix = file_path.suffix.lower()

    extractor_map = {
        ".pdf": extract_from_pdf,
        ".docx": extract_from_docx,
        ".xlsx": extract_from_xlsx,
    }
```



```

if suffix not in extractor_map:
    raise ValueError(f"지원하지 않는 파일 형식입니다: '{suffix}'")

return extractor_map[suffix](file_path)

```

패턴: 전략 패턴(Strategy Pattern) `extractor_map` 딕셔너리가 확장자별로 다른 함수를 선택합니다. if-elif-else 분기 대신 딕셔너리 매핑을 사용하면 새 형식을 추가할 때 한 줄만 넣으면 돼요.

PDF 파서를 예로 보겠습니다. 핵심은 `extract_text()` or `""` 한 줄입니다.

```

# extractor.py — PDF 파서

def extract_from_pdf(file_path: str | Path) -> dict:
    """PDF 파일에서 텍스트를 페이지별로 추출합니다."""
    file_path = Path(file_path)

    pages_data = []
    with open(file_path, "rb") as f:
        reader = pypdf.PdfReader(f)
        for page_num, page in enumerate(reader.pages, start=1):
            page_text = page.extract_text() or ""
            pages_data.append({"page": page_num, "text": page_text.strip()})

    full_text = "\n\n".join(p["text"] for p in pages_data if p["text"])
    return {
        "source_path": str(file_path.resolve()),
        "file_name": file_path.name,
        "file_type": "pdf",
        "pages": pages_data,
        "full_text": full_text,
    }

```

`page.extract_text()` 가 `None` 을 반환하는 경우가 있습니다. **이미지로 된 PDF** — 스캔 문서나 캡처 화면 — 는 텍스트 레이어가 없어서 pypdf가 아무것도 꺼내지 못해요. or `""` 방어 코드 없이는 `None.strip()` 에서 바로 예러가 납니다. 실제로 예제 파일 중 ‘HR_정보보안서약서.pdf’가 이미지 기반 PDF입니다. 이 파일은 텍스트가 하나도 추출되지 않아 파싱에서 텍스트 청크가 0개인 걸 **그림 4-4** 에서 확인할 수 있습니다.

이 문제는 CH10에서 Vision LLM을 사용해 해결합니다. 지금은 “텍스트 기반 PDF만 파싱된다”는 한계만 알고 넘어갑니다.

형식별 파서 함수는 각각 다른 라이브러리를 사용하지만 모두 같은 구조의 딕셔너리를 반환합니다.

함수	라이브러리	핵심 동작	주의점
<code>extract_from_pdf()</code>	pypdf	페이지별 <code>extract_text()</code> 호출	이미지 기반 PDF는 빈 문자열 반환 (CH10에서 해결)

함수	라이브러리	핵심 동작	주의점
<code>extract_from_docx()</code>	python-docx	단락 순회 + Heading → 마크다운 헤더 변환	페이지 개념 없음 (전체가 1페이지)
<code>extract_from_xlsx()</code>	openpyxl	시트별 행 단위 읽기 → 마크다운 표 변환	첫 행을 헤더로 사용

반환 구조 — 세 파서 모두 같은 형태입니다:

```
{
    "source_path": "절대 경로",
    "file_name": "파일명",
    "file_type": "pdf | docx | xlsx",
    "pages": [{"page": 번호, "text": "텍스트"}], ...],
    "full_text": "전체 텍스트"
}
```

2.9 [설명] chunker.py — 청킹 알고리즘

추출된 텍스트를 500자 단위로 자릅니다. 이 함수가 CH04의 핵심입니다. 청킹이 RAG 검색 품질을 좌우하거든요.

```
# chunker.py - 텍스트 분할

DEFAULT_CHUNK_SIZE = 500
DEFAULT_OVERLAP = 100

def split_text_into_chunks(
    text: str,
    chunk_size: int = DEFAULT_CHUNK_SIZE,
    overlap: int = DEFAULT_OVERLAP,
) -> list[str]:
    """텍스트를 Fixed-size 방식으로 청크 리스트로 분할합니다."""
    text = text.strip()
    if not text:
        return []

    chunks = []
    step = chunk_size - overlap # 다음 청크 시작 위치 이동 단계
    start = 0

    while start < len(text):
        end = start + chunk_size
```

```

    chunk = text[start:end].strip()
    if chunk:
        chunks.append(chunk)
    start += step

return chunks

```

`step = chunk_size - overlap` 한 줄이 핵심입니다. 500자 청크에서 100자 오버랩이면 다음 청크는 400자 뒤에서 시작해요. 이전 청크의 마지막 100자가 다음 청크의 처음 100자와 겹칩니다.

청크 크기와 오버랩을 바꾸면 검색 결과가 달라집니다. `--chunk-size 300 --overlap 50` 으로 다시 돌려보고, 검색 결과가 어떻게 바뀌는지 직접 비교해 보세요. CH08(검색 품질 튜닝)에서 이 실험을 본격적으로 다룹니다.

각 청크에는 출처 추적용 **메타데이터** 가 함께 붙습니다.

필드	예시	용도
id	hr_취업규칙_v1_0_text_p003_c0005	ChromaDB upsert 키
file_name	HR_취업규칙_v1.0.pdf	출처 파일명
page	3	출처 페이지
chunk_index	5	문서 내 순번
chunk_type	text	텍스트 / 이미지 캡션 구분

청크 ID는 문서ID_text_p페이지_s순번 형식입니다. 이 ID 덕분에 같은 파이프라인을 여러 번 실행해도 데이터가 중복되지 않아요 (upsert). `chunker.py` 의 나머지 함수들입니다.

함수	역할	비고
<code>make_doc_id(file_name)</code>	파일명에서 문서 고유 ID 생성. 공백/특수문자를 _로 치환	청크 ID의 접두사로 사용
<code>chunk_extract_result(extract_result)</code>	<code>extract.py</code> 의 추출 결과 1건을 청크 리스트로 변환	페이지별로 <code>split_text_into_chunks()</code> 호출
<code>chunk_all_documents(extract_result)</code>	여러 문서의 추출 결과를 일괄 청킹	<code>main.py</code> 에서 호출하는 진입점

2.10 [설명] store.py — 임베딩 + ChromaDB 저장

이 모듈이 “양념 + 냉장고 정리”를 담당합니다. `SentenceTransformer("jhgan/ko-sroberta-multitask")` 로 임베딩 모델을 로드하고(최초 실행 시 약 400MB 다운로드, 이후 캐시 재사용) 청크를 배치 단위로 임베딩한 뒤 ChromaDB에 저장합니다. 배치 임베딩 — `embed_chunks()`:

```
# store.py - 배치 임베딩

BATCH_SIZE = 64

def embed_chunks(chunks, model):
    """청크 리스트를 배치 단위로 임베딩합니다."""
    ids = [c["id"] for c in chunks]
    documents = [c["text"] for c in chunks]

    # 메타데이터에서 ChromaDB 허용 타입이 아닌 값 변환 (None → "")
    metadatas = []
    for c in chunks:
        meta = {}
        for k, v in c["metadata"].items():
            if v is None:
                meta[k] = ""
            elif isinstance(v, bool):
                meta[k] = str(v)
            else:
                meta[k] = v
        metadatas.append(meta)

    # 배치 단위 임베딩 계산
    all_embeddings = []
    for batch_start in range(0, len(documents), BATCH_SIZE):
        batch_texts = documents[batch_start : batch_start + BATCH_SIZE]
        batch_embeddings = model.encode(
            batch_texts, show_progress_bar=False, normalize_embeddings=True
        )
        all_embeddings.extend(batch_embeddings.tolist())

    return ids, documents, all_embeddings, metadatas
```

여기서 주의할 점이 두 가지 있습니다. 1. **메타데이터 정제** — ChromaDB는 메타데이터에 `None` 이나 `bool` 을 허용하지 않습니다. `None` → 빈 문자열, `bool` → 문자열로 변환해야 해요. 빠뜨리면 저장할 때 에러가 납니다. 2. `normalize_embeddings=True` — 벡터를 단위 벡터로 정규화합니다. 코사인 유사도 계산에 필요하거든요.

ChromaDB 저장 — `store_chunks_to_chroma()`:

```
# store.py - ChromaDB 저장

def store_chunks_to_chroma(chunks, chroma_dir, collection_name, ...):
    """청크를 임베딩하여 ChromaDB에 저장합니다."""
    # === PROCESS: Step 1 - 임베딩 모델 로드 ===
    model = load_embedding_model(embedding_model_name)

    # === PROCESS: Step 2 - ChromaDB 초기화 ===
    client = chromadb.PersistentClient(
        path=chroma_dir,
        settings=Settings(anonymized_telemetry=False),
    )
    collection = client.get_or_create_collection(
        name=collection_name,
        metadata={"hsw:space": "cosine"},
    )

    # === PROCESS: Step 3 - 임베딩 계산 ===
    ids, documents, embeddings, metadatas = embed_chunks(chunks, model)

    # === PROCESS: Step 4 - ChromaDB에 배치 업서트 ===
    for batch_start in range(0, len(ids), BATCH_SIZE):
        batch_end = batch_start + BATCH_SIZE
        collection.upsert(
            ids=ids[batch_start:batch_end],
            documents=documents[batch_start:batch_end],
            embeddings=embeddings[batch_start:batch_end],
            metadatas=metadatas[batch_start:batch_end],
        )
```

`PersistentClient` 가 핵심입니다. 메모리가 아닌 디스크(`data/chroma_db/`)에 저장하므로 프로그램을 종료해도 데이터가 남아 있어요. `hsw:space: "cosine"` 으로 코사인 유사도를 사용합니다.

2.11 더 알아보기

마크다운 변환 — 파이프라인은 추출한 텍스트를 마크다운 형식으로 변환하여 `data/markdown/`에 저장한 뒤 임베딩합니다. 임베딩 모델도 마크다운 구조(제목 #, 표 |, 리스트 ~)를 학습했기 때문에 구조가 잘 보존된 텍스트일수록 의미를 더 정확하게 벡터로 변환해요. `extract_pdf.py`, `extract_docx.py`, `extract_xlsx.py`를 단독으로 실행하면 형식별로 개별 변환할 수도 있습니다.

배치 크기 튜닝 — `BATCH_SIZE = 64`가 기본값입니다. GPU 메모리가 부족하면 줄이고 충분하면 늘릴 수 있어요. ko-sroberta는 CPU에서도 동작하지만 GPU가 있으면 임베딩 속도가 크게 빨라집니다.

2.12 이것만은 기억하세요

- 문서는 조각내야 찾습니다. 손질(파싱) → 다지기(청킹) → 양념(임베딩) → 냉장고 정리(벡터 DB). 이 네 단계가 RAG의 기초 체력입니다.

- 임베딩은 “의미를 숫자로 바꾸는 것”입니다. 키워드가 달라도 의미가 비슷하면 가까운 벡터가 돼요.
- 다음 챗에서는 이 벡터 DB를 활용해서 **진짜 질문-답변 엔진(RAG Q&A)** 을 만듭니다. “연차 몇 일이야?”라고 물으면 검색한 문서를 근거로 LLM이 자연어로 답변해주는 시스템입니다.

Ch.5: “드디어 답해준다” — RAG Q&A 엔진 (ex05)

한 줄 요약: 검색은 재료, 답변은 요리다. LCEL 파이프라인이 이 레시피다.

핵심 개념: LCEL 파이프라인, 출처 강제(Source Grounding), WindowMemory 멀티턴



사서가 답해준다

지난 챗터에서 사내 문서를 벡터 DB에 저장하고 CLI로 검색까지 해봤습니다. “연차 사용 규정”을 검색하면 관련 문서 조각이 유사도 점수와 함께 나왔죠. 그런데 문제가 생겼습니다.

1.1 “그냥 답을 알려줘”

벡터 검색을 사내 시스템에 붙이고 일주일쯤 지났을 때였습니다.

동료: “야, 병가 쓸 때 증빙 서류 필요해?”

“연차 몇 일 남았는지 어떻게 확인해?”

“신규 서비스 런칭 전략 문서 어디 있어?”

동료들의 질문이 하루에 서너 번씩 날아왔습니다. 매번 같은 유형이에요. 답은 전부 사내 문서 어딘가에 있는데 말이죠.

처음엔 벡터 검색 결과를 공유해봤습니다.

[1] HR_취업규칙_v1.0 (p.3) - 유사도: 87.2%

“제15조(병가) 질병이나 부상으로 인하여 직무를 수행할 수 없을 때에는...”

[2] HR_취업규칙_v1.0 (p.4) - 유사도: 81.5%

“병가 기간이 3일 이상인 경우에는 의사의 진단서를...”

돌아오는 반응은 한결같았어요.

동료: “이걸 내가 읽어?”

“그냥 답을 알려줘.”

맞다. 사람들은 문서 조각을 원하는 게 아니라, **답변**을 원한다.

검색 결과 5개를 받아서 직접 읽고 “아, 3일 미만이면 증빙 불필요고 3일 이상이면 진단서가 필요하구나”라고 해석하는 건 결국 사람 몫이었습니다. 벡터 검색은 **재료를 찾아주는 것이지 요리를 해주는 것**이 아니었어요. 이번 챗터에서는 그 “요리”를 해줄 **RAG Q&A 엔진**을 만들겠습니다. 질문하면 문서를 검색하고 검색 결과를 읽어서 자연어로 답변해주는 시스템이에요.

1.2 도서관 사서가 필요하다

지금까지 만든 벡터 검색은 **도서관의 검색 시스템**과 비슷합니다. “한국 역사”를 검색하면 관련 책이 어디 있는지 알려주죠. 하지만 그 책을 직접 꺼내서 읽어보고 핵심을 정리해서 답해주시는 않습니다.

우리에게 필요한 건 **사서**입니다. 사서에게 “조선시대 과거 제도가 뭐야?”라고 물으면 이런 일이 벌어집니다.

1. **질문을 듣는다** — “조선 과거 제도”가 핵심이군
2. **서가에서 책을 찾는다** — 한국사 개론 3장 부근을 꺼냄
3. **읽고 답변을 정리한다** — “문과·무과·잡과 세 종류가 있었습니다”
4. **출처를 알려준다** — “한국사 개론 3장에 나와 있어요”

이 네 단계가 바로 RAG Q&A 파이프라인입니다.



그림 5-1: RAG Q&A 흐름 — 질문이 들어오면 벡터 검색으로 문서 조각을 찾고, LLM이 읽어서 자연어 답변을 만든다

벡터 검색(CH04)은 2번까지였습니다. 이번 챕터에서 3번과 4번을 추가해요. 사서가 책을 찾는 것뿐 아니라 **읽고 답변까지 해주는** 시스템을 만드는 겁니다.

1.3 파이프라인이라는 이름의 레시피

사서가 일하는 순서를 코드로 옮기려면 어떻게 해야 할까요? 검색 → 프롬프트 → LLM → 파싱, 각 단계를 직접 이어 붙여도 되지만 이걸 편하게 해주는 도구가 있습니다. **LangChain** 입니다. 검색, 프롬프트, LLM 호출 같은 단계를 부품으로 제공하고 이 부품을 조립할 수 있게 해주는 Python 프레임워크예요. RAG 시스템을 만들 때 가장 널리 쓰입니다. LangChain은 이 조립을 **파이프(()) 연산자**로 합니다. LCEL(LangChain Expression Language)이라고 부르죠. 파이프는 주방의 레시피와 같습니다.

질문 → **검색**(벡터 DB에서 문서 조각 찾기) → **프롬프트**(찾은 문서 + 질문을 합치기) → **LLM**(읽고 답변 생성) → **파싱**(답변 텍스트 추출)

각 단계가 파이프(())로 연결되고, 앞 단계의 출력이 다음 단계의 입력이 됩니다. 요리 레시피 순서와 똑같아요.



그림 5-2: LCEL 파이프라인 구조 — 각 단계가 파이프 연산자로 연결된다

이 구조의 장점은 **블록을 바꿀 수 있다**는 겁니다. LLM을 DeepSeek에서 GPT-4o로 바꾸고 싶으면? LLM 블록만 교체하면 돼요. 검색 방식을 바꾸고 싶으면? 검색 블록만 교체하면 됩니다. CH08~10에서 튜닝할 때 이 구조가 빛을 발하기 시작해요.

1.4 “출처를 대”

사서에게 한 가지 더 요구할 게 있습니다. **출처**입니다.

CH01에서 LLM이 그럴듯하게 거짓말하는 걸 봤습니다. RAG로 문서를 넣어줬다고 환각이 완전히 사라지는 건 아니예요. LLM은 문서에 없는 내용을 지어내기도 합니다. 그래서 프롬프트에 규칙을 넣습니다.

“반드시 제공된 문서에서만 답하세요. 답변 마지막에 출처를 명시하세요. 문서에서 찾을 수 없으면 ‘확인되지 않습니다’라고 답하세요.”

이걸 **출처 강제(Source Grounding)** 라고 부릅니다. LLM에게 “근거 없이 답하지 마”라고 제한을 거는 거예요. 출처가 붙으면 독자가 직접 확인할 수도 있고, 신뢰도가 훨씬 올라갑니다.

1.5 사서도 기억력이 있다

도서관에서 사서에게 묻습니다.

방문자: “한국 역사 관련 책 어디 있어요?”

사서: “2층 인문학 서가에 있습니다. ‘한국사 편지’가 가장 인기 있어요.”

바로 이어서 묻습니다.

방문자: “그러면 거기에 세계사 책도 있어?”

여기서 “거기”가 가리키는 건 뭘까요? **2층 인문학 서가**입니다. 이전 대화를 기억하고 있어야 “거기 = 2층 인문학 서가”라는 맥락을 이해할 수 있어요.

사람 사서라면 당연히 기억합니다. 하지만 LLM은 기본적으로 **기억력이 없습니다**. 매 요청이 독립적이라 이전에 뭘 물어봤는지 모릅니다. 그래서 **대화 히스토리**를 직접 관리해야 합니다. 이전 대화를 메모해뒀다가 새 질문이 올 때마다 같이 넘겨주는 거예요. 다만 모든 대화를 다 기억할 수는 없으니 **최근 5장만 유지**하는 슬라이딩 윈도우 방식을 씁니다.

도서관 사서가 메모장을 들고 있다고 생각하면 됩니다. 새 메모가 들어오면 가장 오래된 메모를 지우고 항상 최근 5장만 남겨요.

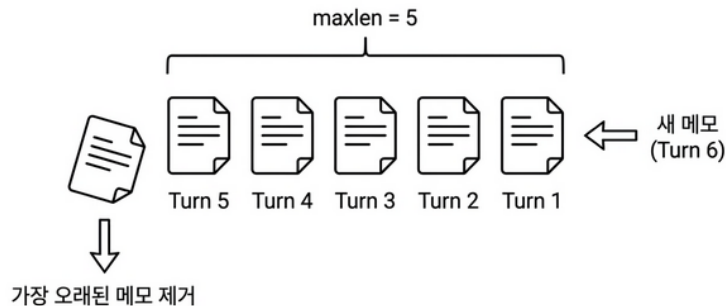


그림 5-3: 메모장에 6번째 메모가 들어오면 가장 오래된 1번째가 빠진다. 항상 최근 5장만 유지.

1.6 이번 버전에서 뭘 만드나

ex05에서는 네 가지를 추가합니다.

기능	비유	코드
LCEL 파이프라인	사서의 업무 순서 (검색→읽기→답변)	<code>rag_chain.py</code>
출처 강제 프롬프트	“근거 문서를 대”	<code>RAG_SYSTEM_PROMPT</code>
멀티턴 대화	최근 5장짜리 메모장	<code>conversation.py</code>

기능	비유	코드
채팅 웹 UI	창구 — 질문을 입력하면 답변이 나오는 화면	<code>chat.html</code> , <code>chat.js</code>

FastAPI 서버에 채팅 UI까지 붙여요. 브라우저에서 바로 질문하면 됩니다. ex04에서 재료를 다 모았습니다. 이번 챕터(ex05)에서 드디어 **요리**해요. 검색만 하던 시스템이 답변을 해줍니다.

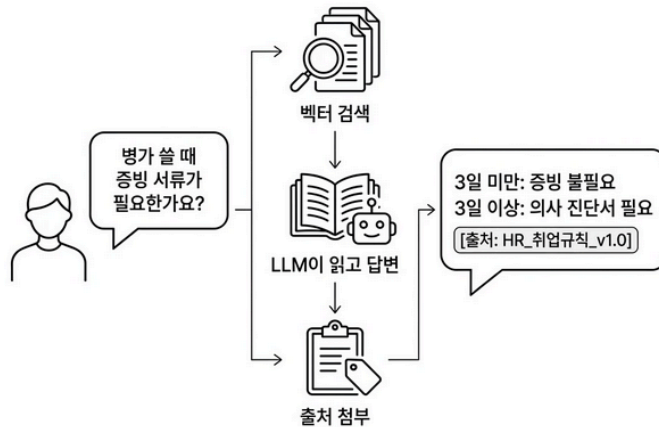


그림 5-4: 드디어 답해준다. 문서를 검색하고, 읽어서, 출처와 함께 자연어로 답변한다.

이제 실습으로 검색 결과를 답변으로 바꾸는 체인을 만들어보겠습니다.

2.1 용어 정리

이야기 속 표현	진짜 이름	정의
부품 + 조립 도구	LangChain	LLM 애플리케이션에 필요한 부품(Retriever, Prompt, LLM, Parser)을 제공하고 파이프라인으로 조립할 수 있게 해주는 Python 프레임워크
사서의 업무 순서	LCEL 파이프라인	LangChain Expression Language. 파이프 연산자()로 Retriever → Prompt → LLM → Parser를 연결하는 체인 조립 방식
근거를 대	출처 강제(Source Grounding)	프롬프트에 “제공된 문서에서만 답하고 출처를 명시하라”는 제약을 거는 기법. 환각을 줄이고 답변 신뢰도를 높인다
5장짜리 메모장	WindowMemory	최근 N턴의 대화만 유지하는 슬라이딩 윈도우 방식의 대화 히스토리 관리. <code>deque(maxlen=k)</code> 기반

이야기 속 표현	진짜 이름	정의
파이프(\)	LCEL 파이프 연산자	A \ B 는 A의 출력을 B의 입력으로 전달. Unix 파이프(cat file \ grep)와 같은 개념
메모장 관리인	ConversationManager	세션별로 WindowMemory를 관리하고, TTL 기반으로 만료된 세션을 정리하는 클래스

2.2 실습 환경 구축

기본 환경(Python 3.12, Ollama)이 없다면 **부록(환경 설정)** 을 먼저 참고하세요.

CH01에서 맛보기로 썼던 LangChain을 이번 챕터에서 본격적으로 활용합니다. ex04에서는 ChromaDB와 sentence-transformers를 직접 사용했지만 ex05에서는 LangChain의 LCEL 파이프라인으로 검색, 프롬프트, LLM을 연결해요.

```
cd ex05
cp .env.example .env
python3.12 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

패키지	버전	역할
langchain	0.3.21	체인 조립 프레임워크
langchain-community	0.3.20	HuggingFaceEmbeddings 등 커뮤니티 통합
langchain-ollama	0.2.3	Ollama LLM 연결
langchain-openai	0.3.7	OpenAI LLM 연결 (선택)
langchain-chroma	0.2.6	ChromaDB Retriever 래퍼
fastapi	0.115.8	웹 API 서버
uvicorn	0.34.0	ASGI 서버

.env 핵심 설정:

```
# 사용할 LLM 제공자 (ollama 또는 openai)
LLM_PROVIDER=ollama
# Ollama에서 사용할 모델 이름
OLLAMA_MODEL=deepseek-r1:8b
# 문서 임베딩(벡터화)에 사용할 모델
```

```

EMBEDDING_MODEL=jhgan/ko-sroberta-multitask
# ChromaDB 데이터가 저장될 로컬 영구 저장소 경로
CHROMA_PERSIST_DIR=./data/chroma_db
# 질문당 검색해서 가져올 관련 문서 조각 개수
RETRIEVER_TOP_K=5
# LLM 프롬프트에 포함할 최근 대화 유지 턴(Turn) 수
CONVERSATION_WINDOW_SIZE=5

```

팁: LLM 선택 — 기본값은 Ollama + **deepseek-r1:8b**입니다. 이후 챕터 부터는 **.env**에서 **LLM_PROVIDER=openai**로 바꾸면 GPT-4o-mini도 쓸 수 있습니다. (단, API 비용이 발생합니다. **.env** 파일에 **OPENAI_API_KEY=sk-xxxxxx** 형태로 key를 등록해서 사용하세요.)

2.3 파일 계층 구조

```

ex05/
├── run.py           [참고] 서버 플로우 실행
├── .env             [참고] 환경 변수
├── README.md        [참고] 프로젝트 설명서
├── requirements.txt [참고] 의존성 목록
├── data/
│   ├── docs/       [참고] 원본 PDF/Word 문서 저장소
│   ├── markdown/   [참고] 마크다운 변환 문서 저장소
│   └── chroma_db/   [참고] 생성된 벡터 DB 영구 저장소
├── src/
│   ├── rag_chain.py [실습] LCEL 파이프라인 + 출처 강제 프롬프트
│   ├── conversation.py [실습] WindowMemory(k=5) 멀티턴 대화
│   ├── llm_factory.py [참고] LLM 인스턴스 생성 (Ollama/OpenAI 분기)
│   ├── vectorstore.py [참고] ChromaDB Retriever 생성 + 문서 파싱/청킹
│   ├── session_manager.py [참고] 세션별 ConversationManager + TTL 관리
│   └── response_parser.py [설명] DeepSeek <think> 제거 + 출처 추출
├── templates/
│   ├── base.html    [참고] 공통 레이아웃
│   └── chat.html     [참고] 채팅 UI 페이지
├── static/
│   ├── css/chat.css [참고] 채팅 UI 스타일
│   └── js/chat.js    [참고] 질문 전송 + 답변 렌더링
└── app/
    ├── main.py       [참고] FastAPI 앱 진입점 + UI 라우팅
    ├── chat_api.py   [설명] POST /api/chat 엔드포인트
    └── session.py     [참고] 세션 쿠키 관리

```

2.4 실습 순서

1. `rag_chain.py` — 파이프라인 구축
2. `conversation.py` — 메모리 연동
3. `chat_api.py` — API 연결
4. 서버 실행 및 웹 UI 테스트

핵심 코드를 먼저 작성하고 마지막에 서버를 띄워 채팅 UI에서 챗봇과 대화해 보겠습니다. 질문이 `/api/chat`으로 들어온 뒤 답변이 나오기까지 전체 흐름을 담당하는 라우터부터 살펴볼게요.

2.5 [설명] chat_api.py — POST /api/chat

이 라우터 코드는 `ex05/app/chat_api.py` 파일에 있습니다. 사서에게 질문을 전달하는 창구 역할이에요. 웹 브라우저나 API 클라이언트가 여기로 질문을 보냅니다.

```
# prefix="/api"로 실제 URL은 /api/chat
router = APIRouter(prefix="/api", tags=["chat"])

@router.post("/chat")
async def chat_endpoint(body: ChatRequest, request: Request):
    # 세션 ID: 요청 본문 > 쿠키 > 신규 생성 순으로 결정
    session_id = body.session_id or get_session_id(request)

    # 대화 매니저에서 이전 대화 히스토리 조회
    conv_manager = get_conversation_manager()
    history_text = conv_manager.get_history_text(session_id)

    # RAG 체인과 Retriever 로드
    chain, retriever = get_rag_chain()

    # Retriever로 관련 문서 검색 (출처 표시에 사용)
    docs = retriever.invoke(question)

    # LCEL 체인 실행: {"question": ..., "history": ...} 형식으로 입력
    # 체인 내부에서 question → 검색 → 포맷 → 프롬프트 → LLM 순서로 처리
    raw_answer = chain.invoke({
        "question": question, # ① 검색 및 프롬프트에 사용
        "history": history_text, # ② 이전 대화 맥락 주입
    })

    # 응답 구조 생성 (answer 정제 + sources 추출)
    response_data = build_response(raw_answer=raw_answer, docs=docs)

    # 세션 히스토리에 이번 대화 저장
    conv_manager.save_turn(session_id, question, response_data["answer"])
```

전체 흐름이 한눈에 보입니다. 세션 확인 → 히스토리 조회 → 문서 검색 → 체인 실행 → 응답 정제 → 히스토리 저장. 사서가 질문을 받고 → 메모장 확인하고 → 서가에서 책 꺼내고 → 읽고 답변하고 → 메모장에 기록하는 것과 같아요.

2.6 실습 1: LCEL 파이프라인 (rag_chain.py)

`chat_api.py` 의 3~5단계에서 호출되는 파이프라인이며 이번 챕터의 핵심입니다. 사서의 업무 순서, 즉 질문 받기 → 문서 찾기 → 읽고 답변하기를 코드로 옮긴 것이에요. 아래 코드를 `ex05/src/rag_chain.py` 에 단계별로 나누어 작성합니다.

1단계: 임포트와 출처 강제 프롬프트

```

from operator import itemgetter

from langchain.schema import Document
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from src.llm_factory import build_llm          # LLM 인스턴스 생성 유틸
from src.vectorstore import build_retriever    # ChromaDB Retriever 생성

# — 출처 강제 프롬프트 —
# 규칙 4개가 환각을 잡는 핵심 장치
RAG_SYSTEM_PROMPT = """당신은 메타코딩 사내 문서 Q&A 비서입니다.
아래에 제공된 문서(Context)만 사용하여 질문에 답변하십시오.

규칙:
1. 반드시 제공된 문서에서만 근거를 찾아 답변하십시오.
2. 문서에서 답을 찾을 수 없으면 "해당 내용은 제공된 문서에서 확인되지 않습니다."라고 답하십시오.
3. 답변 마지막에 근거 문서명을 반드시 명시하십시오. 형식: [출처: 문서명]
4. 추측이나 외부 지식을 사용하지 마십시오.

Context (제공된 문서):
{context}

이전 대화:
{history}
"""

RAG_HUMAN_PROMPT = "질문: {question}"

```

“제공된 문서에서만”, “출처를 명시”, “모르면 모른다고”. 이 세 가지가 환각을 잡는 장치입니다. `{context}` 에는 벡터 검색으로 찾은 문서 조각이, `{history}` 에는 이전 대화 내용이 들어가요.

2단계: 문서 포맷 함수와 LCEL 파이프라인 조립

```

def _format_docs(docs):
    """검색된 Document 목록을 프롬프트에 삽입할 텍스트 형식으로 변환한다."""
    parts = []
    for i, doc in enumerate(docs, start=1):
        source = doc.metadata.get("source", "알 수 없음")
        page = doc.metadata.get("page", "-")
        parts.append(f"[문서 {i}] 출처: {source} (p.{page})\n{doc.page_content}")
    return "\n\n".join(parts)

def build_rag_chain():
    """LCEL 파이프 연산자( | )로 RAG 체인과 Retriever를 조합하여 반환한다."""
    llm = build_llm()          # ① LLM 인스턴스 생성
    retriever = build_retriever() # ② Retriever 생성 (ChromaDB)

    prompt = ChatPromptTemplate.from_messages([
        ("system", RAG_SYSTEM_PROMPT),
        ("human", RAG_HUMAN_PROMPT),
    ])

    # LCEL 파이프: 입력 dict에서 각 키를 꺼내 병렬 처리 후 프롬프트로 합침
    chain = (
        {
            "context": itemgetter("question") | retriever | _format_docs,
            "history": itemgetter("history"),
            "question": itemgetter("question"),
        }
        | prompt
        | llm
        | StrOutputParser()
    )

    return chain, retriever

```

`build_rag_chain()` 이 사서의 업무 순서 전체입니다. `chain` 변수를 따라가 보겠습니다.

- `itemgetter("question") | retriever | _format_docs` — 질문으로 벡터 DB를 검색하고 찾은 문서 조각을 텍스트로 포맷합니다. 사서가 서가에서 책을 꺼내는 단계예요.
- `itemgetter("history")` — 이전 대화 히스토리를 그대로 전달합니다. 사서의 메모장이죠.
- `| prompt | llm | StrOutputParser()` — 검색 결과 + 히스토리 + 질문을 프롬프트에 합치고 LLM을 호출한 뒤 텍스트만 추출합니다.

3단계: 싱글턴 캐시

```
# 앱 시작 시 1회만 초기화
_rag_chain_cache = None
_retriever_cache = None

def get_rag_chain():
    """RAG 체인 싱글턴 — 최초 호출 시 생성, 이후 재사용"""
    global _rag_chain_cache, _retriever_cache
    if _rag_chain_cache is None:
        _rag_chain_cache, _retriever_cache = build_rag_chain()
    return _rag_chain_cache, _retriever_cache
```

최초 호출 시 체인을 한 번 만들고 이후에는 캐시된 것을 재사용합니다. `chat_api.py` 에서 매 요청마다 이 함수를 호출해요.

`build_llm()` 함수는 `llm_factory.py` 에 분리되어 있습니다. `.env` 의 `LLM_PROVIDER` 값에 따라 Ollama 또는 OpenAI를 선택하죠. `temperature=0.1` 로 낮춘 건 의도적입니다. Q&A 비서는 창의적인 답변이 아니라 **정확한 답변**이 필요하니까요.

`rag_chain.py` 에서 사용하는 인프라 함수는 별도 모듈로 분리되어 있습니다.

파일	함수	역할
<code>llm_factory.py</code>	<code>build_llm()</code>	Ollama/OpenAI LLM 인스턴스 생성
<code>vectorstore.py</code>	<code>build_retriever()</code>	ChromaDB Retriever 생성 (DB 없으면 자동 구축)
<code>vectorstore.py</code>	<code>_parse_and_chunk_docs()</code>	PDF/DOCX/XLSX 파싱 → 청킹 (CH04 로직 재사용)

2.7 [설명] response_parser.py — 답변 정제

이 코드는 `ex05/src/response_parser.py` 파일에 분리되어 있습니다. LLM이 내놓는 원문 응답은 깨끗하지 않을 수 있어요. 특히 DeepSeek R1 모델은 `<think>...</think>` 태그로 추론 과정을 포함해서 내보냅니다. 이걸 제거하고 깔끔한 답변만 뽑아내는 게 이 모듈의 역할이에요.

```
def parse_answer_text(raw_answer):
    """LLM 원문 응답에서 <think>...</think> 태그를 제거하고 답변 텍스트만 반환한다."""
    text = raw_answer
    # DeepSeek R1의 <think> 추론 토큰 제거
    text = re.sub(r"<think>.*?</think>", "", text, flags=re.DOTALL)
    text = text.strip()

    # 빈 문자열이면 기본 메시지 반환
    if not text:
        text = "답변을 생성하지 못했습니다. 다시 시도해 주세요."
    return text
```


`re.DOTALL` 플래그는 `.`이 줄바꿈 문자(`\n`)까지 포함해 모든 글자를 매칭할 수 있게 해줍니다. 이 플래그가 없으면 여러 줄로 나뉘어 출력된 `<think>` 태그 내부를 한 번에 지워낼 수 없어요. `.*?`에서 `?`는 Non-greedy(최소 매칭) 옵션으로, 문서 끝자락의 엉뚱한 태그까지 지우지 않고 가장 먼저 만나는 닫는 태그(`</think>`)까지만 딱 맞춰서 지우라는 뜻입니다.

`build_response()` 함수는 정제된 답변과 출처를 하나의 딕셔너리로 묶습니다.

```
def build_response(raw_answer, docs):
    """LLM 원문 응답과 검색 문서로부터 최종 API 응답 딕셔너리를 구성한다."""
    answer = parse_answer_text(raw_answer)
    sources = parse_sources_from_docs(docs)
    return {"answer": answer, "sources": sources}
```

API 응답 형태는 이렇습니다.

```
{
  "answer": "3일 미만 병가는 중병이 불필요하고, 3일 이상은 의사 진단서가 필요합니다.",
  "sources": [
    {"doc": "HR-취업규칙_v1.0", "page": 3, "snippet": "제15조(병가) 질병이나..." }
  ]
}
```

2.8 실습 2: 멀티턴 대화 (conversation.py)

사서의 메모장에 해당하는 `WindowMemory` 클래스입니다. 최근 N턴의 대화만 유지하는 슬라이딩 윈도우를 구현해요. 아래 코드를 `ex05/src/conversation.py`에 작성합니다.

```
from collections import deque

class WindowMemory:
    """최근 N턴의 대화만 유지하는 슬라이딩 윈도우 메모리."""

    def __init__(self, k=5, human_prefix="사용자", ai_prefix="AI 비서"):
        self.k = k
        self.human_prefix = human_prefix
        self.ai_prefix = ai_prefix
        self._turns = deque(maxlen=k)

    def get_history(self):
        """최근 N턴의 대화를 텍스트로 반환한다."""
        lines = []
        for question, answer in self._turns:
            lines.append(f"{self.human_prefix}: {question}")
```

```

        lines.append(f"{self.ai_prefix}: {answer}")
    return "\n".join(lines)

def save_turn(self, question, answer):
    """사용자 질문과 AI 답변 1턴을 저장한다."""
    self._turns.append((question, answer))

def clear(self):
    """히스토리를 초기화한다."""
    self._turns.clear()

```

`deque(maxlen=k)`가 핵심입니다. Python의 `deque`에 `maxlen`을 설정하면 새 항목이 들어올 때 가장 오래된 항목이 자동으로 빠져요. 메모장에 6번째 메모를 붙이면 1번째가 떨어지는 겁니다. `get_history()`는 저장된 대화를 텍스트로 만듭니다. 이 텍스트가 프롬프트의 `{history}`에 들어가요.

사용자: 병가 쓸 때 증빙 필요해?
 AI 비서: 3일 미만은 불필요하고, 3일 이상은 진단서가 필요합니다.
 사용자: 그러면 연차로 대체할 수 있어?
 AI 비서: 네, 병가 대신 연차를 사용할 수 있습니다.

LLM이 이 히스토리를 보면 “그러면”이 병가를 가리킨다는 걸 이해할 수 있어요.

`WindowMemory`를 세션별로 관리하는 기능은 `session_manager.py`에 분리되어 있습니다. `ConversationManager`는 여러 사용자(세션)의 메모장을 따로 관리하는 래퍼예요. 세션별로 `WindowMemory`를 하나씩 만들고 TTL(기본 1시간)이 지나면 자동으로 정리합니다. 사서가 손님 1시간 넘게 안 오면 메모장을 치우는 거라고 생각하면 돼요.

파일	함수/메서드	역할
<code>session_manager.py</code>	<code>ConversationManager</code>	세션별 <code>WindowMemory</code> 관리 + TTL 기반 만료 정리
<code>session_manager.py</code>	<code>get_conversation_manager()</code>	<code>ConversationManager</code> 싱글턴 반환
<code>session_manager.py</code>	<code>get_history_text(session_id)</code>	해당 세션의 대화 히스토리를 텍스트로 반환
<code>session_manager.py</code>	<code>save_turn(session_id, q, a)</code>	질문-답변 1턴을 세션에 저장

주의: 메모리 기반입니다 `ConversationManager`의 세션 데이터는 서버 메모리에만 존재합니다. 서버를 재시작하면 대화 히스토리가 사라져요. 실전 운영 환경에서는 Redis 같은 외부 저장소를 쓰는 게 일반적입니다. (CH07에서 운영을 위한 캐시 개념을 배우지만, 실습 환경의 복잡도를 낮추기 위해 외부 서비스(Redis) 연동 대신 인메모리 방식을 유지합니다.)

팁: 다른 메모리 방식도 있습니다 슬라이딩 윈도우(최근 N턴 유지)는 가장 단순한 방식입니다. 대화가 길어지면 앞부분이 통째로 사라지는 단점이 있죠. 다른 접근도 있습니다. - **Summary Memory** — LLM이 이전 대화를 요약해서 저장. 긴 대화도 맥락을 유지하지만 요약할 때마다 LLM을 한 번 더 호출해요. - **Token Buffer Memory** — 턴 수가 아니라 토큰 수 기준으로 제

한. LLM의 컨텍스트 창을 효율적으로 사용합니다. — **Summary Buffer Memory** — 최근 대화는 원문 그대로, 오래된 대화는 요약. 위 두 방식의 절충안이에요.

이 책에서는 구현이 간단하고 LLM 추가 호출이 없는 슬라이딩 윈도우를 씁니다.

2.9 실행 결과

핵심 코드를 모두 살펴봤습니다. 이제 서버를 실행하고 브라우저에서 직접 질문해 봅시다. FastAPI 서버를 실행합니다.

```
# 주의: Ollama가 미리 실행되어 있어야 합니다. (ollama serve 또는 앱 실행)
# FastAPI 서버 실행
python run.py
```

```
$ python app/main.py
[INFO] 서버 시작: http://0.0.0.0:8000
[INFO] 채팅 UI: http://localhost:8000/chat
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [48416] using WatchFiles
INFO: Started server process [48418]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

그림 5-5: 서버가 시작되면 채팅 UI 주소가 함께 출력된다.

브라우저에서 <http://localhost:8000/chat>을 엽니다.

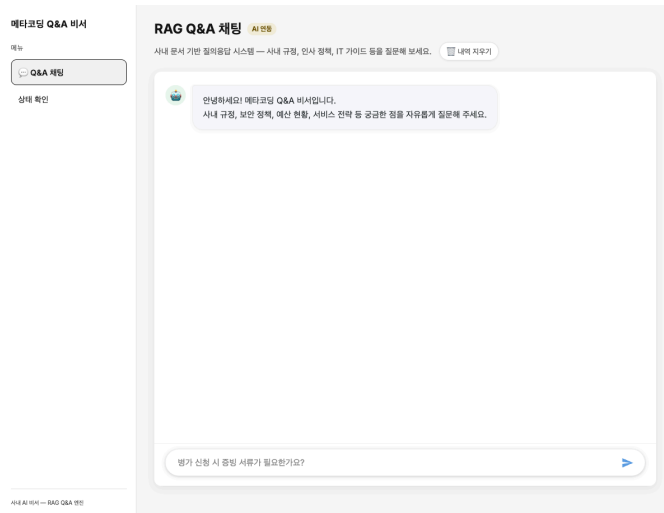


그림 5-6: 브라우저에서 바로 질문할 수 있는 채팅 창구. 사서가 자리에 앉았다.

“평가 쓸 때 증빙 서류가 필요한가요?”를 입력하고 잠시 기다립니다.

참고: 첫 답변 대기 시간 로컬 환경에서 모델을 처음 메모리에 올리고 추론하는 과정에서 약 60~120초 정도 시간이 소요될 수 있습니다. 응답이 올 때까지 조금만 기다려주세요!

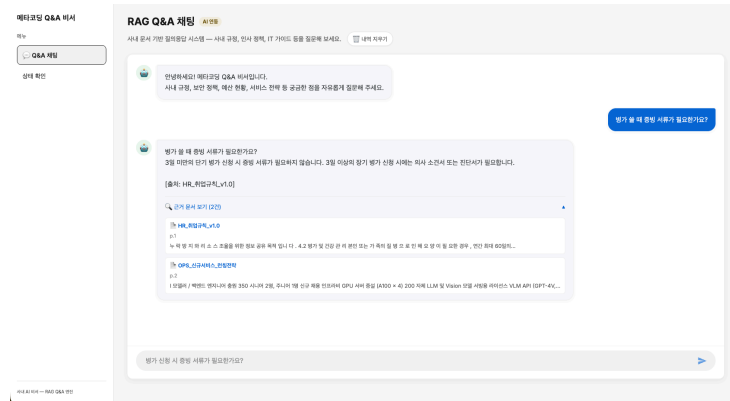


그림 5-7: 드디어 답해준다. 문서를 검색하고, 읽어서, 출처와 함께 자연어로 답변한다.

이어서 “그러면 연차로 대체할 수 있어?”를 입력해 봅니다. 이전 대화를 기억하고 병가 맥락을 이어서 답변해요. 사서가 메모장을 보고 있다는 뜻이죠.

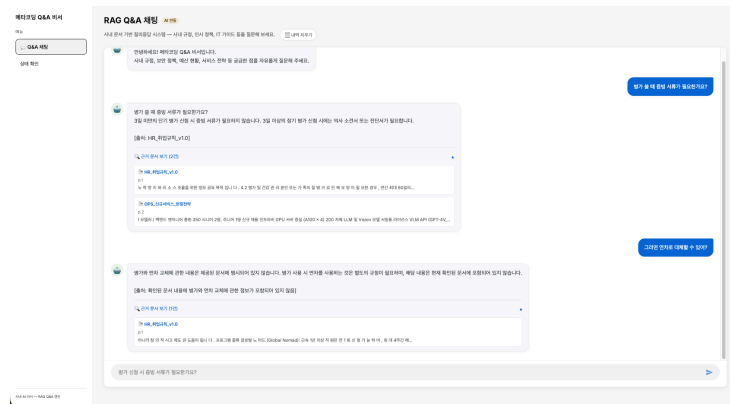


그림 5-8: 이전 대화 맥락(병가)을 기억하고 답변을 이어나간다.

Ctrl + C를 눌러 서버를 종료합니다.

2.10 더 알아보기

LCEL vs 레거시 체인 — LangChain 초기에는 **RetrievalQA**, **ConversationalRetrievalChain** 같은 미리 만들어진 체인을 썼습니다. 하지만 내부가 블랙박스라 커스터마이징이 어려웠어요. LCEL은 각 단계를 파이프로 직접 조립하기 때문에 어디에 무슨 로직이 들어가는지 명확하게 보입니다. LangChain 0.2 이후부터는 LCEL이 권장 방식이에요.

temperature와 RAG — Q&A 시스템에서 **temperature=0.1**을 쓰는 건 일반적입니다. 0에 가까울수록 LLM이 확률 높은 토큰을 선택하므로 일관된 답변이 나와요. 반대로 창의적 글쓰기에서는 0.7~1.0을 씁니다. RAG에서 temperature를 높이면 문서에 없는 내용을 “창작”할 위험이 커집니다.

윈도우 크기 튜닝 — **CONVERSATION_WINDOW_SIZE=5**는 최근 5턴을 기억한다는 뜻입니다. 이 숫자를 키우면 맥락을 더 많이 유지할 수 있지만 프롬프트가 길어져서 연산 비용이 올라가고 응답이 느려져요. 상용 API 모델은 아주 큰 컨텍스트 창을 지원하므로 대화 내용을 전부 밀어 넣어도 무리 없이 동작합니다. 하지만 실습에서 쓰려는 로컬 LLM(DeepSeek-R1:8B) 환경에서는 컨텍스트 창 제한과 처리 속도를 고려하면 최근 5턴만 유지하는 슬라이딩 윈도우가 현실적인 적정선이에요.

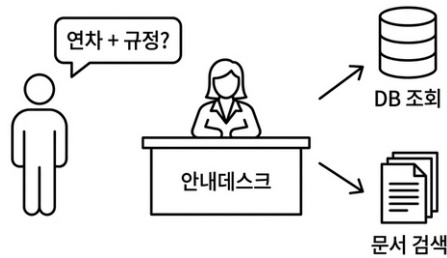
2.11 이것만은 기억하세요

- **검색은 재료, 답변은 요리입니다.** 벡터 검색으로 재료(문서 조각)를 찾고 LLM이 요리(자연어 답변)로 만들어줍니다. LCEL 파이프라인이 이 레시피예요.
- **출처 없는 답변은 근거 없는 주장입니다.** 프롬프트에 출처 강제를 넣어 환각을 잡습니다.
- 다음 챗터에서는 이 Q&A 엔진과 CH02의 사내 시스템(DB)을 합쳐서 “김대리 연차 개수와 사용 규정은?”이라는 **복합 질문**에 답하는 **통합 에이전트**를 만듭니다.

Ch.6: “연차 몇 개, 규정은?” 동시에 — 통합 에이전트 설계 (ex06)

한 줄 요약: AI 비서는 안내데스크다. 질문을 듣고, 맞는 담당자를 찾아 정보를 가져온다.

핵심 개념: QueryRouter (3단계 라우팅), 도구(Tool) — @tool 데코레이터, ReAct 에이전트 (AgentExecutor)



안내데스크 — 통합 에이전트

1.1 RAG가 답 못 하는 질문

CH05에서 RAG Q&A 엔진을 완성했습니다. 질문을 입력하면 사내 문서에서 내용을 찾아 답해주는 엔진입니다. 출처까지 같이 나오니 꽤 만족스러웠습니다.

옆자리 동료가 다가옵니다.

동료: “이게 그 AI 비서예요? 한번 써봐도 되죠?”

나: “당연하죠. 뭐든 물어보세요.”

동료가 채팅창에 질문을 입력합니다.

동료: “A 사원 남은 연차는? 그리고 연차 신청 절차 알려줘.”

잠시 후.

죄송합니다. 해당 직원에 대한 정보를 찾지 못했습니다.

동료가 고개를 갸웃합니다.

동료: “직원 이름도 모르는 AI 비서예요?”

아...

RAG는 사내 문서에서 ‘A 사원’을 찾으려고 했습니다. 당연히 없죠. 직원 연차 정보는 문서가 아니라 DB에 있으니까요. CH02에서 PostgreSQL에 저장해둔 데이터입니다. 문서 검색이 아니라 DB 조회가 필요한 질문인데 AI 비서는 그 차이를 몰랐습니다.

1.2 두 종류의 정보

지금 만들어둔 것을 돌이켜보면:

- ex02 에서 직원/연차/매출 데이터를 PostgreSQL DB에 저장하는 API를 만들었습니다.
- ex04~ex05 에서 사내 문서를 벡터DB에 넣고 검색하는 RAG 엔진을 만들었습니다.

두 개가 따로 놓고 있습니다. AI 비서는 RAG만 쓸 줄 알지 DB에 어떻게 접근하는지는 모릅니다.

“A 사원 연차 몇 개?” → PostgreSQL에서 숫자를 조회하는 질문. “연차 신청 절차?” → 취업규칙 문서를 검색하는 질문. “A 사원 연차 + 신청 절차?” → 둘 다 필요한 복합 질문.

진짜 AI 비서가 되려면 이 두 가지를 상황에 맞게 골라 쓰거나, 필요하면 동시에 써야 합니다.

1.3 안내데스크를 떠올려봅시다

회사에 처음 입사한 날을 떠올려 봅시다. 낯선 건물에 들어서자마자 1층 안내데스크로 향했습니다.

방문자: “인사 관련 서류는 어디서 받아요?”

안내데스크 직원이 잠깐 생각하더니 답해줍니다.

안내데스크: “인사팀은 3층이에요. 엘리베이터 내리셔서 오른쪽입니다.”

며칠 후 조금 복잡한 질문이 생깁니다.

방문자: “신입사원 교육 신청이랑, 노트북 배정은 어디서 해요?”

안내데스크: “교육 신청은 3층 인사팀이고, 노트북은 2층 IT 지원팀이에요. 두 곳 다 가셔야 해요.”

안내데스크는 질문을 듣고 어떤 부서 소관인지 파악해서 담당자를 연결해줍니다. 복합 질문이면 여러 부서로 동시에 안내하고요.

AI 비서도 이렇게 되어야 합니다. 질문을 보고 DB가 필요한지 문서가 필요한지 아니면 둘 다인지 판단합니다. 판단이 끝나면 각 담당자(도구)에게 작업을 맡기고 결과를 하나로 묶어서 답해줍니다. 이번 챕터에서 만들 **통합 에이전트** 가 바로 이 안내데스크입니다.

1.4 안내데스크: QueryRouter

에이전트 안에는 **QueryRouter** 라는 안내데스크의 ‘판단 규칙’이 들어 있습니다. 모든 질문은 3단계를 거쳐 분류되는데, 베테랑 안내 직원이 일하는 방식과 똑같습니다.

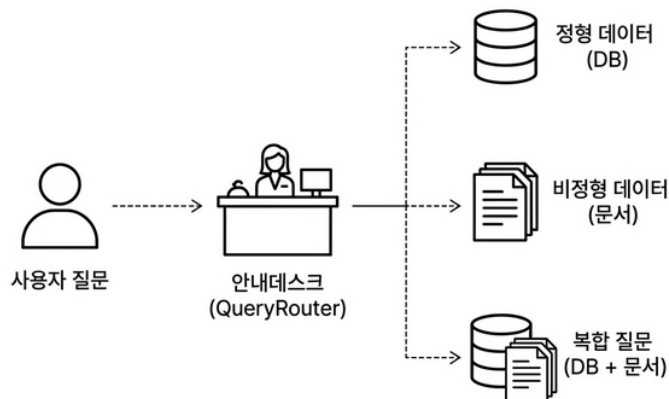


그림 6-m: 사용자의 질문 목적지에 맞게 알맞은 담당자를 배정하는 안내데스크.

1단계: 단어만 들고 바로 알기 (키워드 매칭) 가장 빠르고 확실한 방법입니다. - “연차”, “매출”이 들리면 DB(정형 데이터) 담당자에게 보냅니다. - “절차”, “규정”이 들리면 문서(비정형 데이터) 담당자에게 보냅니다. - 둘 다 들리면 양쪽 모두에게 보냅니다(복합 질문).

2단계: 전문 용어 파악하기 (컬럼명 매칭) 일상적인 단어가 없다면 혹시 개발자가 쓴 `remaining_days` 나 `emp_no` 같은 DB 컬럼명(전문 용어)이 섞여 있는지 확인합니다. 발견되면 DB 담당자에게 넘기면 됩니다.

3단계: 꼼꼼하게 문맥 따져보기 (LLM 판단) 단어만으로 도저히 모르겠으면 시간이 조금 걸리더라도 대규모 언어 모델(LLM)에게 직접 물어봐서 최종 목적지를 정합니다.

대부분의 질문은 바로 알아듣는 **1단계** 에서 끝납니다. 2단계와 3단계는 혹시 모를 상황을 대비한 ‘예비책’이에요. 쉬운 건 빠르게 처리하고 어려운 것만 시간 들여 고민하는 구조입니다.

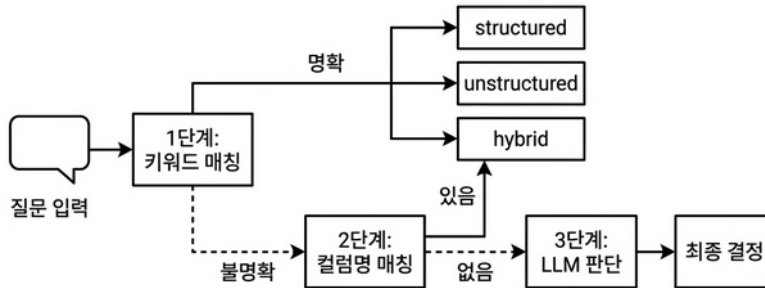


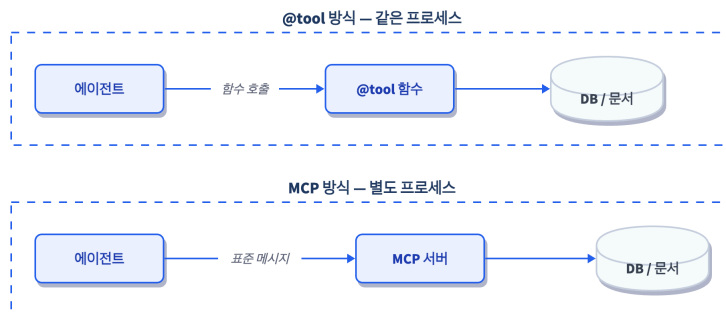
그림 6-1: 대부분의 질문은 1단계 키워드 매칭에서 끝난다. 모호할 때만 더 비싼 방법을 쓴다.

1.5 담당자들: 도구(Tool)

라우터가 방향을 정하면 에이전트가 도구 목록에서 맞는 것을 골라 씁니다. 지금 만들어둔 도구는 네 가지입니다.

도구	역할	비유
<code>leave_balance</code>	직원 연차 잔여 조회	인사팀 연차 담당자
<code>sales_sum</code>	부서별 매출 합계 조회	재무팀 집계 담당자
<code>list_employees</code>	직원 목록 조회	인사팀 명부 담당자
<code>search_documents</code>	문서 벡터 검색	문서실 사서

AI 에이전트가 외부 기능을 호출하는 방법은 크게 두 가지가 있습니다. 하나는 Anthropic이 만든 **MCP(Model Context Protocol)** 입니다. 도구를 별도 서버로 분리하고 표준 메시지 규격으로 통신하는 방식이라 어떤 프레임워크에서든 같은 도구를 재사용할 수 있습니다. 다른 하나는 LangChain의 **@tool** 데코레이터입니다. 같은 프로세스 안에서 함수를 직접 호출하니 설정이 간단합니다.



Tool vs MCP 방식 비교

핵심 개념은 같습니다. “AI가 함수의 이름과 설명을 보고 스스로 호출한다.” 이 책에서는 `@tool` 로 원리를 익히겠습니다. 원리를 이해하면 MCP로 확장하는 건 어렵지 않습니다. 각 도구는 `@tool` 데코레이터로 선언되어 있습니다.

```
@tool
def leave_balance(emp_no: str) -> dict:
    """직원의 연차 잔여 일수를 조회한다."""
    # DB 조회 로직...
    return {"emp_no": emp_no, "remaining_days": 8}
```

데코레이터를 붙이면 LangChain이 도구의 이름과 설명, 파라미터를 자동으로 읽어갑니다. 에이전트는 이 정보를 보고 “이 질문에는 어떤 도구가 맞지?”를 판단하게 됩니다. 이처럼 LLM이 상황에 맞는 도구를 스스로 골라서 호출하는 기능을 **도구 호출(Tool Calling)** 이라고 합니다. 모든 LLM이 지원하는 건 아니라서 OpenAI의 `gpt-4o-mini` 나 Ollama의 `llama3.1:8b` 처럼 Tool Calling을 지원하는 모델을 골라야 합니다.

독스트링이 중요한 이유가 여기 있습니다. 함수 설명이 불명확하면 에이전트가 엉뚱한 도구를 고릅니다. 안내데스크 직원에게 각 팀이 무슨 일을 하는지 제대로 알려줘야 제대로 연결해주는 것과 마찬가지입니다.

1.6 생각하고, 행동하고, 확인한다: ReAct

도구가 준비됐습니다. 이제 에이전트가 이 도구를 어떻게 쓰는지 살펴보겠습니다. 에이전트의 두뇌는 **ReAct 패턴** 으로 동작합니다. Reason(추론) + Act(행동). 이름이 거창해 보이지만 원리는 단순해요. 생각하고 행동하는 걸 반복합니다.

“A 사원 연차 얼마 남았고 연차 신청 절차는?” 이런 질문이 들어오면:

1. **생각**: “연차 잔여를 알려면 `leave_balance` 도구를 써야 해.”
2. **행동**: `leave_balance("A사원")` 호출 → DB에서 8일 반환
3. **관찰**: “8일이구나. 이제 신청 절차도 찾아야 해.”
4. **생각**: “절차는 문서에 있을 거야. `search_documents` 써야지.”
5. **행동**: `search_documents("연차 신청 절차")` 호출 → 취업규칙에서 절차 발견
6. **관찰**: “이제 다 알았다.”
7. **최종 답변 생성**: “A 사원 연차 8일 남아 있고요, 규정에 따르면 3일 전까지...”

이 과정을 최대 10회까지 반복할 수 있습니다. 한 번에 답을 못 찾으면 다시 생각하고 다시 행동합니다. 도중에 오류가 나도 포기하지 않고 다른 방법을 시도해요.

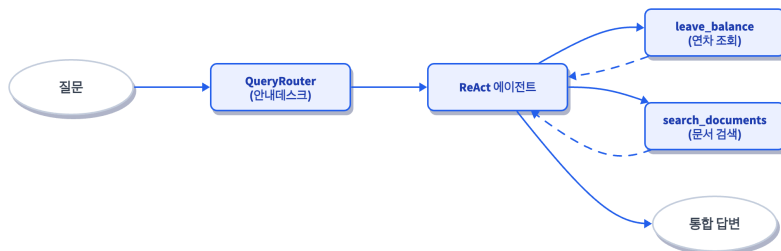


그림 6-2: 질문이 들어오면 QueryRouter가 분류하고, ReAct 에이전트가 필요한 도구를 순서대로 호출한다.

1.7 이제 복합 질문에 답할 수 있다

동료가 다시 같은 질문을 입력합니다.

동료: “A 사원 연차 몇 일 남았어요? 그리고 연차 신청 절차도 알려주세요.”

이번엔 통합 에이전트가 답합니다.

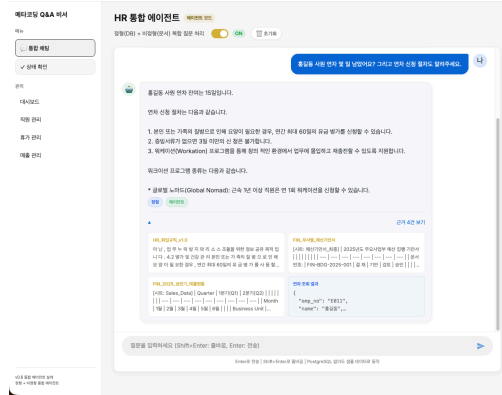


그림 6-3: 한 번의 질문으로 안내데스크가 담당자들을 알아서 찾아 정보를 조합해준다.

확인해 드릴게요.

A 사원(E003) 연차 현황:

– 총 연차: 15일 | 사용: 7일 | 잔여: 8일

연차 신청 절차 (HR_취업규칙_v1.0):

1. 그룹웨어 → 근태관리 → 연차신청
2. 사용일 3일 전까지 신청
3. 팀장 승인 후 자동 처리됩니다.

【출처: HR_취업규칙_v1.0】

동료가 고개를 끄덕입니다.

동료: “이제 진짜 비서 같은데요.”

드디어.

그럼 지금부터 이 안내데스크, **통합 에이전트** 를 직접 만들어 보겠습니다.

1.8 이번 버전에서 뭘 만드나

ex06에서는 세 가지를 통합합니다.

기능	비유	코드
3단계 질문 분류기	안내데스크의 판단 기준	<code>router.py</code>

기능	비유	코드
도구(Tool) 4개	각 담당 부서	<code>mcp_tools.py</code>
ReAct 에이전트	안내데스크 전체	<code>agent.py</code>

이제 실습으로 DB 조회와 문서 검색을 하나로 합쳐보겠습니다.

2.1 용어 정리

비유	기술 용어	정식 정의
안내데스크 전체	에이전트 (Agent)	사용자의 질문을 받아 스스로 판단하고, 필요한 도구를 선택·실행해서 최종 답변을 만들어내는 자율적 프로그램
안내데스크의 분류 기준	QueryRouter	사용자 질문을 분석해서 처리 경로(structured / unstructured / hybrid)를 결정하는 분류기
각 담당자	도구(Tool)	LangChain <code>@tool</code> 데코레이터로 선언된 함수. 에이전트가 선택해서 실행할 수 있는 원자적 작업 단위
생각→행동→관찰 반복	ReAct 패턴	Reason(추론) + Act(행동)의 반복으로 복잡한 질문을 단계적으로 해결하는 에이전트 실행 전략
반복 실행기	AgentExecutor	ReAct 루프를 실행하고 도구 호출을 관리하는 LangChain 컴포넌트

2.2 파일 구조

```

ex06/
├── run.py           [실습] 서버 플로우 실행
├── src/
│   ├── router.py   [실습] 3단계 QueryRouter
│   ├── mcp_tools.py [실습] @tool 4개 (leave_balance, sales_sum, list_employees, search_documents)
│   ├── agent.py     [실습] ReAct 에이전트 (IntegratedAgent)
│   ├── llm_factory.py [참고] LLM 인스턴스 생성 (Ollama/OpenAI 분기)
│   ├── db_helper.py [참고] PostgreSQL 쿼리 + ChromaDB 벡터스토어 구축
│   └── agent_helpers.py [참고] 에이전트 결과 파싱/직렬화/풀백 유틸
├── app/
│   ├── main.py      [설명] FastAPI 앱 진입점
│   ├── chat_api.py  [설명] 에이전트/RAG 모드 선택 API
│   ├── admin_crud.py [참고] 관리자 대시보드 DB CRUD
│   └── admin_views.py [참고] 관리자 대시보드 라우터

```

```

|   └── database.py      [참고] PostgreSQL 연결 래퍼
|   └── templates/
|       ├── chat.html    [참고] 채팅 웹 UI
|       ├── dashboard.html [참고] 관리자 대시보드
|       ├── employees.html [참고] 직원 관리 화면
|       ├── leaves.html   [참고] 휴가 관리 화면
|       └── sales.html     [참고] 매출 관리 화면
|   └── static/
|       ├── css/chat.css  [참고] UI 스타일
|       └── js/chat.js     [참고] 채팅 로직

```

2.3 실습 환경 구축

기본 환경(Python 3.12, Ollama, Docker)이 없다면 **부록(환경 설정)** 을 먼저 참고하세요.

```

cd ex06
cp .env.example .env
python3.12 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
docker compose up -d      # PostgreSQL 실행
pip install -r requirements.txt

```

이전 챕터 Docker 종료: CH02의 Docker가 실행 중이라면 `cd ex02 && docker compose down` 으로 먼저 종료하세요. 같은 포트를 사용합니다.

패키지	역할
<code>langchain</code>	체인/에이전트 프레임워크
<code>langchain-ollama</code>	Ollama LLM 연결
<code>langchain-openai</code>	OpenAI LLM 연결 (선택)
<code>langchain-huggingface</code>	HuggingFace 임베딩
<code>chromadb</code>	벡터 DB
<code>psycopg2-binary</code>	PostgreSQL 드라이버
<code>fastapi</code>	웹 API 서버

주의 Ollama에서 Tool Calling을 지원하는 모델을 골라야 합니다. 모든 모델이 Tool Calling을 지원하는 것은 아닙니다.

참고로 “툴 콜링”은 LLM 제공사마다 명칭이 다릅니다. 개념은 같지만 API 형식이 다르기 때문에 LangChain 같은 프레임워크가 이 차이를 추상화해줍니다.

제공사	명칭	API 형식
OpenAI (GPT)	Function Calling / Tool Use	<code>tool_calls</code> 필드로 응답
Anthropic (Claude)	Tool Use	<code>tool_use</code> 블록으로 응답
Google (Gemini)	Function Calling	<code>function_call</code> 파트로 응답
Ollama (로컬)	Tool Calling	모델마다 지원 여부 다름

Ollama에서 실행 가능한 로컬 모델의 Tool Calling 지원 현황입니다.

모델	Tool Calling	한국어	비고
<code>llama3.1:8b</code>	O	O	이 책에서 사용. 툴 콜링 안정성과 한국어 품질의 균형이 좋음
<code>qwen2.5:7b</code>	O	O	한국어 품질 우수. 대안으로 적합
<code>mistral:7b</code>	O	△	영어 중심. 한국어 사내 문서에는 불리
<code>deepseek-r1:8b</code>	X	O	추론(Chain-of-Thought) 특화. 툴 콜링 미지원
<code>llava:7b</code>	X	X	비전 특화. 툴 콜링·한국어 모두 미지원

이 책에서는 `llama3.1:8b` 를 사용합니다. 8B 크기로 16GB RAM에서 무리 없이 돌아가고, 툴 콜링 정확도와 한국어 응답 품질이 가장 균형 잡혀 있기 때문입니다.

.env 핵심 설정:

```
# Ollama 사용 시
LLM_PROVIDER=ollama
OLLAMA_MODEL=llama3.1:8b # Tool Calling 필수

# OpenAI 사용 시 (권장)
# LLM_PROVIDER=openai
# OPENAI_API_KEY=sk-...
# OPENAI_MODEL=gpt-4o-mini
```

2.4 실습 순서

1. `router.py` — QueryRouter 작성
2. `mcp_tools.py` — 도구(Tool) 작성
3. `agent.py` — ReAct 에이전트 조립
4. `python run.py` — 복합 질문 테스트

핵심 코드를 먼저 작성하고 마지막에 서버를 띄워 채팅 UI에서 챗봇과 대화해 보겠습니다. QueryRouter부터 도구, 통합 에이전트 순서로 살펴봅니다.

2.5 실습 1: QueryRouter — 3단계 질문 분류기 (router.py)

에이전트가 도구를 무작정 고르는 것보다 질문의 의미를 먼저 파악하고 방향을 잡아주면 성능이 훨씬 좋아집니다. 이를 위해 3단계로 질문을 분류하는 `QueryRouter` 를 작성합니다. 아래 코드를 `ex06/src/router.py`에 단계별로 나누어 작성합니다.

1단계: 라우팅 판단 기준 키워드 정의

먼저 질문을 정형(DB)인지 비정형(문서)인지 판별할 기준이 되는 키워드 목록을 정의합니다.

```
"""ex06 - 질문 라우팅 모듈."""
```

```
import json
import os
import re
```

```
# -----
# 1. 라우팅 판단 기준 키워드 정의
# -----
```

```
# 정형 데이터(DB) 관련 키워드 - 숫자/통계/명단 조회
```

```
STRUCTURED_KEYWORDS = [
    "잔여", "잔량", "연차", "휴가", "남은", "몇 일", "며칠",
    "매출", "합계", "총액", "금액", "얼마", "실적",
    "목록", "명단", "직원", "사원", "리스트", "조회",
    "통계", "평균", "부서별", "합산", "입사일", "날짜"
]
```

```
# 비정형 데이터(문서) 관련 키워드 - 절차/정책/안내
```

```
# 주의: "알려줘", "뭐야" 같은 범용 요청 접미사는 제외한다.
```

```
# 이런 표현은 모든 질문에 붙을 수 있어 판별력이 없다.
```

```
UNSTRUCTURED_KEYWORDS = [
    "절차", "방법", "어떻게", "규정", "정책", "기준",
    "온보딩", "입사 안내", "가이드", "매뉴얼",
    "복지", "혜택", "보안", "출장", "비용",
    "설명해", "무엇인가", "어떤가",
]
```

```
# DB 스키마 컬럼/테이블명 — Step 2 매칭 대상
SCHEMA_TERMS = {
    # leave_balance 테이블
    "remaining_days": "structured",
    "used_days": "structured",
    "total_days": "structured",
    # sales 테이블
    "amount": "structured",
    "total_amount": "structured",
    "revenue": "structured",
    # employees 테이블
    "emp_no": "structured",
    "department": "structured",
    "hire_date": "structured",
}
```

사원이나 매출처럼 숫자로 된 데이터는 **structured** 로, 회사 정책이나 규정처럼 문장으로 된 문서 지식은 **unstructured** 키워드로 나누었습니다.

2단계: QueryRouter 공개 인터페이스 구현

분류기 클래스를 선언하고 외부에서 호출할 **classify_query** 메서드를 먼저 작성합니다. 규칙 기반 → 스키마 기반 → LLM 기반(폴백)
3단계 판별 구조가 어떻게 연결되는지 눈여겨보세요.

```
class QueryRouter:
    """질문 유형 분류기."""

    def __init__(self, llm=None):
        """라우터를 초기화한다."""
        self._llm = llm

    # -----
    # 2. 공개 인터페이스
    # -----

    def classify_query(self, query):
        """질문을 분석하여 처리 경로를 반환한다."""
        # ① Step 1: 규칙 기반 키워드 매칭
        step1_result = self._step1_rule_based(query) # ①
        if step1_result is not None:
            return step1_result

        # ② Step 2: DB 스키마 컬럼명 매칭
        step2_result = self._step2_schema_based(query) # ②
        if step2_result is not None:
```

```

        return step2_result

# ③ Step 3: LLM 판단 (폴백)
if self._llm is not None:
    step3_result = self._step3_llm_based(query) # ③
    if step3_result is not None:
        return step3_result

# ④ 기본값: 비정형으로 처리
return "unstructured" # ④

```

3단계: 내부 구현 메서드 (3단계 판단 로직)

인터페이스에서 호출한 `_step1_rule_based`, `_step2_schema_based`, `_step3_llm_based` 를 실제로 구현하는 코드입니다. 가장 저렴하고 빠른 키워드 매칭을 먼저 돌리고 그래도 안 될 때만 LLM을 호출합니다. 속도와 비용을 동시에 잡는 구조입니다.

```

# -----
# 3. 내부 구현 메서드
# -----

def _step1_rule_based(self, query):
    """규칙 기반 키워드 매칭으로 경로를 결정한다."""
    query_lower = query.lower()

    structured_hits = sum(
        1 for kw in STRUCTURED_KEYWORDS if kw in query_lower
    )
    unstructured_hits = sum(
        1 for kw in UNSTRUCTURED_KEYWORDS if kw in query_lower
    )

    # 두 쪽 모두 히트 → hybrid (단, 한 쪽이 2배 이상 우세하면 그 쪽으로 분류)
    if structured_hits > 0 and unstructured_hits > 0:
        if structured_hits >= unstructured_hits * 2:
            return "structured"
        if unstructured_hits >= structured_hits * 2:
            return "unstructured"
        return "hybrid"

    if structured_hits > 0:
        return "structured"
    if unstructured_hits > 0:
        return "unstructured"
    return None

```



```

def _step2_schema_based(self, query):
    """DB 스키마 컬럼명 매칭으로 경로를 결정한다."""
    query_lower = query.lower()
    for term in SCHEMA_TERMS:
        if term in query_lower:
            return SCHEMA_TERMS[term]
    return None

def _step3_llm_based(self, query):
    """LLM에게 질문 분류를 위임한다."""
    prompt = f"""다음 질문을 아래 세 가지 유형 중 하나로 분류하세요.

질문: {query}

유형:
- structured: 숫자, 통계, 목록 등 데이터베이스 조치가 필요한 질문
- unstructured: 절차, 정책, 설명 등 문서 검색이 필요한 질문
- hybrid: 두 가지가 모두 필요한 복합 질문

반드시 JSON 형식으로만 답하세요:
{{"route": "structured|unstructured|hybrid", "reason": "한 줄 근거"}}"""

    try:
        response = self._llm.invoke(prompt)
        content = (
            response.content
            if hasattr(response, "content")
            else str(response)
        )
        # <think> 태그 제거 (DeepSeek-R1 등)
        content = re.sub(r"<think>.*?</think>", "", content, flags=re.DOTALL).strip()
        # JSON 추출
        json_match = re.search(r"\{.*\}", content, re.DOTALL)
        if json_match:
            parsed = json.loads(json_match.group())
            route = parsed.get("route", "unstructured")
            if route in ("structured", "unstructured", "hybrid"):
                return route
    except Exception:
        pass
    return None

```

3단계는 순서가 중요합니다. 확실한 신호(키워드)를 먼저 확인하고 모호할 때만 더 비싼 방법(LLM 호출)을 씁니다. 대부분은 1단계에서 끝나요.

2.6 실습 2: @tool 데코레이터 — 에이전트용 도구 만들기 (mcp_tools.py)

에이전트가 데이터베이스나 문서 저장소에 접근할 때 쓰는 도구입니다. @tool 데코레이터만 붙이면 에이전트가 스스로 판단해서 도구를 사용합니다.

아래 코드를 ex06/src/mcp_tools.py에 단계별로 나누어 작성합니다.

1단계: 도구(Tool) импорт 및 시스템 연결 준비

에이전트가 도구를 인식하게 해주는 tool 데코레이터와 데이터를 다룰 db_helper 함수를 가져옵니다.

```
"""ex06 - MCP 도구 모듈."""

from langchain_core.tools import tool
from src.db_helper import run_query, DB_ERROR_MSG, get_vectorstore
```

2단계: 정형 데이터(DB) 조회 도구 만들기

PostgreSQL에서 데이터를 가져오는 세 가지 도구를 작성합니다. 여기서 Docstring(설명적 주석)이 매우 중요한데 에이전트가 이 내용을 읽고 어떤 도구를 호출할지, 인자(Arguments)는 무엇인지 파악하기 때문입니다.

```
@tool
def leave_balance(emp_no: str) -> dict:
    """직원의 연차 잔여 일수를 조회한다."""
    # ① DB 조회 시도 (이름 또는 번호)
    if emp_no.startswith("E") and emp_no[1:].isdigit():
        rows = run_query(
            """
            SELECT e.emp_no, e.name, e.department,
                   l.total_days, l.used_days,
                   (l.total_days - l.used_days) AS remaining_days
            FROM employees e
            LEFT JOIN leave_balance l ON e.emp_no = l.emp_no
            WHERE e.emp_no = %s
            """,
            (emp_no,),
        )
    else:
        rows = run_query(
            """
            SELECT e.emp_no, e.name, e.department,
                   l.total_days, l.used_days,
                   (l.total_days - l.used_days) AS remaining_days
            FROM employees e
            LEFT JOIN leave_balance l ON e.emp_no = l.emp_no
```

```

        WHERE e.name LIKE %s
        """
        (f"%{emp_no}%",),
    )

# ② DB 결과가 있으면 반환
if rows:
    return rows[0]

# ③ DB 연결 실패 시 에러 반환
return {"error": f"직원 '{emp_no}'을(를) 찾을 수 없습니다. {DB_ERROR_MSG}"}

@tool
def sales_sum(dept: str = "", start_date: str = "", end_date: str = "") -> dict:
    """부서별 또는 전체 매출 합계를 조회한다."""
    # ① 파라미터 기본값 처리
    start = start_date or "2024-11-01" # ①
    end = end_date or "2024-12-31"

    # ② DB 조회 시도
    dept_filter = f"AND e.department LIKE '%{dept}%' " if dept else ""
    rows = run_query(
        f"""
        SELECT e.department, e.name AS employee_name,
               SUM(s.amount) AS total_amount, COUNT(*) AS record_count
        FROM sales s
        JOIN employees e ON s.emp_no = e.emp_no
        WHERE s.sale_date BETWEEN %s AND %s {dept_filter}
        GROUP BY e.department, e.name
        ORDER BY total_amount DESC
        """
        ,
        (start, end),
    )

    # ③ DB 결과 가공
    if rows:
        grand_total = sum(int(r.get("total_amount") or 0) for r in rows)
        return {
            "total_amount": grand_total,
            "record_count": len(rows),
            "dept_filter": dept or "전체",
            "period": f"{start} ~ {end}",
            "top5": rows[:5],
        }

    # ④ DB 연결 실패 시 에러 반환

```

```

    return {"error": DB_ERROR_MSG, "dept_filter": dept or "전체", "period": f"{start} ~ {end}"}

@tool
def list_employees(dept: str = "", name: str = "") -> dict:
    """직원의 기초 정보(부서, 직급, 입사일 등) 범용 목록을 조회한다.
    특정 직원의 입사일이나 기본 정보가 궁금할 때에는 name(이름)에 직원이름을 넣어 검색한다."""
    # ① DB 조회 시도
    conditions = []
    params = []
    sql_base = "SELECT emp_no, name, department, position, hire_date FROM employees "

    if dept:
        conditions.append("department LIKE %s")
        params.append(f"%{dept}%")
    if name:
        conditions.append("name LIKE %s")
        params.append(f"%{name}%")

    if conditions:
        sql = sql_base + " WHERE " + " AND ".join(conditions) + " ORDER BY name"
    else:
        sql = sql_base + " ORDER BY department, name"

    rows = run_query(sql, tuple(params))

    # ② DB 결과 반환
    if rows:
        return {"employees": rows, "count": len(rows), "filter": {"dept": dept, "name": name}}

    # ③ DB 연결 실패 시 에러 반환
    return {"error": DB_ERROR_MSG, "employees": [], "count": 0, "dept_filter": dept or "전체"}

```

3단계: 비정형 데이터(문서) 검색 도구 만들기

마지막으로 ChromaDB를 이용해 사내 문서를 벡터 검색하는 도구를 작성합니다.

```

@tool
def search_documents(query: str, k: int = 3) -> dict:
    """사내 문서에서 관련 내용을 벡터 검색한다."""
    collection = get_vectorstore()
    if collection is not None:
        try:
            results = collection.query(query_texts=[query], n_results=k)

```

```

docs = []
for i, doc in enumerate(results["documents"][0]):
    docs.append({
        "content": doc,
        "source": results["metadatas"][0][i].get("source", "unknown"),
        "score": round(1 - results["distances"][0][i], 4),
    })
return {"results": docs, "total_found": len(docs)}
except Exception:
    pass

return {"results": [], "total_found": 0}

# -----
# 도구 목록 (에이전트에 전달)
# -----

ALL_TOOLS = [leave_balance, sales_sum, list_employees, search_documents]

```

마지막 줄의 `ALL_TOOLS` 는 네 가지 도구를 하나의 리스트로 묶은 것입니다. 이 리스트를 에이전트에게 건네면 에이전트는 “내가 쓸 수 있는 도구가 이 네 가지구나”라고 인식합니다.

에이전트가 어떤 도구를 호출할지는 “"""부서별 또는 전체 매출 합계를 조회한다."""” 같은 독스트링(Docstring)을 보고 결정합니다. `mcp_tools.py` 에서 데이터를 다루는 인프라 함수는 `db_helper.py` 에 분리해 두었습니다.

파일	함수	역할
<code>db_helper.py</code>	<code>run_query(sql, params)</code>	PostgreSQL 쿼리 실행 및 결과 반환
<code>db_helper.py</code>	<code>get_vectorstore()</code>	ChromaDB 벡터스토어 연동 객체 반환

2.7 실습 3: ReAct 에이전트 조립하기 (agent.py)

앞서 만든 라우터와 도구를 합쳐서 완성형 어시스턴트를 만듭니다.

아래 코드를 `ex06/src/agent.py`에 단계별로 나누어 작성합니다.

1단계: 프롬프트 및 도구 импорт

에이전트의 성격을 규정하는 **시스템 프롬프트(System Prompt)** 와 앞서 만든 라우터, 도구를 가져옵니다. 시스템 프롬프트는 LLM에게 “당신은 누구이고, 무엇을 할 수 있고, 어떤 규칙을 지켜야 하는지”를 알려주는 지시문입니다. 도구 목록과 사용 규칙을 여기에 적어 두면 에이전트가 참고해서 행동합니다.

```

"""ex06 - 통합 에이전트 모듈."""

```

```

from src.llm_factory import build_llm

```

```

from src.agent_helpers import (
    parse_agent_result,
    serialize_steps,
    clean_think_tags,
    fallback_response,
)

from src.mcp_tools import ALL_TOOLS
from src.router import QueryRouter

# -----
# 시스템 프롬프트
# -----

SYSTEM_PROMPT = """당신은 사내 HR 및 업무 질문에 답변하는 AI 어시스턴트입니다.

사용 가능한 도구:
- leave_balance: 직원 연차 잔여 조회 (emp_no 또는 이름으로 검색)
- sales_sum: 매출 합계 조회 (부서, 기간 필터 가능)
- list_employees: 직원 목록 조회 (부서 필터 가능)
- search_documents: 사내 문서 검색 (절차, 정책, 안내 등)

규칙:
1. 정형 데이터(숫자/통계/목록)는 DB 조회 도구를 사용하세요.
2. 비정형 질문(절차/정책/설명)은 search_documents를 사용하세요.
3. 복합 질문은 두 종류의 도구를 모두 사용하세요.
4. 답변은 반드시 한국어로 작성하세요.
5. 도구 실행 결과의 핵심 정보만 추출하여 자연스러운 문장으로 답변하세요. 원본 JSON이나 디렉터리를 절대 그대로 출력하지 마세요."""

```

2단계: 에이전트 생성

앞에서 설명한 도구 호출(Tool Calling)을 실제로 구현하는 단계입니다. `create_tool_calling_agent` 는 LLM에게 “이 도구를 쓸 수 있다”고 알려주는 에이전트를 만들고 `AgentExecutor` 는 이 에이전트의 “생각 -> 도구 호출 -> 결과 확인” 루프를 반복 실행하는 실행 기입니다.

```

# -----
# 통합 에이전트 클래스
# -----

class IntegratedAgent:
    """정형 + 비정형 통합 ReAct 에이전트."""

    def __init__(self, llm=None):
        """에이전트를 초기화한다."""
        self._llm = llm or build_llm()
        self._router = QueryRouter(llm=self._llm)

```

```

self._agent_executor = self._build_agent_executor()

def _build_agent_executor(self):
    """LangChain AgentExecutor를 생성한다."""
    try:
        from langchain.agents import AgentExecutor, create_tool_calling_agent
        from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

        # ① 프롬프트 구성 (system + input + scratchpad)
        prompt = ChatPromptTemplate.from_messages([
            ("system", SYSTEM_PROMPT),
            MessagesPlaceholder(variable_name="chat_history", optional=True),
            ("human", "{input}"),
            MessagesPlaceholder(variable_name="agent_scratchpad"),
        ])

        # ② Tool Calling Agent 생성
        agent = create_tool_calling_agent(
            llm=self._llm,
            tools=ALL_TOOLS,
            prompt=prompt,
        )

        # ③ AgentExecutor 래핑 (중간 단계 반환 활성화)
        return AgentExecutor(
            agent=agent,
            tools=ALL_TOOLS,
            verbose=False,
            return_intermediate_steps=True,
            max_iterations=10,
            handle_parsing_errors=True,
        )
    except Exception as e:
        print(f"[경고] AgentExecutor 초기화 실패: {e}. 폴백 모드로 동작합니다.")
        return None

```

`agent_scratchpad` 는 ReAct 루프의 중간 기록이 쌓이는 공간입니다. 에이전트가 `leave_balance` 호출 → 결과 확인 → `search_documents` 호출 → 결과 확인 과정을 여기에 적어두면서 다음 행동을 결정합니다.

`IntegratedAgent` 에서 쓰는 인프라 함수는 별도 모듈로 분리해 두었습니다.

파일	함수	역할
<code>llm_factory.py</code>	<code>build_llm()</code>	<code>LLM_PROVIDER</code> 환경변수에 따라 Ollama/OpenAI 중 선택
<code>agent_helpers.py</code>	<code>parse_agent_result(steps)</code>	중간 단계에서 DB 결과 / 문서 결과를 분리 추출
<code>agent_helpers.py</code>	<code>serialize_steps(steps)</code>	도구 호출 로그를 JSON 직렬화 가능한 형태로 변환

3단계: 라우팅 및 답변 생성 로직 실행

사용자의 질문이 들어왔을 때 맨 먼저 **QueryRouter** 를 태우고, **AgentExecutor** 에 전달해 답변을 받아내는 최종 파이프라인 메서드를 작성합니다.

```
def run(self, query):
    """질문을 처리하고 통합 응답을 반환한다."""
    # ① 질문 유형 미리 분류 (라우터 활용)
    query_type = self._router.classify_query(query) # ①

    # ② 에이전트 실행
    if self._agent_executor is None:
        return fallback_response(self._llm, query, query_type)

    try:
        result = self._agent_executor.invoke({"input": query}) # ②
        answer = result.get("output", "답변을 생성하지 못했습니다.")
        steps = result.get("intermediate_steps", [])

        # ③ 잡음(Think 태그) 제거
        answer = clean_think_tags(answer)

        # ④ 사용한 데이터 기록
        structured_data, unstructured_data = parse_agent_result(steps)

        return {
            "answer": answer,
            "query_type": query_type,
            "structured_data": structured_data,
            "unstructured_data": unstructured_data,
            "steps": serialize_steps(steps), # 디버깅용
        }
    except Exception as e:
        return {
            "answer": f"처리 중 오류가 발생했습니다: {e}",
            "query_type": query_type,
            "structured_data": {},
            "unstructured_data": [],
            "steps": [],
        }
}
```

run 메서드는 사용자의 질문이 들어왔을 때 가장 먼저 실행되는 관문입니다. 흐름을 정리하면 이렇습니다.

1. **질문 분류** (**_router.classify_query**): **QueryRouter** 를 돌려서 이 질문이 정형(DB 대상)인지 비정형(문서 대상)인지 복합인지 판단합니다. 에이전트가 어떤 도구를 집중적으로 써야 할지 미리 힌트를 얻는 셈입니다.

2. **에이전트 실행 (invoke):** `AgentExecutor` 에 질문을 넘기면 에이전트가 도구를 골라 실행하며 ReAct 루프를 돕니다. 최종 답변과 중간 단계 기록을 모아줍니다.
3. **잡음 제거 및 기록 추출:** DeepSeek-R1 같은 사색형(Reasoning) 모델은 `<think>` 태그 내부에 사고 과정을 남기는데 이를 지워 주고 로깅 목적으로 호출된 DB와 문서 내역을 정리해서 반환합니다.

2.8 실행 결과

실습 환경 구축을 마쳤다면 서버를 실행합니다.

```
# 실행
python run.py
```

브라우저에서 <http://localhost:8000>으로 접속하면 채팅 화면이 열립니다.

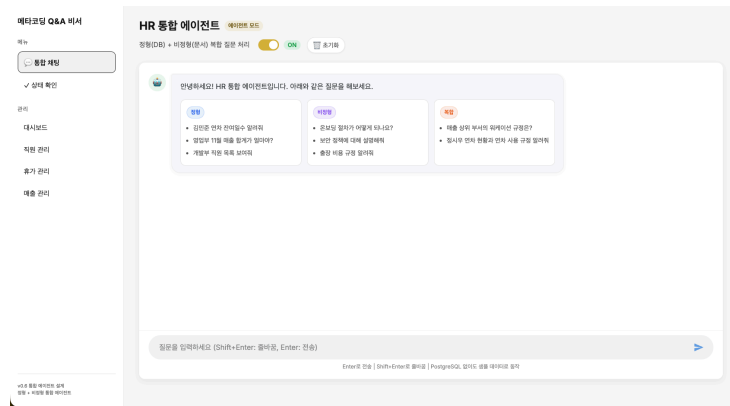


그림 6-2: 로컬호스트로 구동된 에이전트 채팅 인터페이스 화면

이번에는 채팅부터 시작하지 않고 **관리자 대시보드** 에서 데이터를 직접 넣어보겠습니다.

신입사원 등록

사이드바에서 **직원 관리**를 클릭합니다. `/admin/employees` 페이지가 열립니다.

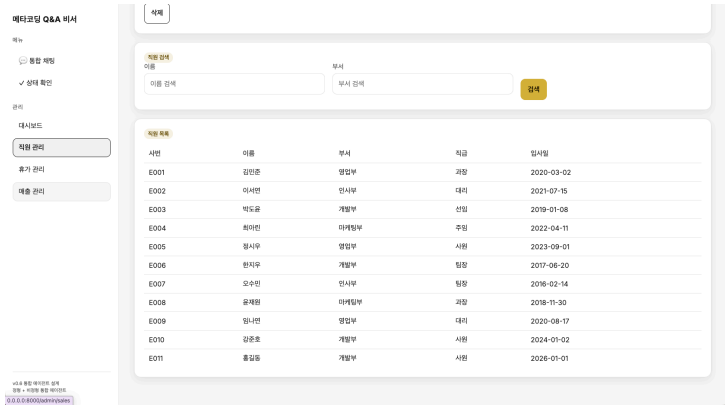


그림 6-3: 관리자 대시보드 — 직원 관리 화면. 사번은 자동 생성된다.

신입사원을 등록합니다.

- 이름: 홍길동
- 부서: 개발부
- 직급: 사원
- 입사일: 2026-01-01

등록 버튼을 누르면 사번이 자동으로 부여됩니다. 기존 데이터에 E001~E010이 있으니 **E011**이 됩니다.

채팅으로 확인

이제 사이드바에서 **통합 채팅**을 클릭합니다. 채팅창에 이렇게 입력합니다.

홍길동 입사일 언제야?

잠시 기다리면 에이전트가 답합니다.

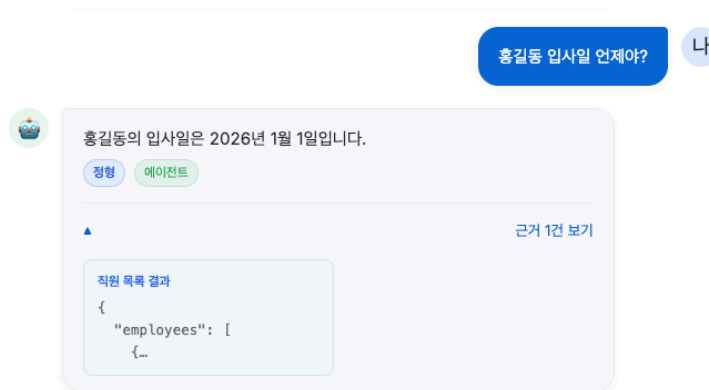
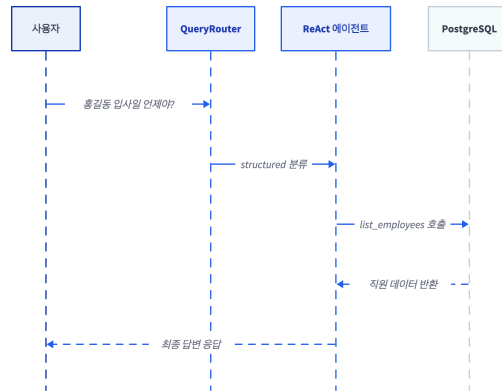


그림 6-4: 방금 등록한 신입사원의 정보를 에이전트가 DB에서 찾아 답한다.

방금 대시보드에서 등록한 데이터를 에이전트가 찾아냈습니다. 무슨 일이 벌어진 건지 시퀀스 다이어그램으로 뜯어보겠습니다.



시퀀스 다이어그램 — 정형 데이터 조회 흐름

1. QueryRouter가 “입사일”이라는 키워드를 보고 **structured (정형)** 로 분류했습니다.
2. ReAct 에이전트가 **list_employees** 도구를 골랐습니다.
3. **list_employees** 가 PostgreSQL에서 “홍길동”을 찾아 결과를 돌려줬습니다.
4. 에이전트가 결과를 읽고 자연어 답변을 만들었습니다.

대시보드에서 데이터를 넣고 채팅에서 바로 물어보는 것, DB와 AI가 연결된 에이전트의 핵심입니다.

복합 질문도 테스트

이번에는 DB와 문서를 동시에 써야 하는 질문을 던져보겠습니다.

홍길동 연차 몇 일 남았어? 그리고 연차 신청 규정은?

홍길동님의 휴가 잔여 일수는 15일입니다.

또한, 연차 사용 규정은 다음과 같습니다.

- 연차는 최대 15일을 사용할 수 있습니다.
- 연차 신청은 AI HR 봇을 통해 신청이 가능합니다.
- 시스템은 팀원 간의 일정 충돌 여부를 자동으로 확인합니다.

아래와 같이 연차 사용에 대한 추가 정보를 제공합니다.

- 연차, 반차, 병가: 별도 승인 절차 없이 이용할 수 있습니다.
- 위케이션 기간 동안 발생하는 숙박비의 50% (최대 100만원 한도)를 회사가 바우처 형태로 지원합니다.

더욱 자세한 내용은 사내 규정 및 정책 문서를 참조하시기 바랍니다.

복합 에이전트

근거 4건 보기

HR_취업규칙_v1.0

제1차 1차입규칙 (다 단 편 집 행)문서번호: HR-2026-001 버전: v2.0 (Draft) 대외비 (Confidential) 4. 휴가 및 리 프 레 시 (Leave & Refresh) 4.1 스마트 휴 가 승 인 (Smart Approval) 메타프롬프트는 구성원의 자 율 행 동 ...

HR_취업규칙_v1.0

각 인, 임 무 누 락 방 지 와 리 소 스 조 출을 위한 정보 공유 목적 입 니 다. 4.2 별 가 및 건 강 관 리 본 인 또는 가 족의 불 병 으 로 인 해 요 양 이 필 요한 경우, 연 간 최 대 60일의 유 급 방 가 를 사 용 할 수 있습 니 다. 3일 미만 : 준 병 서 용 입 ...

HR_취업규칙_v1.0

수 있습 니 다. 지 방 사원 위 커 미 션 기 간 동안 발 생 하 는 숙 박 비의 50%(연 간 최 대 100만원 한 도)를 회 사 에서 바 우 처 형 태 로 지 원 합 니 다. 또한, 한 자 재 사의 한 달 한 입 무 환 경 제 입 을 위 해 프 로 미 트 하 니 다. 노 이 즈 캔 슬 링 제 도 ...

get_leave_balance

```
{
  "employee_name": "홍길동",
  "dept": "개발부", ...
}
```

그림 6-7: 연차 잔여(DB)와 신청 규정(문서)을 동시에 가져온다.

에이전트가 **leave_balance** → **search_documents** 순서로 두 도구를 호출했습니다. 한 번의 질문으로 DB 조회와 문서 검색을 동시에 해낸 겁니다.

Ctrl + C를 눌러 서버를 종료합니다. Docker 컨테이너도 **docker compose down**으로 정리합니다.

2.9 더 알아보기

QueryRouter 없이 에이전트만 써도 되지 않나요?

에이전트에게 모든 판단을 맡기면 매번 LLM 호출이 발생합니다. 간단한 “연차 조회” 질문에도 “어떤 도구를 쓸까?” 고민하는 LLM 비용이 붙습니다. QueryRouter가 빠르게 사전 분류를 해주면 에이전트가 처음부터 방향을 잡고 시작할 수 있어요.

Ollama와 OpenAI 중 어떤 걸 써야 하나요?

에이전트에서는 Tool Calling이 핵심입니다. LLM이 “지금 어떤 도구를 써야지”를 스스로 결정해야 하거든요. OpenAI의 **gpt-4o-mini**는 Tool Calling 성능이 검증되어 있어 결과가 안정적입니다. Ollama를 쓸 때는 반드시 Tool Calling을 지원하는 모델(**llama3.1:8b**)을 골라야 합니다. `.env`에서 **LLM_PROVIDER=openai**로 바꾸면 나머지 코드는 그대로입니다. `build_llm()` 함수가 환경 변수를 보고 알아서 LLM을 바꿔주거든요.

2.10 이것만은 기억하세요

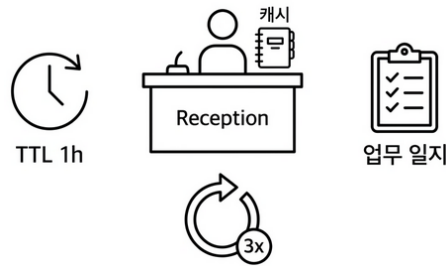
- **AI 비서는 안내데스크입니다.** 질문을 듣고 맞는 담당자(도구)를 찾아 정보를 가져옵니다.
- QueryRouter가 질문을 분류하고 **@tool** 데코레이터가 도구를 만들고 ReAct 에이전트가 반복 실행하면서 답을 완성합니다.

다음 챕터(ex07)에서는 이 에이전트를 실제로 운영하면서 생기는 문제를 다룹니다. 같은 질문에 매번 LLM을 호출하는 비용 문제, 응답이 느려지는 속도 문제, 그리고 누가 얼마나 쓰는지 모르는 추적 문제입니다.

Ch.7: “실제로 써보니” — 운영을 위한 안정화 (ex07)

한 줄 요약: 사서에게 기억력과 업무 일지를 줬다. 같은 질문엔 바로 답하고, 하루 몇 건 처리했는지 기록한다.

핵심 개념: ResponseCache(TTL), EmbeddingCache, TokenTracker, ConnectHRAgent



운영 안정화 — 캐시와 모니터링

1.1 “왜 이렇게 느려?”

CH06에서 통합 에이전트를 완성했습니다. 동료들도 만족했구요. DB 조회도 되고 문서 검색도 되고 출처까지 붙여서 답변합니다. 그런데 일주일쯤 지나자 불만이 들어오기 시작합니다.

동료 A: “병가 증빙 필요하지 아까도 물어봤는데, 또 물어보니까 또 20초 걸리네요.”

동료 B: “우리 팀에서 이거 하루에 30번은 쓰는데, 비용이 얼마나 나가는 건지 모르겠어요.”

같은 질문을 반복해도 매번 LLM을 호출합니다. 로컬 모델 기준 10~20초씩 걸리고, API를 쓰면 돈도 듭니다. 누가 얼마나 쓰는지 추적할 방법이 없었습니다. 문제는 또 있습니다. LLM이 가끔 에러를 냅니다. 네트워크 타임아웃이나 파싱 실패 같은 것인데, ex06에서는 에러가 나면 그대로 멈춰버렸습니다.

1.2 사서에게 메모장을 하나 더 줬다

CH05에서 사서에게 대화용 메모장(WindowMemory)을 줬습니다. 최근 5턴을 기억하는 메모장이었죠. 이번에는 다른 종류의 메모장을 추가합니다.

답변 메모장 (ResponseCache)

누군가 같은 질문을 또 했습니다. 사서가 이미 한 번 찾아본 자료라면 서가까지 다시 갈 필요가 없겠죠. 메모장에 적어둔 답을 바로 읽어주면 됩니다. 다만 메모장에는 유통기한이 있습니다. 1시간이 지나면 메모를 지워요. 사내 규정이 바뀌었을 수도 있으니까요. 이것 **TTL(Time To Live)** 이라고 부릅니다.

임베딩 메모장 (EmbeddingCache)

사서가 서가에서 자료를 찾으려면 질문 문장을 벡터(숫자 배열)로 바꿔야 합니다. 이 변환에도 시간이 걸려요. 같은 문장을 여러 번 변환할 필요는 없겠죠. 한 번 계산한 벡터를 파일로 저장해두면 다음엔 바로 꺼내 씁니다.

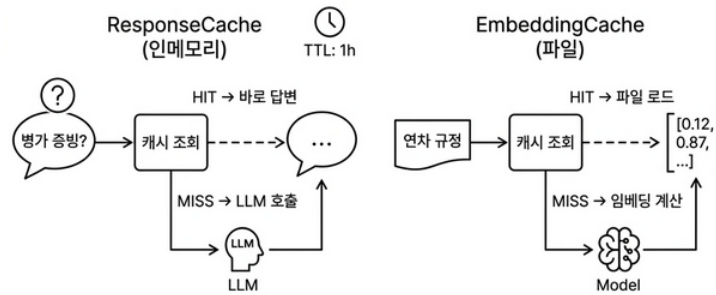


그림 7-1: 같은 질문이 또 오면 LLM을 다시 부르지 않는다. 같은 텍스트를 또 임베딩하지 않는다.

1.3 업무 일지를 쓰기 시작했다

도서관에 이용 통계 시스템이 없다고 생각해 봅니다. 사서는 하루 종일 자료를 찾아주는데 관장이 “이번 달 이용자가 몇 명이야?”라고 물으면 대답을 못 합니다. 기록이 없으니까요.

우리 시스템도 마찬가지였습니다. 동료 B가 “비용이 얼마나 나가는 건지 모르겠어요”라고 했을 때 답할 수가 없었어요. LLM 호출 한 번에 토큰이 얼마나 들고 하루에 총 몇 건 처리하는지 아무도 몰랐습니다.

그래서 사서에게 업무 일지를 쓰게 했습니다.

- 오늘 총 몇 건 처리했는지
- 입력/출력 토큰을 얼마나 썼는지
- 평균 응답 시간은 얼마인지
- API 비용은 얼마나 나갔는지

실무에서는 **Langfuse** 나 **LangSmith** 같은 전문 모니터링 도구가 이 역할을 합니다. 호출별 토큰 수와 비용, 지연 시간을 자동으로 수집해서 대시보드로 보여주죠. 하지만 도구를 붙이기 전에 “무엇을 측정해야 하는지”부터 이해해야 합니다. 그래서 이 책에서는 **TokenTracker** 라는 간단한 클래스를 직접 만들어 봅니다. 매 호출마다 한 줄씩 기록이 쌓이고 나중에 “이번 달 토큰 비용이 얼마야?”라고 물으면 바로 답할 수 있어요. 도서관 이용 통계처럼 운영을 개선하려면 먼저 측정부터 해야 합니다.

1.4 재시도도 배웠다

도서관 사서가 서가에서 자료를 꺼내려는데 하필 그 선반이 점검 중입니다. ex06의 사서는 “지금 못 찾겠어요”라고 말하고 돌아섰습니다. 이용자는 같은 질문을 처음부터 다시 해야 했죠.

LLM도 마찬가지입니다. 네트워크 오류나 타임아웃, 파싱 실패 등으로 가끔 응답을 못 합니다. ex06에서는 에러가 나면 그대로 멈췄습니다. 이제 사서에게 규칙을 하나 알려줬습니다. “한 번 실패하면 **3번까지 다시 시도해**. 시도 사이에는 2초씩 쉬고.” 점검 중이던 선반도 잠깐 뒤에 다시 가면 열려 있을 수 있으니까요. 이렇게 캐시와 업무 일지, 재시도까지 운영에 필요한 안정 장치를 모두 갖추게 됐습니다.

1.5 이번 버전에서 뭘 만드나

ex07에서는 ex06의 에이전트를 운영 가능한 수준으로 감쌉니다.

기능	비유	코드
응답 캐시	사서의 답변 메모장 (1시간 유효)	<code>cache.py</code>

기능	비유	코드
임베딩 캐시	단어→벡터 변환 결과 저장	<code>cache.py</code>
토큰 추적	사서의 업무 일지	<code>monitoring.py</code>
재시도 로직	“3번까지 다시 시도”	<code>agent_config.py</code>

새 기능을 추가하는 게 아닙니다. 기존 에이전트를 **안정적으로 운영** 할 수 있게 감싸는 것이죠. 도구도 한 가지 바꿉니다. ex06에서 `mcp_tools.py` 하나에 몰아넣었던 도구 4개를 `tools/` 디렉토리 아래 파일 하나씩으로 분리했습니다. 도구가 늘어나도 관리가 수월해 집니다.

이제 실습으로 캐시, 비용 추적, 에러 처리를 적용해보겠습니다.

2.1 용어 정리

이야기 속 표현	진짜 이름	정의
답변 메모장	<code>ResponseCache</code>	TTL(유효시간) 기반 인메모리 응답 캐시. SHA-256 해시 키로 질문을 식별하고, 만료 전까지 동일 질문에 캐시된 답변을 반환한다
임베딩 메모장	<code>EmbeddingCache</code>	파일 기반 임베딩 벡터 캐시. pickle로 벡터를 저장하고 프로세스 재시작 후에도 유지된다
업무 일지	<code>TokenTracker</code>	LLM 호출별 입출력 토큰 수, 비용, 응답 시간을 누적 집계하는 추적기
캐시+재시도가 붙은 사서	<code>ConnectHRAgent</code>	ex06의 <code>IntegratedAgent</code> 에 캐시, 모니터링, 재시도를 통합한 운영용 에이전트
3번까지 다시 시도	Retry 로직	LLM 호출 실패 시 최대 N회까지 재시도하는 안정성 패턴. 재시도 사이에 대기 시간 포함

2.2 파일 계층 구조

```

ex07/
├── run.py           서버 실행 진입점
├── src/
│   ├── agent_config.py  [실습] ConnectHRAgent - 캐시/모니터링/재시도 통합
│   ├── cache.py         [실습] ResponseCache(TTL) + EmbeddingCache ← 신규
│   ├── monitoring.py    [실습] TokenTracker + setup_logging ← 신규
│   ├── llm_factory.py   [참고] LLM 인스턴스 생성 (CH05~06 패턴 동일)
│   └── agent_helpers.py  [참고] RAG 체인 구성 + 라우팅 매핑

```

```

|   └── router.py          [참고] 3단계 QueryRouter (CH06에서 이미 학습)
|   └── tools/
|       ├── __init__.py    [참고] 도구 패키지 초기화
|       ├── leave_balance.py [참고] 연차 조회 도구
|       ├── sales_sum.py   [참고] 매출 합계 도구
|       ├── list_employees.py [참고] 직원 목록 도구
|       └── search_documents.py [참고] 문서 검색 도구
└── app/
    ├── main.py            [참고] FastAPI 앱 진입점
    ├── chat_api.py        [참고] Agent API 엔드포인트
    ├── database.py        [참고] PostgreSQL 연결
    ├── templates/
    │   └── chat.html      [참고] 채팅 웹 UI
    └── static/
        ├── css/chat.css  [참고] UI 스타일
        └── js/chat.js     [참고] 채팅 로직

```

2.3 실습 환경 구축

기본 환경(Python 3.12, Ollama, Docker)이 없다면 **부록(환경 설정)** 을 먼저 참고하세요.

```

cd ex07
cp .env.example .env
python3.12 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
docker compose up -d      # PostgreSQL 실행
pip install -r requirements.txt

```

이전 챕터 Docker 종료: CH06의 Docker가 실행 중이라면 `cd ex06 && docker compose down` 으로 먼저 종료하세요.

패키지	역할
<code>langchain</code>	체인/에이전트 프레임워크
<code>langchain-ollama</code>	Ollama LLM 연결
<code>langchain-chroma</code>	ChromaDB Retriever
<code>sentence-transformers</code>	ko-sroberta 임베딩
<code>psycopg2-binary</code>	PostgreSQL 드라이버

패키지	역할
<code>fastapi</code>	웹 API 서버

2.4 실습 순서

ex07에서 새로 추가되는 파일은 `cache.py` 와 `monitoring.py` 두 개뿐입니다. 나머지는 ex06에서 가져와 구조만 정리한 겁니다. 핵심 코드를 아래 순서로 살펴봅니다.

1. `agent_config.py` — 캐시·모니터링·재시도를 통합한 `ConnectHRAgent`. ex07의 중심입니다.
2. `cache.py` — `agent_config.py`의 ①(캐시 조회)과 ⑤(캐시 저장)에서 사용하는 모듈입니다. ex07에서 새로 만듭니다.
3. `monitoring.py` — ④(토론 추적)에서 사용하는 모듈입니다. 역시 ex07에서 새로 만듭니다.
4. 실행 결과 확인 — `python run.py` 로 서버를 띄우고 채팅창에서 캐시 적중을 확인합니다.

2.5 실습 1: ConnectHRAgent — 캐시/모니터링/재시도 통합 (agent_config.py)

이 파일이 ex07의 핵심입니다. ex06의 `IntegratedAgent` 에 캐시와 모니터링, 재시도를 엮은 운영용 에이전트죠. 아래 코드를 ex07/`src/agent_config.py` 에 단계별로 나누어 작성합니다.

1단계: 임포트와 운영 설정 상수

```
import os
import time

from langchain.agents import AgentExecutor, create_tool_calling_agent
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

from .cache import response_cache
from .monitoring import token_tracker
from .tools import get_leave_balance, get_sales_sum, list_employees, search_documents
from .router import QueryRouter
from .llm_factory import build_llm
from .agent_helpers import build_rag_chain, classify_route

# --- 운영 설정 상수 ---
AGENT_MAX_ITERATIONS = 10      # Agent 최대 반복 횟수
AGENT_TIMEOUT_SECONDS = 60     # 실행 최대 대기 시간 (초)
RETRY_MAX_ATTEMPTS = 3         # 최대 재시도 횟수
RETRY_DELAY_SECONDS = 2.0     # 재시도 간격 (초)
```

ex06에는 없던 네 줄이 추가됐습니다. 타임아웃, 재시도 횟수, 재시도 간격을 상수로 빼둔 것이 운영 안정성의 시작입니다. `cache`, `monitoring`, `llm_factory`, `agent_helpers` 같은 모듈도 임포트하는데, ex06에서 한 파일에 몰아넣었던 기능을 역할별로 분리한 겁니다.

참고 파일: `llm_factory.py` 는 LLM 인스턴스 생성(Ollama/OpenAI 분기), `agent_helpers.py` 는 RAG 체인 구성과 라우팅 매핑을 담당합니다. 둘 다 CH05~06에서 이미 학습한 패턴과 동일하므로 코드 설명은 생략합니다.

2단계: ConnectHRAgent 클래스 — 초기화와 AgentExecutor 구성

```

SYSTEM_PROMPT = """당신은 사내 AI 비서입니다. ...""" # ex06과 동일

class ConnectHRAgent:
    def __init__(self):
        self.llm = build_llm()
        self._router = QueryRouter(llm=self.llm)
        self.tools = [list_employees, get_leave_balance, get_sales_sum, search_documents]
        self.rag_chain = build_rag_chain(self.llm)
        self.agent_executor = self._build_agent_executor()

    def _build_agent_executor(self):
        prompt = ChatPromptTemplate.from_messages([
            ("system", SYSTEM_PROMPT),
            MessagesPlaceholder(variable_name="chat_history", optional=True),
            ("human", "{input}"),
            MessagesPlaceholder(variable_name="agent_scratchpad"),
        ])
        agent = create_tool_calling_agent(self.llm, self.tools, prompt)
        return AgentExecutor(
            agent=agent,
            tools=self.tools,
            max_iterations=AGENT_MAX_ITERATIONS,
            max_execution_time=AGENT_TIMEOUT_SECONDS,
            handle_parsing_errors=True,
            return_intermediate_steps=True,
            verbose=True,
        )

```

`__init__` 의 구조는 ex06과 같습니다. 달라진 점은 `_build_agent_executor()` 에서 `max_iterations` 와 `max_execution_time` 을 1단계의 상수로 설정한 부분이에요. 무한 루프나 지나치게 긴 실행을 막아줍니다.

3단계: `_run_with_retry` — 재시도 로직

```

def _run_with_retry(self, query, chat_history=None):
    for attempt in range(1, RETRY_MAX_ATTEMPTS + 1):
        try:
            result = self.agent_executor.invoke({

```

```

        "input": query,
        "chat_history": chat_history or [],
    })
    return result
except Exception:
    if attempt < RETRY_MAX_ATTEMPTS:
        time.sleep(RETRY_DELAY_SECONDS)
    return {"output": "3회 재시도 후 실패했습니다.", "intermediate_steps": []}

```

`for attempt in range(1, 4)` — 최대 3번 시도합니다. 실패하면 2초 쉬고 다시 돌려요. 3번 다 실패하면 에러 메시지를 반환합니다. 네트워크 지연이나 모델 과부하처럼 일시적으로 불안정한 상황에서 잘 먹히는 패턴입니다.

4단계: `run()` — 캐시 → 라우팅 → 실행 → 추적 → 저장

```

def run(self, query, chat_history=None, use_cache=True):
    start_time = time.time()

    # ① 캐시 조회
    if use_cache:
        cached = response_cache.get(query)
        if cached is not None:
            cached["from_cache"] = True
            return cached

    # ② Router로 경로 결정
    route = classify_route(query, router=self._router)

    # ③ 경로별 실행
    if route == "rag" and self.rag_chain is not None:
        answer = self.rag_chain.invoke(query)
        result = {"output": answer, "route": route, "intermediate_steps": []}
    else:
        result = self._run_with_retry(query, chat_history)
        result["route"] = route

    # ④ 토큰 사용량 기록
    latency_ms = (time.time() - start_time) * 1000
    provider = os.getenv("LLM_PROVIDER", "ollama").lower()
    if provider == "openai":
        model = os.getenv("OPENAI_MODEL", "gpt-4o-mini")
    else:
        model = os.getenv("OLLAMA_MODEL", "deepseek-r1:8b")
    token_tracker.record(
        model=model,
        input_tokens=len(query.split()) * 2,  # 추정값

```

```

        output_tokens=len(result["output"].split()) * 2,
        operation="agent_run",
        latency_ms=latency_ms,
    )

    # ⑤ 캐시 저장
    if use_cache:
        response_cache.set(query, result)

    return result

```

`run()` 메서드가 ex07의 핵심 흐름입니다. ①~⑤ 다섯 단계를 순서대로 따라가면 돼요.

- ①(캐시 조회)과 ⑤(캐시 저장) — 같은 질문이 캐시에 있으면 LLM을 호출하지 않고 바로 반환합니다. 질문이 30번 반복되면 첫 번째만 LLM을 부르고 나머지 29번은 캐시에서 가져옵니다.
- ④(토큰 추적) — 매 호출마다 토큰 수와 응답 시간을 기록합니다. `LLM_PROVIDER` 환경 변수를 확인해서 올바른 모델명을 넘기고, `TokenTracker`가 모델별 단가표를 갖고 있어서 OpenAI라면 실제 비용이 계산되고 로컬 모델이면 \$0입니다.

마지막으로 싱글턴 패턴을 추가합니다:

```

_agent_instance = None

def get_agent():
    global _agent_instance
    if _agent_instance is None:
        _agent_instance = ConnectHRAgent()
    return _agent_instance

```

2.6 실습 2: ResponseCache + EmbeddingCache (cache.py)

`run()` 의 ①, ⑤ 단계에서 사용하는 캐시 모듈입니다. ex07에서 새로 만드는 파일이에요. 아래 코드를 `ex07/src/cache.py` 에 단계별로 나누어 작성합니다.

1단계: 상수 정의

```

import hashlib
import pickle
import time
from pathlib import Path

DEFAULT_RESPONSE_TTL = 3600          # 응답 캐시 TTL (초): 1시간
DEFAULT_EMBEDDING_CACHE_DIR = ".outputs/embedding_cache"

```

2단계: ResponseCache — TTL 기반 인메모리 캐시

```
class ResponseCache:
    def __init__(self, ttl=DEFAULT_RESPONSE_TTL, max_size=1000):
        self.ttl = ttl
        self.max_size = max_size
        self._store = {}
        self._hits = 0
        self._misses = 0

    def _make_key(self, query, context=""):
        raw = f"{query}::{context}"
        return hashlib.sha256(raw.encode("utf-8")).hexdigest()

    def get(self, query, context=""):
        key = self._make_key(query, context)
        entry = self._store.get(key)
        if entry is None:
            self._misses += 1
            return None
        value, expires_at = entry
        if time.time() > expires_at:
            del self._store[key]
            self._misses += 1
            return None
        self._hits += 1
        return value

    def set(self, query, value, context=""):
        key = self._make_key(query, context)
        if len(self._store) >= self.max_size:
            oldest_key = min(self._store, key=lambda k: self._store[k][1])
            del self._store[oldest_key]
        self._store[key] = (value, time.time() + self.ttl)

    def stats(self):
        total = self._hits + self._misses
        hit_rate = (self._hits / total * 100) if total > 0 else 0.0
        return {
            "total_items": len(self._store),
            "hits": self._hits,
            "misses": self._misses,
            "hit_rate_percent": round(hit_rate, 2),
        }
```

`_store` 디렉터리에 (값, 만료시각) 튜플을 저장합니다. `get()` 할 때 현재 시각이 만료시각을 넘었으면 삭제하고요. 키는 `query::context` 문자열의 SHA-256 해시입니다. 1,000개가 꽉 차면 만료 시각이 가장 임박한 항목부터 지웁니다. `stats()` 에서 보이는 적중률 (`hit_rate`)이 높을수록 LLM 호출을 아끼고 있다는 뜻입니다.

3단계: EmbeddingCache — 파일 기반 임베딩 캐시

```
class EmbeddingCache:
    def __init__(self, cache_dir=DEFAULT_EMBEDDING_CACHE_DIR):
        self.cache_dir = Path(cache_dir)
        self.cache_dir.mkdir(parents=True, exist_ok=True)
        self._hits = 0
        self._misses = 0

    def _make_cache_path(self, text):
        key = hashlib.sha256(text.encode("utf-8")).hexdigest()
        return self.cache_dir / f"{key}.pkl"

    def get(self, text):
        cache_path = self._make_cache_path(text)
        if not cache_path.exists():
            self._misses += 1
            return None
        with open(cache_path, "rb") as f:
            embedding = pickle.load(f)
        self._hits += 1
        return embedding

    def set(self, text, embedding):
        cache_path = self._make_cache_path(text)
        with open(cache_path, "wb") as f:
            pickle.dump(embedding, f)
```

`ResponseCache`와 구조가 같습니다. 차이점은 메모리 대신 **파일**로 저장한다는 점이에요. 텍스트를 SHA-256 해시로 변환해서 파일 명으로 쓰고, 임베딩 벡터는 pickle로 직렬화합니다. pickle은 Python 객체를 바이트 형태로 변환해서 파일에 저장하는 표준 라이브러리입니다.

4단계: 싱글턴 인스턴스

```
response_cache = ResponseCache()
embedding_cache = EmbeddingCache()
```

모듈이 임포트될 때 인스턴스가 하나 생성됩니다. 어디서든 `from .cache import response_cache` 로 같은 인스턴스를 공유하게 되죠.

ResponseCache vs EmbeddingCache 비교

	ResponseCache	EmbeddingCache
저장 위치	메모리	파일 (.pkl)
재시작 시	사라짐	유지됨
TTL	있음 (기본 1시간)	없음 (영구)
용도	LLM 응답 재사용	임베딩 벡터 재계산 방지

2.7 실습 3: TokenTracker + JSON 로깅 (monitoring.py)

이야기 파트에서 소개한 Langfuse 같은 전문 도구의 원리를 이해하기 위해 직접 만드는 모듈입니다. `ex07/src/monitoring.py` 에 단계별로 작성해 봅니다.

1단계: TokenTracker — 토큰 사용량 추적기

```
from datetime import datetime, timezone

class TokenTracker:
    COST_PER_1K_TOKENS = {
        "gpt-4o-mini": {"input": 0.00015, "output": 0.0006},
        "deepseek-r1:8b": {"input": 0.0, "output": 0.0}, # 로컬: 무료
    }

    def __init__(self):
        self._records = []
        self._total_input_tokens = 0
        self._total_output_tokens = 0

    def record(self, model, input_tokens, output_tokens, operation="chat", latency_ms=0.0):
        cost_table = self.COST_PER_1K_TOKENS.get(model, {"input": 0.0, "output": 0.0})
        cost_usd = (
            (input_tokens / 1000 * cost_table["input"])
            + (output_tokens / 1000 * cost_table["output"])
        )
        self._records.append({
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "model": model,
            "input_tokens": input_tokens,
            "output_tokens": output_tokens,
            "cost_usd": round(cost_usd, 6),
        })
```

```

        "latency_ms": round(latency_ms, 2),
    })
    self._total_input_tokens += input_tokens
    self._total_output_tokens += output_tokens

def summary(self):
    total_cost = sum(r["cost_usd"] for r in self._records)
    avg_latency = (
        sum(r["latency_ms"] for r in self._records) / len(self._records)
        if self._records else 0.0
    )
    return {
        "total_calls": len(self._records),
        "total_tokens": self._total_input_tokens + self._total_output_tokens,
        "total_cost_usd": round(total_cost, 6),
        "avg_latency_ms": round(avg_latency, 2),
    }

```

`COST_PER_1K_TOKENS` 에 모델별 토큰 단가를 넣어줬습니다. 로컬 모델(deepseek-r1:8b)은 비용이 0이고 OpenAI API는 1,000토큰당 가격이 있어요. `record()` 로 호출마다 기록하고 `summary()` 로 전체 통계를 볼 수 있습니다.

2단계: `setup_logging` — JSON 구조화 로깅

```

import json
import logging

class JsonFormatter(logging.Formatter):
    def format(self, record):
        return json.dumps({
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
        }, ensure_ascii=False)

def setup_logging(level="INFO", use_json=True):
    root_logger = logging.getLogger()
    root_logger.setLevel(getattr(logging, level.upper()))
    root_logger.handlers.clear()
    formatter = JsonFormatter() if use_json else logging.Formatter(
        "%(asctime)s | %(levelname)-8s | %(name)s | %(message)s"
    )
    handler = logging.StreamHandler()
    handler.setFormatter(formatter)
    root_logger.addHandler(handler)

```


`setup_logging(use_json=True)` 로 설정하면 로그가 JSON 형식으로 출력됩니다.

```
{
  "timestamp": "2026-03-05T09:15:23Z",
  "level": "INFO",
  "logger": "src.agent_config",
  "message": "[ConnectHRAgent] 처리 완료 (경로: rag, 소요: 1523ms)"
}
```

운영 환경에서 로그를 Elasticsearch나 CloudWatch 같은 시스템으로 보낼 때 JSON 형식이 파싱하기 쉽습니다.

3단계: LangfuseMonitor (선택) + 싱글턴

```
class LangfuseMonitor:
    def __init__(self):
        self.enabled = False
        # Langfuse 패키지가 설치되어 있고 .env에 키가 설정된 경우만 활성화

    def trace(self, name, input_data, output_data, metadata=None):
        if not self.enabled:
            return
        # Langfuse 서버로 추적 데이터 전송

token_tracker = TokenTracker()
langfuse_monitor = LangfuseMonitor()
```

이야기 파트에서 소개한 **Langfuse** 와의 연동 래퍼입니다. Langfuse 패키지가 없어도 코드는 정상 동작해요. `.env` 에 `LANGFUSE_PUBLIC_KEY` 와 `LANGFUSE_SECRET_KEY` 를 설정하면 자동으로 활성화됩니다.

2.8 [참고] 도구 모듈화

ex06에서는 `mcp_tools.py` 하나에 도구 4개가 다 들어 있었습니다. ex07에서는 `tools/` 디렉토리 아래 파일 하나에 도구 하나씩 분리했어요. 도구가 늘어나도 파일만 추가하면 됩니다.

파일	도구	역할
<code>tools/leave_balance.py</code>	<code>get_leave_balance(name)</code>	직원 연차 잔여 조회
<code>tools/sales_sum.py</code>	<code>get_sales_sum(department)</code>	부서별 매출 합계 조회
<code>tools/list_employees.py</code>	<code>list_employees(department)</code>	직원 목록 조회
<code>tools/search_documents.py</code>	<code>search_documents(query)</code>	문서 벡터 검색

2.9 실행 결과

코드 작성이 끝났습니다. 서버를 띄우고 캐시와 토큰 추적이 실제로 동작하는지 확인해 봅니다.

```
python run.py
```

서버가 시작되면 브라우저에서 <http://localhost:8000> 으로 접속합니다.

1. 첫 질문 처리 — 에이전트가 도구를 호출한다

채팅창에서 “김민준 연차 잔여일수 알려줘”를 입력합니다. 터미널 로그를 보면 에이전트가 어떤 순서로 일하는지 한눈에 볼 수 있어요.

```
ex07 — 첫 질문 처리 로그

$ python run.py
INFO:src.agent_config: [ConnectHRAgent] 질문 수신: 김민준 연차 잔여일수 알려줘

> Entering new AgentExecutor chain...
INFO:httpx: HTTP Request: POST http://localhost:11434/api/chat "HTTP/1.1 200 OK"

Invoking: `get_leave_balance` with `{'employee_name': '김민준'}`

INFO:src.tools.leave_balance: [get_leave_balance] 조회 대상: 김민준
INFO:src.tools.leave_balance: [get_leave_balance] 결과: {'employee_name': '김민준', 'dept': '영업부', 'total_leaves': 15.0, 'used_leaves': 7.0, 'remaining_leaves': 8.0}
INFO:httpx: HTTP Request: POST http://localhost:11434/api/chat "HTTP/1.1 200 OK"

> Finished chain.
INFO:src.monitoring: [TokenTracker] 사용량 기록: model=llama3.1:8b, input=8, output=12, cost=$0.000000, latency=15933ms
INFO:src.cache: [ResponseCache] 저장: key=5e1dc59daa6e... (만료: 3600초 후)
INFO:src.agent_config: [ConnectHRAgent] 처리 완료 (경로: db, 소요: 15933ms)
INFO: 127.0.0.1:61781 - "POST /api/chat HTTP/1.1" 200 OK
```

그림 7-2: 첫 질문 처리 흐름. 에이전트가 도구를 호출하고, 토큰을 기록하고, 캐시에 저장한다.

로그를 위에서 아래로 읽어봅니다.

1. **질문 수신** — ConnectHRAgent가 질문을 받습니다.
2. **AgentExecutor 체인 시작** — LLM이 질문을 분석하고, `get_leave_balance` 도구를 선택합니다.
3. **도구 호출** — 김민준의 연차 데이터를 DB에서 조회합니다. 잔여 8일이 돌아옵니다.
4. **LLM 응답 생성** — 조회 결과를 바탕으로 자연어 답변을 만듭니다.
5. **TokenTracker 기록** — 입력 8토큰, 출력 12토큰, 소요 15,933ms를 기록합니다.
6. **ResponseCache 저장** — 답변을 캐시에 저장합니다. 3,600초(1시간) 후 만료됩니다.

전체 처리에 약 16초가 걸렸습니다. LLM 호출 두 번(질문 분석 + 답변 생성)이 시간을 거의 다 잡아먹었네요.

2. 캐시 적중 — 같은 질문엔 바로 답한다

같은 질문을 여러 번 더 입력해 봅니다. 터미널 로그가 확연히 달라집니다.

```

ex07 — 캐시 적중 로그

$ python run.py

INFO:src.agent_config:[ConnectHRAgent] 질문 수신: 김민준 연차 잔여일수 알려줘
INFO:src.cache:[ResponseCache] 적중: key=5e1dc59daa6e... (잔여 TTL: 3386초)
INFO:src.agent_config:[ConnectHRAgent] 캐시 응답 반환
INFO: 127.0.0.1:61917 - "POST /api/chat HTTP/1.1" 200 OK

INFO:src.agent_config:[ConnectHRAgent] 질문 수신: 김민준 연차 잔여일수 알려줘
INFO:src.cache:[ResponseCache] 적중: key=5e1dc59daa6e... (잔여 TTL: 3379초)
INFO:src.agent_config:[ConnectHRAgent] 캐시 응답 반환
INFO: 127.0.0.1:61919 - "POST /api/chat HTTP/1.1" 200 OK

INFO:src.agent_config:[ConnectHRAgent] 질문 수신: 김민준 연차 잔여일수 알려줘
INFO:src.cache:[ResponseCache] 적중: key=5e1dc59daa6e... (잔여 TTL: 3375초)
INFO:src.agent_config:[ConnectHRAgent] 캐시 응답 반환
INFO: 127.0.0.1:61919 - "POST /api/chat HTTP/1.1" 200 OK

```

그림 7-3: 같은 질문을 반복하면 캐시에서 즉시 반환한다. LLM을 호출하지 않는다.

첫 질문과 비교하면 차이가 확실합니다.

	첫 질문	캐시 적중
LLM 호출	2회 (분석 + 생성)	0회
DB 조회	1회	0회
소요 시간	~16초	즉시
로그	AgentExecutor 체인 전체	[ResponseCache] 적중 한 줄

잔여 TTL: 3386초 — 캐시가 저장된 지 약 3분이 지났다는 뜻입니다($3600 - 3386 = 214$ 초). TTL이 0이 되면 캐시가 만료되고 다음에 같은 질문이 들어오면 다시 LLM을 호출합니다.

3. 상태 확인 대시보드

사이드바의 “**상태 확인**” 을 클릭하면 캐시 적중률과 토큰 사용량을 한눈에 볼 수 있습니다.

서버 상태 상태 ● ok 버전 ex07 제목 LangChain 운영 안정화	응답 캐시 (ResponseCache) 적중률 83.33% 적중 5회 미스 1회 저장 항목 1건 TTL 3600초 최대 크기 1000건
토큰 사용량 (TokenTracker) 총 요청 0회 입력 토큰 8 출력 토큰 12 추정 비용 \$0.0000 평균 응답 15933ms	

그림 7-4: 상태 확인 페이지에서 캐시 적중률 83.33%와 토큰 사용량을 확인한다.

세 가지 카드가 표시됩니다.

- **서버 상태** — 서버가 정상(ok)인지 버전은 무엇인지 보여줍니다.
- **응답 캐시 (ResponseCache)** — 적중률 83.33%는 6번 질문 중 5번을 캐시에서 답했다는 뜻입니다. 미스 1건은 첫 질문이에요. 캐시가 비어 있었으니 당연히 미스입니다.
- **토큰 사용량 (TokenTracker)** — 입력/출력 토큰 수와 추정 비용을 보여줍니다. 로컬 모델(Ollama)은 비용이 \$0이고, OpenAI API를 쓰면 모델별 단가표에 따라 실제 비용이 계산돼요.

이 대시보드는 <http://localhost:8000/api/stats> API의 JSON 데이터를 웹 UI로 시각화한 겁니다.

2.10 더 알아보기

TTL을 얼마나 잡아야 하나요? 사내 문서가 자주 바뀌면 짧게(30분), 거의 안 바뀌면 길게(24시간) 잡으면 됩니다. 기본값 1시간은 대부분의 사내 환경에서 적당합니다. `.env`에서 `CACHE_TTL=3600`으로 조정할 수 있어요.

ResponseCache가 메모리 기반이면 서버 재시작하면 다 날아가지 않나요? 맞습니다. 운영 환경에서는 Redis 같은 외부 캐시를 쓰는 게 일반적이에요. 이 책에서는 개념 이해를 위해 인메모리 구현을 사용합니다. Redis로 바꾸려면 `get()`과 `set()`만 Redis 클라이언트로 교체하면 됩니다.

토큰 수가 추정값인 이유 Ollama는 응답에 토큰 사용량을 포함하지 않습니다. 단어 수 $\times 2$ 는 한국어 기준 대략적인 추정이에요. OpenAI API를 쓰면 정확한 토큰 수가 응답에 포함됩니다.

2.11 이것만은 기억하세요

- **같은 질문엔 캐시로 답합니다.** ResponseCache가 TTL 동안 답변을 기억해서 LLM 재호출을 막아줍니다.
- **운영은 기록에서 시작합니다.** TokenTracker로 토큰 사용량과 비용을 추적해야 운영 비용을 관리할 수 있어요.
- 다음 챗터부터는 이 에이전트의 **검색 품질**을 올립니다. “영똥한 문서를 가져온다”는 문제를 청킹 최적화와 리랭킹, 하이브리드 검색으로 해결합니다.

Ch.8: “엉뚱한 문서를 가져온다” — 검색 품질 튜닝 (ex08)

한 줄 요약: 검색이 바뀌면 답변이 바뀐다. RAG의 품질은 LLM이 아니라 Retriever가 결정한다.

핵심 개념: 청킹 최적화, 리랭킹, 하이브리드 검색



검색 품질 튜닝 — 청킹, 리랭킹, 하이브리드

1.1 “병가 규정을 물어봤는데 출장 규정이 나왔다”

CH07에서 캐시와 모니터링까지 달았습니다. 운영은 안정됐어요. 같은 질문엔 캐시로 빠르게 답하고 토큰도 추적합니다. 그런데 동료가 이상한 걸 하나 발견했습니다.

동료 C: “병가 규정이 어떻게 되는지 물어봤는데요, 출장 규정을 가져와서 답변하더라고요.”

확인해보니 정말 그랬습니다. “병가 신청 절차가 어떻게 되나요?”라고 물었는데 AI 비서가 출장 규정 문서를 근거로 답변을 만들어낸 겁니다. 출장비 정산 기한이라든가 숙박비 한도라든가, 병가와와는 아무 관계도 없는 이야기를 그럴듯하게 늘어놓고 있었습니다. 문제는 LLM이 아닙니다. LLM은 받은 문서를 가지고 성실하게 답변했을 뿐이에요. **잘못된 문서를 가져다 준 건 검색 단계**입니다.

비유를 하나 들어보겠습니다.

1.2 사서의 검색 실력

CH04에서 우리는 사서에게 서가를 만들어줬습니다. 문서를 잘게 쪼개고(청킹), 숫자 배열로 바꿔서(임베딩), 서가에 꽂아줬죠 (ChromaDB). 질문이 들어오면 사서가 서가에서 비슷한 문서를 꺼내옵니다.

그런데 이 사서의 검색 실력이 아직 초보 수준이에요.

제목만 보고 찾는 사서 — 지금 사서는 문서를 일정한 크기(500자)로 잘라서 꽂아줬습니다. 가위로 종이를 자르듯 내용과 상관없이 500자마다 끊었어요. “병가”와 “출장”이 같은 문서 안에 있으면? 하나의 조각에 두 내용이 섞입니다. 사서가 그 조각을 꺼내면 병가를 물어봤는데 출장 이야기가 따라오는 거죠.

더 나은 사서는 어떻게 할까요?

문단을 보고 찾는 사서 — 문서에는 보통 빈 줄이 있습니다. “제1조”와 “제2조” 사이, 단락이 바뀌는 곳이에요. 이 사서는 빈 줄이 보이면 거기서 먼저 끊습니다. 그래도 너무 길면 그때 글자 수로 잘라요. 500자씩 무조건 자르는 것보다 주제가 섞일 확률이 낮습니다. 이게 **재귀 문자 청킹(Recursive Character Chunking)**입니다.

목차를 보고 찾는 사서 — 이 사서는 더 꼼꼼합니다. 문서를 자를 때 “여기서 주제가 바뀌네?”를 감지해요. 연차 이야기를 하다가 출장 이야기로 넘어가는 지점을 알아채고 거기서 끊습니다. 빈 줄이 없어도 내용의 흐름을 읽어내죠. 이렇게 하면 하나의 조각에 하나의 주제만 담깁니다. 이게 **시맨틱 청킹(Semantic Chunking)**입니다.

내용까지 훑고 다시 확인하는 사서 — 검색 결과를 10개 가져왔다고 합시다. 이 중에 진짜 관련 있는 건 3~4개이고 나머지는 비슷하게 생겼지만 관련 없는 문서예요. 사서가 10개를 펼쳐놓고 질문을 다시 읽으면서 “이건 맞고, 이건 아니고”를 하나하나 확인합니다. 이게 **리랭킹(ReRanking)**입니다.

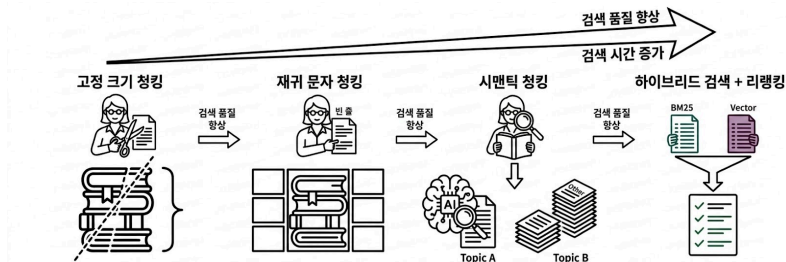


그림 8-1: 사서가 성장하는 과정. 가위로 자르던 사서가, 문단을 보고, 의미를 읽고, 두 가지 방법을 섞어 찾게 된다.

그리고 하나 더 있습니다. 지금 사서는 한 가지 방법으로만 찾아요. 의미가 비슷한 문서를 찾는 벡터 검색이죠. 하지만 “BM25”라는 정확한 키워드 매칭 방식도 있습니다.

두 가지 방법을 섞어 쓰는 사서 — “연차 신청”이라는 질문에 벡터 검색은 “휴가 사용”이라는 의미적으로 비슷한 문서를 찾아옵니다. BM25는 “연차”와 “신청”이라는 단어가 정확히 들어간 문서를 찾아오고요. 둘을 합치면? 의미도 맞고 키워드도 맞는 문서가 상위에 올라옵니다. 이게 **하이브리드 검색**입니다.

1.3 이번 버전에서 뭘 바꾸나

이번 장은 지금까지와 조금 다릅니다. 새로운 기능을 “만드는” 게 아니라 기존 검색 파이프라인을 “실험하고 개선하는” 장이에요. 세 가지 실험을 순서대로 진행합니다.

순서	실험	뭘 알아내나
실험 1	청킹 전략 + Retriever 튜닝	어떤 방식으로 자르고, 몇 개를 가져올지
실험 2	리랭킹	가져온 문서를 다시 정렬하면 정확도가 올라가나
실험 3	하이브리드 검색	BM25 + Vector를 섞으면 어떤 효과가 있나

비유로 돌아가면, 사서의 서가 정리법과 꺼내오는 규칙(실험 1)을 바꾸고 꺼낸 문서를 재확인(실험 2)하고 두 가지 검색법을 동시에 쓰는 방법(실험 3)을 배우는 겁니다. 이 세 가지 실험이 끝나면 “병가를 물어봤는데 출장 규정이 나오는” 문제는 사라집니다.

이제 실습으로 검색 품질을 직접 튜닝해보겠습니다.

2.1 용어 정리

이야기 속 비유	진짜 용어	정식 정의
가위로 500자마다 자르기	고정 크기 청킹(Fixed-size Chunking)	텍스트를 고정 문자 수(예: 500자)로 분할하는 방식. 오버랩을 두어 문맥 유실을 줄인다
문단·줄바꿈에서 자르기	재귀 문자 청킹(Recursive Character Chunking)	문단(\n\n), 줄바꿈(\n), 마침표 순서로 분할 지점을 찾는 방식. 고정 크기보다 주제 혼합이 적다
주제가 바뀌는 지점에서 자르기	시맨틱 청킹(Semantic Chunking)	임베딩 유사도로 인접 문장 간 의미 변화를 감지하여, 의미 단위로 분할하는 방식
꺼낸 문서를 다시 확인	ReRanking (리랭킹)	초기 검색 결과를 Cross-Encoder 모델로 (query, document) 쌍을 재채점하여 순위를 재정렬하는 기법
두 가지 방법을 섞어 찾기	Hybrid Search (하이브리드 검색)	BM25(키워드 매칭)와 Vector Search(의미 검색)를 가중 결합하는 검색 방식. alpha로 비율 조정
몇 개 가져올지	k값 (top-k)	검색 시 반환할 최대 문서 수. 작으면 정확도 중시, 크면 재현율 중시
몇 점 이상만 가져올지	Similarity Threshold	유사도 점수가 이 값 이하인 문서를 필터링하는 임계값
질문 단어와 문서 단어 매칭	BM25	단어의 출현 빈도(TF)와 역문서 빈도(IDF)를 기반으로 관련성을 계산하는 전통적 키워드 검색 알고리즘

2.2 파일 계층 구조

```

ex08/
├── tuning/
│   ├── step1_chunk_experiment/
│   │   ├── __main__.py
│   │   ├── data.py
│   │   ├── strategies.py
│   │   ├── analysis.py
│   │   ├── experiments.py
│   │   ├── display.py
│   │   └── retriever.py
│   ├── step2_reranker/
│   │   ├── __main__.py
│   │   ├── data.py
│   │   ├── reranker.py
│   │   ├── experiments.py
│   │   └── display.py
│   └── step3_hybrid_search/

```

[실습] 청킹 전략 + Retriever 튜닝
 CLI 진입점 (--step 1-1 ~ 1-5)
 샘플 문서·질의 상수
 청킹 전략 (Fixed / Recursive / Semantic)
 통계·코사인 유사도 계산
 실험 실행기 (크기 / 오버랩 / 전략 비교)
 Rich 테이블·벡터 검색 비교 출력
 Retriever 파라미터 튜닝 (k / threshold / metadata)

[실습] Cross-Encoder 리랭킹 전후 비교
 CLI 진입점 (--max-queries)
 샘플 문서·질의 상수
 CrossEncoderReranker 클래스
 리랭킹 전후 비교 실험
 Rich 테이블 출력

[실습] BM25+Vector 앙상블, alpha 조정

└─ __main__.py	CLI 진입점 (--max-queries)
└─ data.py	샘플 문서·질의 상수
└─ retrievers.py	BM25 / Vector / Ensemble 검색기
└─ experiments.py	alpha별 비교 실험
└─ display.py	Rich 테이블 출력

이번 챕터의 코드는 모두 **독립 실험 모듈**입니다. ex07의 에이전트와 직접 통합하지 않고 각각 독립적으로 실행해서 결과를 비교합니다. 실험 결과를 확인한 뒤 “어떤 설정이 우리 데이터에 맞는지”를 결정하고 이후 챕터에서 에이전트에 반영해요.

2.3 실습 환경 구축

기본 환경(Python 3.12, Ollama, Docker)이 없다면 **부록(환경 설정)** 을 먼저 참고하세요.

```
cd ex08
cp .env.example .env
python3.12 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
docker compose up -d      # PostgreSQL 실행
pip install -r requirements.txt
```

이전 챕터 Docker 종료: CH07의 Docker가 실행 중이라면 `cd ex07 && docker compose down` 으로 먼저 종료하세요.

패키지	역할
langchain-experimental	시맨틱 청킹
langchain-text-splitters	텍스트 분할기
rank-bm25	BM25 키워드 검색
sentence-transformers	ko-sroberta 임베딩 + Cross-Encoder 리랭킹
rich	터미널 출력 서식
easyocr	OCR 문자 인식

2.4 실습 순서

1. 청킹 + Retriever 실험
2. 리랭킹 전후 비교

3. 하이브리드 검색

세 가지 실험을 순서대로 진행합니다. 먼저 청킹 전략(Fixed/Recursive/Semantic)과 Retriever 파라미터를 비교하고, 리랭킹으로 검색 순위를 재정렬한 뒤, 마지막으로 BM25와 벡터 검색을 합쳐 하이브리드 검색을 테스트해요.

2.5 실습 1: 청킹 전략 실험 (tuning/step1_chunk_experiment/)

“병가를 물어봤는데 출장 규정이 나오는” 문제의 시작점이 바로 여기입니다. 문서를 어떻게 자르느냐가 검색 품질의 첫 번째 관문이에요. 이 실험 파일은 세 가지 실험을 **--step** 옵션으로 하나씩 진행합니다.

실험 1-1: 청크 크기 비교 (300 / 500 / 1000자)

가장 먼저 확인할 건 “몇 자로 자를 것인가”입니다. 같은 문서를 300자, 500자, 1000자로 잘라보고 결과를 비교해봅시다. 고정 크기 청킹의 핵심 코드입니다.

```
def fixed_size_chunking(text, chunk_size=500, overlap=100):
    chunks, start = [], 0
    while start < len(text):
        chunk = text[start:start + chunk_size].strip()
        if chunk:
            chunks.append(chunk)
            start += chunk_size - overlap # 핵심: 오버랩만큼 뒤로
    return chunks
```

chunk_size - overlap 이 핵심입니다. 500자를 잘랐으면 다음 시작점을 400으로 잡아서 앞 청크 끝 100자와 다음 청크 시작 100자가 겹치게 됩니다. 문장이 잘리더라도 앞뒤 문맥이 남도록 하는 장치예요.

```
python -m tuning.step1_chunk_experiment --step 1-1
```

변수	청크 수	평균 크기	최소 크기	최대 크기	실행 시간
Fixed-size (10000)	4	2975	2885	3085	8.000s
Fixed-size (10000)	3	5000	4975	5025	8.000s
Fixed-size (10000)	2	5000	4975	5025	8.000s

청크 크기별 비교 결과

300자로 자르면 청크 수가 많아집니다. 검색은 정밀해지지만 하나의 청크에 담긴 문맥이 짧아요. 1000자로 자르면 청크 수는 적지만 “연차 규정”과 “출장 규정”이 하나의 청크에 섞일 위험이 있습니다. **500자가 정밀도와 문맥의 균형점이에요.**

실험 1-2: 오버랩 비율 비교 (10% / 20% / 30%)

청크 크기를 500자로 정했으면 다음은 “앞뒤를 얼마나 겹칠 것인가”입니다.

오버랩이 없으면 문장이 잘리는 지점에서 문맥이 끊깁니다. “연차유급휴가를 사용하고자 할 때에는 사용 예정일 3일 전까지”가 두 청크로 갈라지면 어느 쪽 청크를 가져와도 불완전한 정보가 돼요. 오버랩은 이 문제를 줄이는 장치입니다.

```
python -m tuning.step1_chunk_experiment --step 1-2
```

[이미지: 오버랩 비율별 비교 결과]

오버랩이 크면 문맥 유실은 줄어들지만 중복 저장량이 늘어납니다. 30% 오버랩(150자)이면 청크의 거의 3분의 1이 겹쳐요. **20% 오버랩(100자)이 실용적인 기본값**입니다.

실험 1-3: 전략 비교 — 긴 문서 (Fixed vs Recursive vs Semantic)

이번 실험이 핵심입니다. 지금까지의 Fixed-size 청킹은 내용과 상관없이 글자 수로 잘랐습니다. “연차 규정”과 “출장 규정”이 연이어 나오면 하나의 청크에 두 주제가 섞이는 문제를 막을 수가 없어요. 이 문제를 줄이는 방법이 두 가지 있습니다.

재귀 문자 청킹은 문단(\n\n), 줄바꿈(\n), 마침표(.) 순서로 먼저 끊어봅니다. “제1조”와 “제2조” 사이에 빈 줄이 있으면 거기서 자르는 거죠. 그래도 500자를 넘으면 그때 글자 수로 자릅니다. 고정 크기 청킹보다 주제가 섞일 확률이 낮지만 빈 줄 없이 이어지는 문서에는 효과가 제한적이에요.

[이미지: Recursive Character Chunker 원리]

시맨틱 청킹은 더 근본적입니다. 인접한 문장의 임베딩을 비교해서 “의미가 크게 달라지는 지점”에서 분할합니다.

```
from langchain_experimental.text_splitter import SemanticChunker

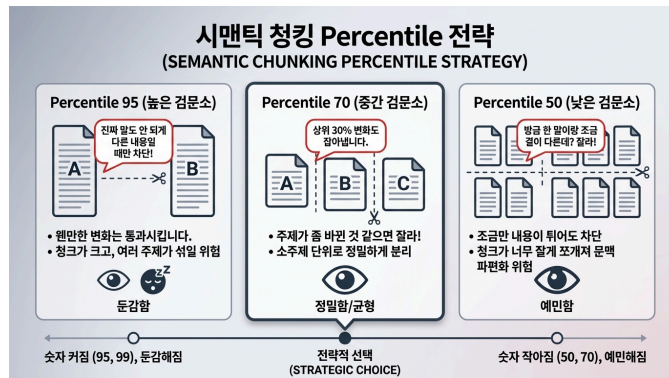
chunker = SemanticChunker(
    embeddings,                    # 한국어 임베딩 모델
    breakpoint_threshold_type="percentile", # 유사도 변화의 백분위수 기준
    breakpoint_threshold_amount=70    # 상위 30% 변화 지점에서 분할
)

chunks = chunker.split_text(text)
```

SemanticChunker는 문장마다 임베딩 벡터를 계산한 뒤 인접 문장 간 코사인 유사도를 측정합니다. 유사도가 급격히 떨어지는 지점이 곧 주제가 바뀌는 지점이고, 거기서 잘라냅니다. 여기서 **breakpoint_threshold_amount=70**이 중요합니다. 이 값이 바로 **Percentile**, 쉽게 말해 “검문소의 높이”예요.

[이미지: Semantic Chunker 원리]

시맨틱 청커는 인접 문장끼리 얼마나 닮았는지(유사도)를 계산한 뒤, 모든 유사도 차이(Gap)를 작은 순서부터 큰 순서로 줄을 세웁니다. 그리고 지정한 Percentile 이상의 Gap에서 “여기서 자른다”고 판단하는 거예요.



Semantic Percentile 원리

- **Percentile 95 (높은 검문소)**: 웬만한 변화는 통과시킵니다. “진짜 말도 안 되게 다른 내용이 나올 때만 차단해!” → 청크가 크고 여러 주제가 섞일 위험이 있어요.
- **Percentile 70 (중간 검문소)**: 상위 30%의 변화도 잡아냅니다. “주제가 좀 바뀐 것 같으면 잘라!” → 소주제 단위로 정밀하게 분리됩니다.
- **Percentile 50 (낮은 검문소)**: 조금만 내용이 튀어도 차단합니다. “방금 한 말이랑 조금 결이 다른데? 잘라!” → 청크가 너무 잘게 쪼개져서 문맥이 파편화될 수 있어요.

숫자가 커질수록(95, 99) 둔감해지고 작아질수록(50, 70) 예민해집니다. **percentile=70**으로 설정한다는 건 “유사도 변화가 아주 극단적이지 않더라도(상위 30% 수준만 되어도) 새로운 주제로 인정하고 자르겠다”는 뜻입니다.

문서 유형	추천 Percentile	이유
뉴스 기사, 짧은 블로그	90 ~ 95	문서가 짧으므로 잘게 쪼개면 맥락이 사라집니다
기술 문서, 매뉴얼	70 ~ 80	소주제(설치, 설정, 주의사항)가 명확히 나뉘어야 검색 품질이 올라갑니다
논문, 긴 보고서	60 ~ 75	논리 구조가 촘촘하므로 미세한 주제 변화를 잡아내야 합니다

이번 실험에서는 10개 규정(약 4,500자)으로 구성된 취업규칙을 사용합니다. 기술 문서에 해당하니 **percentile=70**으로 설정하겠습니다.

```
python -m tuning.step1_chunk_experiment --step 1-3
```

[이미지: 청킹 전략 비교 결과]

통계표 아래에 실제 벡터 검색 결과가 나옵니다. “재택근무 중 외출하려면 어떻게 해야 하나요?”라는 질문으로 각 전략의 청크를 검색한 결과입니다.

전략	검색된 청크	유사도	청크 크기	정답 포함
Fixed-size	#4	0.546	500자	O

전략	검색된 청크	유사도	청크 크기	정답 포함
Recursive	#5	0.634	461자	O
Semantic	#11	0.643	384자	O

세 전략 모두 정답(“사전에 팀장에게 보고”)을 포함한 청크를 가져왔지만, **품질 차이가 뚜렷합니다.**

- **Semantic(시맨틱)** 이 유사도 0.643으로 가장 높습니다. percentile=70 덕분에 31개 청크로 잘게 나뉘고, 재택근무 외출 규정만 담긴 384자 청크가 정확히 검색돼요.
- **Recursive(재귀 문자)** 는 유사도 0.634로 근소한 차이입니다. 문장 경계에서 깔끔하게 끊었지만 461자 청크에 다른 내용이 약간 섞여 있습니다.
- **Fixed-size(고정 크기)** 는 유사도 0.546으로 가장 낮아요. 500자를 기계적으로 채우다 보니 관련 없는 내용이 끼어들어 유사도가 떨어집니다.

전략	추천 상황
고정 크기	대량 문서를 빠르게 처리해야 할 때 (속도 우선)
재귀 문자	짧은 문서, 빈번한 업데이트 환경 (속도-품질 균형)
시맨틱	긴 문서에서 주제 분리가 중요할 때 (품질 우선, percentile 조정 필수)

--percentile 옵션으로 검문소 높이를 직접 바꿔서 실험해보세요:

```
# 둔감하게 - 큰 변화만 감지 (청크가 커짐)
python -m tuning.step1_chunk_experiment --step 1-3 --percentile 95

# 권장값 - 소주제 단위로 분리
python -m tuning.step1_chunk_experiment --step 1-3 --percentile 70

# 예민하게 - 작은 변화도 감지 (청크가 잘게 쪼개짐)
python -m tuning.step1_chunk_experiment --step 1-3 --percentile 50
```

한 걸음 더 — 짧은 문서에서는? (step 1-4)

그런데 짧은 문서에서도 percentile만 낮추면 Semantic이 이기지 않을까?

1098자짜리 짧은 문서에서 percentile을 40까지 낮춰봐도 결과는 같습니다:

Percentile	Semantic 청크 수	Semantic 유사도	Recursive 유사도	결과
95	2개	0.407	0.670	Recursive WIN
70	7개	0.407	0.670	Recursive WIN

Percentile	Semantic 청크 수	Semantic 유사도	Recursive 유사도	결과
50	11개	0.640	0.670	Recursive WIN
40	13개	0.639	0.670	Recursive WIN

짧은 문서에는 문장 수 자체가 적어서 “의미 변화”를 감지할 재료가 부족합니다. Recursive는 `\n\n`, `\n`, `.` 순서로 문단과 문장 경계에서 자르니까 짧은 문서를 2~3개 청크로 나눌 때 주제별로 정확히 떨어져요. 이걸 percentile 조정으로 따라잡을 수 없습니다.

결론: 시맨틱 청킹은 “문서 길이 + percentile 조정” 두 조건이 모두 맞아야 합니다. 짧은 문서에서는 Recursive가 최선이예요. 직접 확인해보세요.

```
python -m tuning.step1_chunk_experiment --step 1-4
```

실험 1-5: Retriever 파라미터 튜닝

청킹 전략을 정했으면 다음 질문은 “서가에서 몇 개를 꺼낼 것인가”입니다. 조정할 수 있는 파라미터가 세 가지 있습니다.

k(top-k) — 검색 결과를 몇 개 가져올지 정합니다. k=3이면 상위 3개만, k=10이면 10개를 가져옵니다. 적게 가져오면 LLM에 넘기는 컨텍스트가 짧아 토큰을 절약하지만 정보를 놓칠 수 있고, 많이 가져오면 관련 없는 문서(노이즈)가 섞여요.

threshold — “유사도가 이 점수 이하면 가져오지 마라”라는 필터입니다. threshold=0.0이면 유사도가 0.01이든 전부 가져옵니다. 0.2로 올리면 유사도 0.2 미만인 저품질 문서가 걸러지고요. 너무 높이면(0.5 이상) 관련 있는 문서까지 잘려 나갑니다.

metadata_filter — 문서에 붙어 있는 태그(부서, 문서 유형 등)로 검색 범위를 좁힙니다. “인사팀 문서만 검색”이라고 지정하면 재무팀 출장 규정 같은 문서가 아예 후보에서 빠져요.

파라미터	권장값	이유
k(top-k)	5	k=3은 정보를 놓칠 수 있고, k=10은 노이즈가 섞입니다
threshold	0.2	0.0이면 저품질 문서까지 전부, 0.3 이상이면 필요한 문서까지 잘립니다
metadata_filter	상황별	“인사팀 문서만”처럼 범위를 좁히면 정확도가 올라갑니다

먼저 권장 설정으로 전체 비교 실험을 실행해보세요:

```
python -m tuning.step1_chunk_experiment --step 1-5
```

k값(3, 5, 10), threshold(0.0~0.5), metadata 필터(부서별, 문서유형별)의 결과가 표로 출력됩니다. 결과를 확인한 뒤 `--k`, `--threshold`, `--department` 옵션으로 직접 값을 바꿔가며 실험해보세요.

```
# k를 줄이고 threshold를 높이면? → 정확도 증시, 적은 결과
python -m tuning.step1_chunk_experiment --step 1-5 --k 3 --threshold 0.3

# HR 부서 문서만 검색하면? → 노이즈 제거, 범위 한정
python -m tuning.step1_chunk_experiment --step 1-5 --k 10 --department HR

# 재무팀 + 높은 threshold → 가장 관련 높은 재무 문서만
python -m tuning.step1_chunk_experiment --step 1-5 --threshold 0.5 --department FINANCE
```

k=3으로 줄이면 “출장비 정산” 같은 질문에 재무팀 문서 하나만 나옵니다. k=10으로 늘리면 HR 문서까지 섞여 들어오고요. threshold를 0.5로 올리면 유사도가 낮은 문서가 잘려 나가는데 너무 높으면 관련 있는 문서까지 사라집니다. 이 균형을 직접 찾아보는 게 이 실험의 목적이예요.

2.6 실습 2: Cross-Encoder 리랭킹 (tuning/step2_reranker/)

검색이 10개를 가져왔습니다. 이 중에 진짜 관련 있는 건 몇 개일까요? 초기 벡터 검색은 “대략 비슷한 문서”를 빠르게 가져오는 데는 좋지만, 미세한 관련성 차이를 구분하기엔 약합니다. 리랭커가 이 차이를 잡아내요. `ex08/tuning/step2_reranker/reranker.py`의 핵심은 Cross-Encoder가 (질문, 문서) 쌍을 직접 읽고 채점하는 부분입니다.

```
pairs = [(query, doc["content"]) for doc in documents]
scores = self.model.predict(pairs) # Cross-Encoder가 각 쌍을 직접 채점
# 점수 내림차순 정렬 → 상위 top_k만 반환
```

벡터 검색은 질문과 문서를 **따로** 임베딩한 뒤 코사인 유사도를 봅니다(Bi-Encoder). Cross-Encoder는 질문과 문서를 **한 번에** 읽어요. 그래서 더 정확하지만 문서마다 모델 추론이 필요해서 느립니다. “넓게 가져와서(top_k=10~20) → 좁게 정제(top_k=5)”하는 **2단계 패턴**을 쓰는 이유가 여기에 있습니다.

“연차 신청 절차”를 검색하면 리랭킹 전에는 3위였던 “연차 신청은 3일 전 인사담당자에게...” 문서가 리랭킹 후 1위로 올라옵니다. Cross-Encoder가 질문과 문서를 함께 읽었기 때문에 더 정확하게 판단할 수 있는 거죠.

```
# 실행
python -m tuning.step2_reranker
```

[이미지: 리랭킹 전후 비교] 그림 8-3: 리랭킹 전에는 3위였던 문서가 리랭킹 후 1위로 올라온다. Cross-Encoder가 질문-문서 관련성을 더 정확히 판단한다.

2.7 실습 3: 하이브리드 검색 (tuning/step3_hybrid_search/)

마지막 실습입니다. 벡터 검색만으로는 부족한 경우가 있어요. “BM25”라는 키워드 기반 검색을 섞으면 어떨까요? `ex08/tuning/step3_hybrid_search/retrievers.py`에는 검색기가 세 개 들어 있습니다.

BM25Retriever는 `rank-bm25` 라이브러리로 키워드 매칭 검색을 합니다. “연차”라는 단어가 정확히 있는 문서에 높은 점수를 줘요.

VectorRetriever 는 CH04에서 ChromaDB가 해주던 일을 numpy로 직접 구현한 버전입니다. `model.encode()` 로 임베딩을 만들고 코사인 유사도를 계산합니다.

벡터 검색은 “연차”와 “휴가”가 의미적으로 비슷하다는 걸 알지만, BM25는 “연차”라는 글자가 정확히 있는 문서만 찾습니다. 각각 장단점이 있으니 둘을 합치면 더 좋겠죠?

EnsembleRetriever — 두 검색을 합치는 앙상블

BM25와 Vector 각각에서 후보를 가져온 뒤 **alpha** 로 가중 합산합니다.

```
# 하이브리드 검색의 핵심: alpha로 두 검색의 비율 조정
hybrid_score = self.alpha * vector_score + (1 - self.alpha) * bm25_score
```

alpha=0.0이면 BM25만(키워드 정확 매칭), alpha=1.0이면 Vector만(의미 유사도), alpha=0.5면 반반이에요. 한국어 사내 문서에서는 **alpha=0.5~0.7** 이 대체로 좋은 결과를 냅니다.

```
# 실행 (sentence-transformers, rank-bm25 필요)
python -m tuning.step3_hybrid_search
```

[이미지: 하이브리드 검색 결과] 그림 8-4: alpha=0.5일 때 BM25 점수와 Vector 점수가 합산되어 최종 순위가 결정된다.

alpha 값	의미	적합한 경우
0.0	BM25만 사용	고유명사, 약어, 정확한 수치가 중요할 때
0.5	반반 혼합	대부분의 사내 문서 검색 (권장)
0.7	Vector 중심	동의어·유사 표현을 포괄해야 할 때
1.0	Vector만 사용	키워드보다 의미가 중요할 때

2.8 더 알아보기

Semantic Chunking은 왜 Fixed-size보다 나은가요?

핵심은 “무엇을 기준으로 자르느냐”입니다. Fixed-size 청킹은 글자 수만 봅니다. 500자가 되면 자르죠. “연차 규정”이 480자째에서 시작해도 500자에서 잘려버립니다. 다음 청크는 연차 규정의 뒷부분과 출장 규정의 앞부분이 하나로 합쳐져요. 이러면 “연차”를 검색했을 때 출장 규정이 딸려 나옵니다.

Semantic Chunking은 의미의 변화를 봅니다. 문장마다 임베딩 벡터를 구하고 인접한 두 문장의 코사인 유사도를 계산해요. “연차 유급 휴가는 15일이 부여된다”와 “연차 신청은 3일 전까지 제출한다”의 유사도는 높습니다(둘 다 연차 이야기). 하지만 “연차 신청은 3일 전까지 제출한다”와 “재택근무는 입사 6개월 이상 직원에 한해 가능하다”의 유사도는 낮습니다(주제가 바뀜). 유사도가 크게 떨어지는 이 지점에서 잘라내는 게 Semantic Chunking이에요.

단점도 있습니다. 임베딩 모델을 로드해야 해서 처리 시간이 길고 청크 크기가 불균일합니다. 문서가 자주 바뀌는 환경에서는 Recursive Character 청킹(단락이나 문장 경계에서 분할)이 속도와 품질의 균형점이에요.

그럼 Semantic Chunking은 언제 쓰는 게 좋나요?

“몇 자 이상이면 시맨틱을 쓰세요”라는 절대적 기준은 없습니다. 시맨틱 청킹이 얼마나 잘 먹히느냐는 글자 수보다 **문서의 구조와 내용이 얼마나 복잡한지**에 달려 있어요.

청크 크기 구간	특성	추천 전략
300~500자 이하	문장이 짧고 구조가 단순. 재귀 문자 청킹만으로도 의미 단위 보존 가능	Recursive
500~1,000자 이상	한 청크 안에 여러 소주제가 섞일 가능성 증가. 재귀 문자 청킹은 중요한 맥락 중단을 잘라버릴 수 있음	Semantic

핵심은 “내 문서에서 하나의 주제가 평균 몇자인가?”입니다. 취업규칙처럼 한 조항이 500자 이상 이어진다면 시맨틱 청킹이 주제 경계를 정확히 잡아내서 검색 품질을 끌어올립니다. 글자 수와 무관하게 시맨틱 청킹이 특히 유리한 경우도 있습니다.

- **주제 전환이 빈번한 문서** — 한 페이지에 여러 상품 스펙이나 규정이 나열되는 경우
- **비정형 구조** — 문단 구분이 모호하거나 줄바꿈이 불규칙한 데이터
- **고도의 추론이 필요한 RAG** — 단순 키워드 매칭이 아니라 “문맥”을 정확히 짚어야 하는 경우

--step 1-3 으로 긴 문서(10개 규정, 약 4,500자)에서 시맨틱 청킹의 유사도가 역전하는 것을 직접 확인할 수 있습니다. --
 step 1-4 는 짧은 문서에서 percentile을 아무리 조정해도 Recursive가 이기는 것을 보여줍니다.

BM25와 Vector Search는 언제 강점이 다른가요?

- **BM25가 유리한 경우**: 고유명사, 약어, 정확한 수치가 중요할 때입니다. “BRS-2024”라는 코드를 찾을 때 벡터 검색은 “업무 규정 시스템”과 비슷한 문서를 가져오지만 BM25는 “BRS-2024”가 정확히 적힌 문서를 찾아요.
- **Vector가 유리한 경우**: 동의어나 유사 표현을 포괄해야 할 때입니다. “연차”와 “유급 휴가”가 같은 뜻이라는 걸 벡터 검색은 알지만 BM25는 모릅니다.
- **둘을 합친 하이브리드가 유리한 경우**: 대부분의 실제 업무 환경이 여기에 해당합니다. 사내 문서에는 고유 용어와 일반 표현이 섞여 있기 때문이에요.

2.9 이것만은 기억하세요

- **검색이 바뀌면 답변이 바뀝니다**. RAG의 품질은 LLM이 아니라 Retriever가 결정해요. LLM에게 영똥한 문서를 주면 아무리 똑똑한 모델이라도 영똥한 답을 할 수밖에 없습니다.
- **청킹은 의미 단위로 하세요**. Fixed-size 청킹은 빠르지만 주제가 섞입니다. Semantic Chunking이 이상적이고 Recursive Character 청킹이 현실적인 균형점이에요.
- **리랭킹은 2단계 전략으로**. 넓게 가져와서(top_k=10~20) Cross-Encoder로 좁게 정제(top_k=5)합니다.
- **하이브리드 검색이 단일 검색보다 낫습니다**. BM25(키워드) + Vector(의미)를 alpha로 가중 결합하면 한쪽만 쓸 때의 약점을 보완해줍니다.
- 다음 챗에서는 검색이 아니라 **질문 자체**를 다룹니다. 사용자가 “규정 알려줘”처럼 모호하게 물어봤을 때 질문을 구체화하는 Query Rewriting을 배워요.

Ch.9: “질문을 제대로 이해 못한다” — 질의+결과 고급 튜닝 (ex09)

한 줄 요약: 검색 전에 질문을 바꾸고, 검색 후에 결과를 가공한다. 검색 엔진의 앞뒤를 튜닝하는 장.

핵심 개념: Parent Document Retriever, Contextual Compression, HyDE, Multi-Query

[이미지: 질문 해석 튜닝 — 질문을 바꿔야 답변이 달라진다]

1.1 “휴가”라고 물었는데 못 찾는다

CH08에서 검색 품질을 튜닝했습니다. 청킹 크기를 실험하고, 리랭커로 순서를 바로잡고, BM25와 벡터 검색을 합쳐서 하이브리드로 만들었습니다. 검색 정확도가 꽤 올라갔습니다. 동료가 다시 테스트합니다.

동료: “휴가 규정 알려줘.”

AI 비서가 대답합니다. 하지만 가져온 문서가 이상합니다. “복리후생 안내” 문서의 일부분이 나왔습니다. 정작 찾고 싶었던 “취업규칙 제15조 연차유급휴가” 규정은 빠져 있습니다.

왜일까요? 문서에는 “연차유급휴가”라고 적혀 있습니다. 동료는 “휴가”라고 물었습니다. 같은 뜻인데 단어가 다릅니다. 한 번 더 테스트 해봅니다.

동료: “WFH 정책이 뭐야?”

AI 비서: “관련 문서를 찾지 못했습니다.”

문서에는 “재택근무”라고 적혀 있습니다. WFH(Work From Home)를 재택근무로 연결하지 못한 겁니다. CH08에서 해결한 건 “어떤 문서를 가져오느냐”였습니다. 검색 엔진 자체의 성능을 올렸습니다. 그런데 지금 문제는 다릅니다. **검색 엔진이 잘 동작해도, 질문 자체를 이해 못하면 소용없습니다.**

1.2 사서의 언어 능력이 자라다

도서관 비유를 이어가봅시다.

CH08에서 사서는 서가 정리 방법을 배웠습니다. 책을 잘 분류하고, 중요한 책은 앞으로 빼놓고, 키워드 색인과 주제 색인을 동시에 쓰는 법을 배웠습니다.

그런데 방문자가 이렇게 물으면 어떨까요?

방문자: “쉬는 날 규정 좀 알려주세요.”

초보 사서는 서가에서 “쉬는 날”이라는 단어를 찾습니다. 없습니다. 서가에는 “연차유급휴가”라고 적혀 있으니까요.

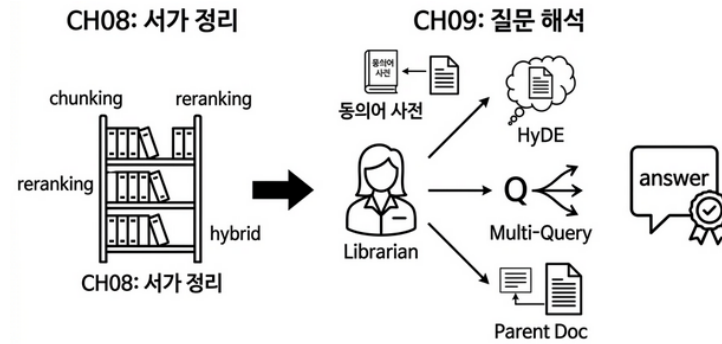
경험 많은 사서는 다릅니다.

첫째, **단어를 번역합니다.** “쉬는 날”이라는 말을 들으면 “아, 연차유급휴가를 찾는 거구나”라고 자동 변환합니다. 이게 **약어/동의를 확장**입니다.

둘째, **답을 먼저 상상합니다.** “쉬는 날 규정이라면… 아마 ‘1년 근속 시 15일 유급휴가’ 같은 내용이 있는 문서겠지.” 그리고 그 상상한 답과 비슷한 실제 문서를 찾습니다. 이게 **HyDE(Hypothetical Document Embeddings)**입니다. 질문을 검색하는 게 아니라, 상상한 답변을 검색하는 겁니다.

셋째, **질문을 여러 갈래로 바꿔봅니다.** “쉬는 날 규정”이라는 한 가지 질문을 “연차 사용 방법”, “유급휴가 일수”, “휴가 신청 절차”처럼 여러 각도로 바꿔서 각각 검색합니다. 이게 **Multi-Query**입니다.

넷째, **찾은 조각에서 원본 전체를 꺼냅니다**. 검색에서 “연차 신청은 3일 전 서면으로 합니다”라는 짧은 조각이 걸렸다면, 그 조각이 속한 원래 문서 전체(“취업규칙 제15조~16조”)를 꺼내옵니다. 이게 **Parent Document Retriever**입니다. 조각만 보면 맥락을 놓칠 수 있으니, 부모 문서 전체를 보여주는 겁니다.



CH08: 어떤 문서를 가져오나 vs CH09: 질문을 어떻게 이해하나

그림 9-1: CH08이 서가 정리(검색 엔진 튜닝)였다면, CH09는 사서의 언어 능력(질의 변환)과 요약 능력(결과 가공)이다.

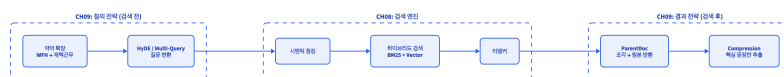
1.3 CH08과 CH09의 차이

혼동하기 쉬우니 정리합니다.

[이미지: CH08 vs CH09 비교] 그림 9-2: CH08은 서가 정리(검색 엔진), CH09는 검색 전 질의 변환 + 검색 후 결과 가공. 둘은 택일 이 아니라 조합하는 관계다.

	CH08: 검색 튜닝	CH09: 질의 + 결과 튜닝
해결하는 문제	영동한 문서를 가져온다	질문 이해도 못하고, 결과 가공도 약하다
비유	서가 정리 방법 개선	사서의 언어 능력 + 요약 능력 향상
핵심 기법	청킹, 리랭킹, 하이브리드 검색	질의: 약어 확장, HyDE, Multi-Query / 결과: ParentDoc, Compression
질문은	그대로 둔다	바꾼다 (검색 전)
검색 결과는	그대로 둔다	가공한다 (검색 후)
문서 인덱스는	바꾼다	그대로 둔다

CH08은 “같은 질문에 더 좋은 문서를 가져오게” 만들었습니다. CH09는 **검색 전**에는 질문을 변환하고(약어 확장, HyDE, Multi-Query), **검색 후**에는 결과를 가공합니다(ParentDoc으로 원본 반환, Compression으로 핵심만 추출). 검색 엔진의 앞뒤를 모두 다루는 셈입니다.



CH08 vs CH09 비교: 검색 전(질의 전략) → 검색 엔진 → 검색 후(결과 전략)

이번 챗터에서는 두 가지를 다룹니다.

1. **고급 Retriever** — 검색 결과를 가공하는 전략 (ParentDoc, SelfQuery, Compression)
2. **Query Rewrite** — 검색 전에 질문을 변환하는 기법 (약어 확장, HyDE, Multi-Query)

이제 실습으로 쿼리 변환과 근거 표시를 구현해보겠습니다.

2.1 용어 정리

이야기 속 표현	진짜 이름	정의
조각에서 원본 전체를 꺼낸다	Parent Document Retriever	작은 자식 청크로 검색하되, 매칭된 청크가 속한 원본(부모) 문서 전체를 반환하는 검색 전략. 짧은 청크의 정밀한 매칭과 긴 문서의 풍부한 컨텍스트를 동시에 확보한다
LLM이 질문에서 필터를 뽑아낸다	Self-Query Retriever	자연어 질문에서 메타데이터 필터(topic, source 등)를 자동 추출하여 필터링 후 검색하는 전략. “휴가 관련 규정”이라고 물으면 topic=휴가 필터를 자동 적용한다
관련 부분만 추려낸다	Contextual Compression	검색된 문서에서 질문과 관련된 문장만 추출하여 LLM에 전달하는 전략. 긴 문서에서 핵심만 뽑아 토큰을 절약한다
답을 먼저 상상한다	HyDE	Hypothetical Document Embeddings. 질문 대신 가상의 답변 문서를 생성하고, 그 문서의 임베딩으로 실제 문서를 검색하는 기법. 질문과 문서의 의미 간극을 줄인다
질문을 여러 갈래로 바꾼다	Multi-Query	하나의 질문을 여러 표현으로 변환하여 각각 검색한 뒤 결과를 합치는 기법. 검색 범위를 넓혀 누락을 줄인다
약어를 풀어쓴다	약어/동의어 확장	사내 약어(WFH→재택근무)와 동의어(연차→유급휴가)를 사전으로 관리하여 쿼리를 확장하는 전처리 기법

2.2 파일 계층 구조

ex09/	
├── tuning/	
│ ├── step1_advanced_retriever/	[실습 1] 고급 Retriever 실험
│ │ ├── __main__.py	[실습] CLI (--step 1-1 ~ 1-3, --top_k, --query)
│ │ ├── data.py	[참고] 부모 문서 · 자식 청크 · 테스트 쿼리
│ │ ├── retrievers.py	[참고] ParentDoc · SelfQuery · Compression 구현
│ │ ├── display.py	[참고] Rich 테이블 출력
│ │ └── experiments.py	[참고] 실험 1-1 ~ 1-3 실행기
│ └── step2_query_rewrite/	[실습 2] Query Rewrite 실험

└─ <code>__main__.py</code>	【실습】 CLI (<code>--step 2-1 ~ 2-3, --query, --num_queries</code>)
└─ <code>data.py</code>	【참고】 약어 사전 · HyDE 템플릿 · 검색 문서
└─ <code>rewriters.py</code>	【참고】 약어확장 · HyDE · Multi-Query 구현
└─ <code>display.py</code>	【참고】 Rich 테이블 출력
└─ <code>experiments.py</code>	【참고】 실험 2-1 ~ 2-3 실행기

CH08과 마찬가지로 `--step` 옵션으로 서브실험을 하나씩 실행합니다. 파라미터를 바꿔가며 결과를 비교해보세요.

2.3 실습 1: 고급 Retriever (tuning/step1_advanced_retriever/)

CH08에서 리랭커와 하이브리드 검색으로 “가져오는 문서의 품질”을 높였다면, 여기서는 “검색과 반환 방식 자체”를 바꿉니다. 세 가지 Retriever를 서브실험으로 나누어 비교합니다. 실험에 사용할 데이터는 `data.py`에 정의되어 있습니다. 부모 문서 3개(PARENT_DOCUMENTS — 휴가 규정, 재택근무, 정보보안), 자식 청크 12개(CHILD_CHUNKS — 부모 문서에서 핵심 내용을 한 줄로 요약), 토픽 키워드(TOPIC_KEYWORDS — SelfQuery 필터에 사용)가 들어 있습니다.

파라미터	설명	권장값
<code>--step</code>	실험 단계 (1-1, 1-2, 1-3)	—
<code>--top_k</code>	검색 결과 반환 수	2~3
<code>--query</code>	직접 질문 입력	(자동 테스트 쿼리 사용)

실험 1-1: ParentDocument Retriever — 조각으로 찾고, 원본을 돌려준다

ex09/tuning/step1_advanced_retriever/retrievers.py의 `ParentDocumentRetriever` 클래스가 핵심입니다.

CH04에서 문서를 500자씩 잘랐습니다. 검색할 때는 이 짧은 청크가 효율적입니다. 하지만 LLM에게 답변을 생성하라고 줄 때는 짧은 조각만으로는 맥락이 부족합니다. Parent Document Retriever는 이 문제를 해결합니다. 검색은 작은 청크로, 반환은 원본 문서로. 부모 문서는 제15조~16조 전체 텍스트이고, 자식 청크는 핵심 내용을 한 줄로 요약한 것입니다. 검색은 짧은 자식 청크의 임베딩으로, 반환은 긴 부모 문서로 합니다. 핵심 클래스입니다.

```
class ParentDocumentRetriever:
    """자식 청크 임베딩으로 검색하고 부모 문서 전체를 반환합니다."""

    def __init__(self, parent_docs, child_chunks, embeddings):
        self.parent_docs = {doc["id"]: doc for doc in parent_docs}
        self.child_chunks = child_chunks
        # 자식 청크 임베딩 사전 계산
        texts = [c["content"] for c in child_chunks]
        self.chunk_vectors = embeddings.embed_documents(texts)

    def search(self, query, top_k=2):
        query_vec = self.embeddings.embed_query(query)
```

```

scored = []
for i, chunk in enumerate(self.child_chunks):
    sim = _cosine_similarity(query_vec, self.chunk_vectors[i])
    scored.append((sim, chunk))
scored.sort(key=lambda x: x[0], reverse=True)

# 상위 자식 → 부모 문서 역참조 (중복 제거)
seen_parents = set()
for sim, chunk in scored:
    pid = chunk["parent_id"]
    if pid not in seen_parents and len(results) < top_k:
        seen_parents.add(pid)
        parent = self.parent_docs.get(pid)
        # child_chunk + parent_content 함께 반환

```

`search()`의 흐름은 두 단계입니다. 모든 자식 청크에 대해 임베딩 코사인 유사도를 계산하고, 상위 청크에서 `parent_id`를 꺼내 부모 문서 전체를 반환합니다. `seen_parents`로 같은 부모가 중복 반환되는 걸 막습니다.

```
python -m tuning.step1_advanced_retriever --step 1-1
```

[이미지: ParentDocument Retriever 실험] 그림 9-3: 짧은 자식 청크(유사도 0.73)로 정확하게 검색하고, 긴 부모 문서 전체(423자)를 반환한다.

설정을 바꿔 실험해보세요.

```

# 반환 수를 바꿔보기
python -m tuning.step1_advanced_retriever --step 1-1 --top_k 3

# 다른 질문으로 테스트
python -m tuning.step1_advanced_retriever --step 1-1 --query "재택근무 신청 조건"

```

실험 1-2: SelfQuery Retriever — 질문에서 필터를 자동 추출

같은 파일의 `SelfQueryRetriever` 클래스입니다. 필터 추출에 사용하는 `TOPIC_KEYWORDS`는 `data.py`에 정의되어 있습니다.

“연차 관련 규정”이라고 물으면 사람은 “아, 휴가 관련 문서를 찾으면 되겠구나”라고 자연스럽게 범위를 좁힙니다. SelfQuery Retriever는 이 과정을 자동화합니다. 질문에서 메타데이터 필터를 추출하여 **검색 범위를 먼저 좁히고** 임베딩 검색을 수행합니다. `data.py`에 정의된 토픽 키워드 사전입니다.

```

TOPIC_KEYWORDS = {
    "휴가": ["연차", "휴가", "유급", "반차"],
    "재택근무": ["재택", "원격", "WFH", "홈피스"],
    "복리후생": ["복지", "복리", "자기계발", "건강검진", "교육비"],

```

```

"출장": ["출장", "여비", "출장비"],
"평가": ["성과", "평가", "KPI"],
}

```

이 사전을 기반으로 질문에서 토픽을 자동 추출합니다.

```

class SelfQueryRetriever:
    def extract_filter(self, query):
        """쿼리에서 토픽 필터를 자동 추출합니다."""
        query_lower = query.lower()
        for topic, keywords in TOPIC_KEYWORDS.items():
            if any(kw in query_lower for kw in keywords):
                return {"topic": topic}
        return {}

    def search(self, query, top_k=3):
        filters = self.extract_filter(query)
        # 필터에 맞는 청크만 남기고 임베딩 검색
        for i, chunk in enumerate(self.child_chunks):
            if filters and chunk_topic != filters["topic"]:
                continue # 토픽이 다르면 건너뛰
            sim = _cosine_similarity(query_vec, self.chunk_vectors[i])

```

TOPIC_KEYWORDS 사전에서 “연차”, “휴가”, “유급” 같은 키워드가 쿼리에 있으면 **topic=휴가** 필터를 자동 적용합니다. 12개 자식 청크 중 4개만 남기고 검색하므로, 관련 없는 문서가 끼어들 확률이 줄어듭니다.

```
python -m tuning.step1_advanced_retriever --step 1-2
```

[이미지: SelfQuery Retriever 실험] 그림 9-4: “연차 신청 절차와 팀장 승인 방법”에서 **topic=휴가** 필터를 자동 추출, 12개 → 4개 청크로 범위를 좁힌 뒤 검색한다.

```

# 다른 토픽으로 테스트 - 자동으로 topic=복리후생 필터 적용
python -m tuning.step1_advanced_retriever --step 1-2 --query "직원 교육비 지원 한도"

```

실험 1-3: Contextual Compression — 긴 문서에서 핵심만 추출

같은 파일의 **ContextualCompressionRetriever** 클래스입니다. Parent Document Retriever와 정반대 방향의 접근입니다. 검색된 문서에서 **질문과 관련된 문장만 추출**하여 LLM에 전달합니다. 긴 문서를 그대로 넘기면 토큰이 낭비되고, 관련 없는 내용이 답변을 오염시킬 수 있습니다.

```
def _compress(self, query, document):
    """문서에서 쿼리 관련 문장만 추출합니다."""
    query_vec = self.embeddings.embed_query(query)
    sentences = [s.strip() for s in document.replace("\n", ". ").split(".")
                  if s.strip() and len(s.strip()) > 5]
    sent_vecs = self.embeddings.embed_documents(sentences)
    scored = []
    for i, sent in enumerate(sentences):
        sim = _cosine_similarity(query_vec, sent_vecs[i])
        scored.append((sim, sent))
    scored.sort(key=lambda x: x[0], reverse=True)
    return ". ".join(s for _, s in scored[:3]) + "." # 상위 3문장
```

문서를 문장 단위로 분리하고, 각 문장의 임베딩과 쿼리 임베딩의 유사도를 비교합니다. 상위 3문장만 추출하면 원본의 40% 수준으로 압축됩니다.

```
python -m tuning.step1_advanced_retriever --step 1-3
```

[이미지: Contextual Compression 실험] 그림 9-5: 423자 원본에서 질문 관련 3문장만 추출하여 40% 압축. 토큰 절약과 답변 집중도를 동시에 확보한다.

Parent Document와 Compression을 **조합**하면 더 효과적입니다. 부모 문서 전체를 가져온 뒤, 관련 문장만 압축하여 LLM에 전달하면 풍부한 컨텍스트와 토큰 절약을 동시에 얻을 수 있습니다.

2.4 실습 2: 질문을 바꿔서 검색한다 (tuning/step2_query_rewrite/)

검색 엔진을 건드리지 않고, 질문 자체를 바꿔서 검색 품질을 올리는 접근입니다. 세 가지 기법을 서브실험으로 나누어 비교합니다. 실험에 사용할 데이터는 `data.py`에 정의되어 있습니다. 약어 사전 12개(`ABBREVIATION_MAP` — WFH→재택근무, OT→초과근무 등), 동의어 사전 6개(`SYNONYM_MAP`), HyDE 템플릿 6개(`HYDE_TEMPLATES`), 검색 대상 문서 10개(`SEARCH_DOCUMENTS`)가 들어 있습니다.

파라미터	설명	권장값
<code>--step</code>	실험 단계 (2-1, 2-2, 2-3)	—
<code>--query</code>	직접 질문 입력	(자동 테스트 쿼리 사용)
<code>--num_queries</code>	Multi-Query 변형 수 (2-3 전용)	3

실험 2-1: 약어/동의어 확장 — LLM 없이 바로 되는 첫 번째 개선

`ex09/tuning/step2_query_rewrite/rewriters.py`의 `expand_abbreviations()` 함수가 핵심입니다. 약어 사전은 같은 패키지의 `data.py`에 있습니다.

가장 간단하면서도 효과 큰 방법입니다. 사내에서 쓰는 약어와 동의어를 사전으로 등록해둡니다.

```

ABBREVIATION_MAP = {
    "연차": "연차유급휴가",
    "WFH": "재택근무",
    "4대보험": "국민연금, 건강보험, 고용보험, 산재보험",
    "PIP": "성과개선계획",
    "반차": "반일 연차",
    # ... 12개 항목
}

def expand_abbreviations(query):
    expanded = query
    changes = []
    for abbrev, full_form in ABBREVIATION_MAP.items():
        if abbrev in expanded:
            expanded = expanded.replace(abbrev, full_form)
            changes.append((abbrev, full_form))
    return expanded, changes

```

“WFH 정책이 뭐야?” → “재택근무 규정이 뭐야?”로 바뀝니다. LLM 호출 없이 문자열 치환만으로 됩니다.

동어의 확장은 원본 쿼리를 여러 변형으로 늘립니다. “연차 사용 규정을 알려주세요” 하나가 “유급휴가 사용 규정을 알려주세요”, “휴가 사용 규정을 알려주세요” 등으로 확장됩니다.

```
python -m tuning.step2_query_rewrite --step 2-1
```

[이미지: 약어/동어의 확장 실험] 그림 9-6: WFH→재택근무, 4대보험→국민연금/건강보험/고용보험/산재보험으로 자동 확장. LLM 없이 즉시 적용 가능하다.

```

# 다른 질문으로 테스트
python -m tuning.step2_query_rewrite --step 2-1 --query "반반차 신청 방법"

```

이 사전은 조직마다 다릅니다. 도입 후 첫 한 달간 사용자들이 자주 쓰는 약어를 모아서 사전에 추가하면 됩니다.

실험 2-2: HyDE — 답을 먼저 상상하고, 그 답과 비슷한 문서를 찾는다

같은 파일의 `compare_hyde_vs_direct()` 함수입니다. 가상 답변 생성에 사용하는 `HYDE_TEMPLATES`는 `data.py`에 정의되어 있습니다. HyDE는 직관적이지 않은 기법입니다. 질문을 검색하는 게 아니라, **가상의 답변을 생성해서 그 답변으로 검색**합니다.

왜 이게 효과적일까요? 질문(“휴가 규정 알려줘”)과 문서(“제15조 연차유급휴가. 사용자는 1년간 80퍼센트 이상 출근한 근로자에게…”)는 표현이 많이 다릅니다. 질문은 짧고 구어체이고, 문서는 길고 공문서체입니다. 임베딩 벡터 간 거리가 멀 수 있습니다. 하지만 가상 답변(“연차유급휴가 규정에 따르면, 직원은 1년 이상 근무 시 15일의 유급휴가를 받습니다”)은 실제 문서와 표현이 비슷합니다. 가상 답변의 임베딩으로 검색하면 실제 문서가 더 잘 잡힙니다. 핵심은 직접 검색과 HyDE 검색의 **임베딩 유사도 비교**입니다.

```
def compare_hyde_vs_direct(query, documents, embeddings):
    doc_vectors = embeddings.embed_documents(documents)

    # 직접 검색: query → document
    query_vec = embeddings.embed_query(query)
    direct_scores = [_cosine_similarity(query_vec, dv) for dv in doc_vectors]

    # HyDE 검색: hypothetical → document
    hypo_doc = generate_hypothetical_document(query)
    hypo_vec = embeddings.embed_query(hypo_doc)
    hyde_scores = [_cosine_similarity(hypo_vec, dv) for dv in doc_vectors]

    # 문서별 direct vs hyde 점수 비교
    for i, doc in enumerate(documents):
        results.append({
            "direct_score": direct_scores[i],
            "hyde_score": hyde_scores[i],
            "improvement": hyde_scores[i] - direct_scores[i],
        })
```

10개 실제 문서에 대해 “질문으로 직접 검색한 유사도”와 “가상 답변으로 검색한 유사도”를 나란히 비교합니다.

```
python -m tuning.step2_query_rewrite --step 2-2
```

[이미지: HyDE 실험] 그림 9-7: “연차 신청 절차” 질문에서 직접 검색(0.56) vs HyDE(0.91). 가상 문서가 실제 문서와 어투가 비슷해서 유사도가 크게 올라간다.

HyDE가 항상 좋은 건 아닙니다. 가상 문서가 잘못되면 오히려 관련 없는 문서를 가져올 수 있습니다. HyDE는 “답변이 구체적인 사실 문서로 존재하는” 환경(사내 규정, 법률 문서 등)에서 가장 효과적입니다.

실험 2-3: Multi-Query — 한 질문을 여러 각도로

같은 파일의 `generate_multi_queries()`와 `search_multi_query()` 함수입니다.

한 질문을 여러 표현으로 바꿔서 각각 검색한 다음, 결과를 합칩니다. 원본 쿼리로는 못 잡는 문서를 변형 쿼리로 잡을 수 있습니다.

```
def search_multi_query(queries, documents, embeddings, top_k=3):
    doc_vectors = embeddings.embed_documents(documents)

    all_results = {}
    for query in queries:
        q_vec = embeddings.embed_query(query)
        scores = [(_cosine_similarity(q_vec, dv), i)
```

```

        for i, dv in enumerate(doc_vectors)]
    scores.sort(key=lambda x: x[0], reverse=True)
    all_results[query] = scores[:top_k]

# 병합 + 중복 제거: 같은 문서가 여러 쿼리에서 잡히면 최고 점수 유지
doc_best_score = {}
for query, results in all_results.items():
    for score, idx in results:
        if idx not in doc_best_score or score > doc_best_score[idx]:
            doc_best_score[idx] = score

```

“연차 신청 절차”라는 질문을 “유급휴가 신청 절차”, “연차 사용 규정”, “연차 신청 절차에 대한 규정이 있습니까?” 등으로 변형합니다. 각 변형으로 검색한 결과를 합치면, 단일 쿼리로는 놓치던 문서까지 잡을 수 있습니다.

```
python -m tuning.step2_query_rewrite --step 2-3
```

[이미지: Multi-Query 실험] 그림 9-8: 원본 1개 쿼리 → 4개 변형으로 검색 범위 확대. 병합 후 중복 제거로 고유 문서를 확보한다.

```

# 변형 수를 바꿔보기
python -m tuning.step2_query_rewrite --step 2-3 --num_queries 5

# 다른 질문으로 테스트
python -m tuning.step2_query_rewrite --step 2-3 --query "재택근무 조건"

```

2.5 더 알아보기

약어 사전은 수동 관리가 필요합니다. ABBREVIATION_MAP과 SYNONYM_MAP은 하드코딩된 사전입니다. 운영 환경에서는 관리자 UI에서 사전을 편집할 수 있게 만들거나, 사용자 질문 로그에서 빈번한 미매칭 패턴을 분석해서 사전을 갱신하는 방법이 있습니다.

기법 선택 가이드입니다. 어떤 기법을 먼저 적용할지 헛갈린다면 이 순서를 추천합니다.

순서	기법	비용	효과	적용 조건
1	약어/동义词 확장	없음 (사전만)	즉각적	사내 약어가 있다면 무조건
2	ParentDocument	인덱싱 시 추가 매핑	문맥 보강	원본이 길고 조각이 짧을 때
3	HyDE	LLM 1회 호출	높음	사실 기반 문서(규정, 법률 등)
4	Multi-Query	LLM 1회 + 검색 N회	누락 방지	질문이 모호하거나 넓을 때

순서	기법	비용	효과	적용 조건
5	SelfQuery	필터 추출	범위 축소	메타데이터가 잘 정리된 환경
6	Compression	임베딩 비교	토큰 절약	문서가 길고 토큰 제한이 뻥뻥할 때

2.6 CH08과 CH09의 조합

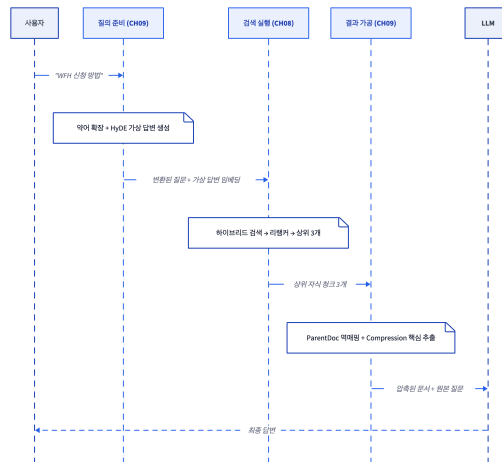
CH08과 CH09는 **같은 파이프라인의 다른 단계**를 다룹니다. CH09가 질문을 준비하고, CH08이 검색하고, 다시 CH09가 결과를 가공합니다. 세 단계가 어떻게 맞물리는지 아래 그림에서 확인합니다.

[이미지: CH08과 CH09 파이프라인 조합] 그림 9-9: 질의 전략(CH09, 파란색) → 검색 전략(CH08, 주황색) → 결과 가공(CH09, 초록색). 파이프라인에서 LLM 호출이 필요한 단계는 빨간 별표로 표시.

단계	챕터	담당 기법	LLM 필요	역할
질의 준비 1	CH09	약어/동의어 확장	아니오	사내 용어를 정식 명칭으로 변환
질의 준비 2	CH09	HyDE / Multi-Query	HyDE: 예	검색에 유리한 형태로 질문을 재구성
검색 실행 1	CH08	하이브리드 검색	아니오	BM25 + 벡터로 자식 청크 후보 검색
검색 실행 2	CH08	리랭커	예	Cross-Encoder로 후보를 재정렬
결과 가공 1	CH09	ParentDoc / SelfQuery	아니오	자식 청크 → 부모 문서 역매핑
결과 가공 2	CH09	Compression	아니오	선별된 문서에서 관련 문장만 추출
답변	—	LLM	예	최종 문서를 보고 답변 생성

시맨틱 청킹(CH08)과 부모-자식 매핑(CH09 ParentDoc)은 **인덱싱 시점의 사전 작업**입니다. 문서를 처음 저장할 때 시맨틱 청킹으로 자식 청크를 만들고, 동시에 부모-자식 매핑을 설정합니다. 위 파이프라인은 검색 시점의 흐름입니다.

“WFH 신청 방법”이 실제로 이 파이프라인을 타는 순서입니다.



전체 파이프라인 시퀀스: 질의 준비 → 검색 실행 → 결과 가공 → 최종 답변

2.7 이것만은 기억하세요

- **검색 엔진의 앞뒤를 모두 다룹니다.** 검색 전에는 질문을 바꾸고(약어 확장, HyDE, Multi-Query), 검색 후에는 결과를 가공합니다 (ParentDoc, Compression). CH08이 검색 엔진 자체를 튜닝했다면, CH09는 검색 엔진에 넣는 입력과 꺼내는 출력을 튜닝합니다.
- **검색은 작게, 반환은 크게.** Parent Document Retriever처럼 짧은 청크로 정밀 검색하고 긴 원본을 돌려주면, 검색 정확도와 답변 품질을 동시에 올릴 수 있습니다.
- **가장 쉬운 것부터.** 약어/동어의 사전은 LLM 호출 없이 즉시 적용 가능합니다. 사내 용어가 있다면 가장 먼저 도입하세요.

Ch.10: “PDF 이미지까지 잡아라” — 고급 문서 처리와 평가 (ex10)

한 줄 요약: 측정해야 개선할 수 있다. 느낌이 아니라 숫자로 품질을 측정해야 진짜 개선할 수 있다.
핵심 개념: 비전 LLM(Vision LLM), 광학 문자 인식(OCR), 정확도(Precision@k)

[이미지: 사서의 눈 + 성적표]

1.1 표가 깨졌다

사서를 뽑고, 훈련시키고, 전문 기술까지 가르쳤습니다. 이제 남은 건 하나 — 사서가 정말 일을 잘하는지 시험을 보는 것입니다.

CH04에서 PDF를 파싱했습니다. pypdf로 텍스트를 추출하고, 청킹하고, 벡터DB에 넣었습니다. 텍스트가 주된 문서에서는 잘 동작했습니다. 어느 날 팀장이 PDF 파일 하나를 던져줍니다.

팀장: “이 정보보안서약서도 검색되게 해줘.”

별생각 없이 기존 파이프라인에 넣습니다. pypdf로 텍스트 추출하고, 청킹하고, 벡터DB에 넣는 그 흐름.

(빈 문자열)

…아무것도 안 나온다? PDF를 직접 열어봅니다.

[이미지: 정보보안서약서 원본] 그림 10-0: 팀장이 건넨 정보보안서약서. 결재란, 도장, 하이라이트가 있는 전형적인 사내 문서.

스캔본이었습니다. 종이 문서를 스캐너로 찍어서 PDF로 만든 것입니다. 전체가 하나의 커다란 이미지입니다. 결재란에 서명이 있고, 빨간 도장도 찍혀 있고, 금지 사항에 하이라이트까지 되어 있습니다. pypdf는 PDF 안의 텍스트 레이어를 읽는 도구입니다. CH04에서 언급했던 pdfplumber(표 추출 전용 라이브러리)도 마찬가지입니다. 둘 다 PDF 파일 구조에서 텍스트를 꺼내는 라이브러리입니다. 그런데 스캔본에는 텍스트 레이어가 없습니다. 사진을 아무리 뒤져봐야 글자 데이터는 없습니다.

결재란에 누가 서명했는지, 금지 사항이 뭔지, 공개등급이 뭔지 — 사람은 한눈에 다 보이는데.

사람이 PDF를 열면 “이건 대외비 문서고, AI 도구 사용에 제한이 있구나”를 바로 파악합니다. 지금까지 우리 사서가 문서를 읽는 방법은 pypdf뿐이었습니다. 글자 데이터가 있는 문서는 잘 읽었지만 사진 앞에서는 속수무책입니다.

1.2 사서에게 눈을 달아주자

지금까지 우리 사서는 글만 읽었습니다. 텍스트를 추출하고, 벡터로 바꾸고, 검색해서 답했습니다. 하지만 사내 문서에는 글이 아닌 정보가 많습니다.

- 조직도 (박스와 화살표)
- 매출 추이 차트 (막대그래프, 선그래프)
- 복잡한 표 (셀 병합, 다단 구조)
- 스캔한 종이 문서 (아예 이미지만 있는 PDF)

이런 문서에서 텍스트만 뽑으면 정보의 절반을 잃습니다.

해결 방법은 두 가지입니다.

방법 1: 광학 문자 인식(OCR) — 이미지 속 글자를 읽는다

광학 문자 인식(OCR, Optical Character Recognition)은 이미지에서 글자를 인식하는 기술입니다. 스캔한 종이 문서처럼 텍스트가 이미지 형태로 박혀 있을 때 씁니다. 사서에게 **확대경**을 준 것과 같습니다. 작은 글씨를 읽게 해주지만, 조직도의 “박스 안에 있다”거나 표의 “이 셀과 저 셀의 관계”까지는 이해하지 못합니다.

방법 2: 비전 LLM(Vision LLM) — 이미지를 이해한다

그래서 등장하는 것이 비전 LLM(Vision LLM)입니다. 텍스트만 이해하던 LLM에 이미지를 볼 수 있는 능력을 추가한 모델입니다. 이미지를 보고 “이건 조직도이고, 인사팀은 경영지원본부 산하입니다”라고 설명할 수 있습니다. 글자를 읽는 것이 아니라 **그림을 이해**하는 것입니다. 사서에게 **눈**을 달아준 셈입니다. 확대경은 글씨를 읽게 해주지만, 눈은 그림 전체를 이해하게 해줍니다.

그럼 처음부터 비전 LLM한테 다 말하면 안 되나?

솔직한 의문입니다. 확대경이 있다가 놓치는 것을 눈이 다 잡아주는데, 그냥 눈만 쓰면 되지 않을까. 실제로 요즘 추세가 그렇습니다. OCR을 아예 건너뛰고 처음부터 비전 LLM에게 문서를 통째로 보여주는 방식이 늘고 있습니다. 확대경 없이 눈만 쓰는 사서입니다.

문제는 시간입니다. 눈으로 한 페이지를 읽는 데 10초씩 걸립니다. 사내 문서가 수백 페이지면 한참 기다려야 합니다. 확대경은 같은 페이지를 1~2초면 읽습니다. 문서의 구조에 맞춰 전략적으로 선택할 수 있어야 합니다. 그런데 확대경이 글자를 읽긴 읽었는데...

원본: 기안 검토 승인 정보보안 서약서 (스캔본) 문서번호: 2026

OCR: 기인 승인 정보보안 서사서 (스스년) 단서면요 2026

“서약서”가 “서사서”로, “스캔본”이 “스스년”으로 읽혔습니다. 처음부터 눈으로 봤으면 그런 일은 없었을 겁니다.

이것이 하이브리드의 어려운 점입니다. 확대경이 뭔가를 읽어왔다고 해서 그게 정확한 글자인지는 또 다른 문제입니다. 글자 수만 세서는 품질까지 판단할 수 없습니다. 기술 파트에서 이 문제를 어떻게 풀어가는지 직접 실험해봅니다.

[이미지: OCR vs Vision LLM] 그림 10-1: 광학 문자 인식(OCR)은 이미지 속 글자를 읽고, 비전 LLM은 이미지의 의미를 이해한다.

1.3 매번 골라야 해?

OCR은 빠르지만 표나 도장은 못 읽습니다. 비전 LLM은 다 이해하지만 느립니다. 문서가 10개면 하나하나 “이건 OCR, 이건 비전 LLM”이라고 골라줘야 할까요?

...귀찮은데. 사서가 문서를 펼쳤을 때 스스로 판단하게 하면 됩니다. “이 페이지는 글자가 잘 뵈히니까 OCR로 충분하고, 이 페이지는 이미지뿐이니까 눈으로 봐야겠다.” 사서에게 **판단력**까지 주는 것입니다.

1.4 그런데, 진짜 좋아진 건 맞아?

CH08에서 청킹을 바꿔보고, 리랭킹을 적용하고, 하이브리드 검색을 도입했습니다. CH09에서 질의 재작성(Query Rewrite)과 근거 시스템까지 추가했습니다. 그때마다 “오, 좋아졌다”고 느꼈습니다. 그런데 **느낌**입니다. “좋아진 것 같다”와 “85점에서 92점으로 올랐다”는 완전히 다른 이야기입니다.

팀장이 묻습니다.

팀장: “AI 비서 검색 정확도가 몇 퍼센트야?”

나: “음... 꽤 좋아졌는데요...”

팀장: “숫자로.”

느낌이 아니라 측정이 필요합니다. 학교에서 시험을 보면 성적표가 나오듯이, AI 비서에게도 성적표가 필요합니다.

이것이 **평가 프레임워크(Evaluation Framework)**입니다. 질문을 던지고, 나온 답을 정답과 비교해서 점수를 매깁니다.

- **정확도(Precision@k):** 가져온 문서 k개 중 정답이 몇 개인가?
- **재현율(Recall@k):** 정답 문서 전체 중 몇 개를 찾았는가?

- **MRR(Mean Reciprocal Rank)**: 첫 번째 정답이 몇 번째에 나왔는가?
- **환각률(Hallucination Rate)**: 답변에 출처 없는 내용이 섞여 있는가?

성적표가 있으면 “리랭킹을 적용하니깐 정확도(Precision@3)가 0.60에서 0.85로 올랐다”고 말할 수 있습니다. 느낌이 아니라 숫자입니다.

[이미지: 평가 프레임워크] 그림 10-2: RAG 엔진의 성적표. 질문마다 검색 정확도와 환각률을 수치로 측정한다.

1.5 성적표를 받았으니, 진짜 서비스를 만들자

성적표가 나왔습니다. 점수가 마음에 들든 안 들든, 이제 할 일은 명확합니다. 지금까지 만든 기술을 하나로 엮어서 **사람이 쓸 수 있는 화면**을 만드는 것입니다.

CH08의 하이브리드 검색, CH09의 약어 확장과 리랭킹 — 이 기술들이 코드 안에 흩어져 있습니다. 이걸 웹 UI 하나로 합치면, 브라우저에서 질문을 던지고 답변과 원본 문서 이미지를 바로 확인할 수 있습니다. 그리고 성적표로 “정말 좋아졌는지” 숫자로 비교합니다.

1.6 이번 버전에서 뭘 만드나

ex10은 마지막 버전입니다. 네 개 실습을 순서대로 따라가면 이미지까지 이해하는 RAG 챗봇이 완성됩니다.

실습	기능	비유	코드
1	OCR vs 비전 LLM 비교	사서의 읽기 방식 비교	<code>tuning/step1_document_parser/</code>
2	하이브리드 이미지 처리	사서의 자동 판단	<code>tuning/step2_hybrid_parser/</code>
3	RAG 평가 프레임워크	사서의 성적표	<code>tuning/step3_eval_framework/</code>
4	문서 캡처와 근거 표시	사서의 작업 공간	<code>src/</code>

실습 1에서 OCR과 비전 LLM 두 가지 읽기 방식을 비교합니다. 실습 2에서는 이 둘을 자동으로 선택하는 하이브리드 파이프라인을 만듭니다 — OCR로 충분하면 OCR, 스캔 이미지라 글자가 안 나오면 비전 LLM으로 전환하는 방식입니다. 실습 3에서 성적표를 만들어 품질을 숫자로 증명합니다.

마지막 실습 4에서 CH08~09 기술이 적용된 웹 UI를 직접 만들면서, 문서 캡처·근거 이미지 표시·검색 성능 비교까지 완성합니다.

1.7 전체 프로젝트 회고

CH01에서 LLM에게 “우리 회사 휴가 규정 알려줘”라고 물었습니다. LLM은 그럴듯하게 거짓말했습니다. 환각이었습니. “아, LLM은 우리 회사 문서를 모르는구나.” 그 깨달음에서 이 여정이 시작됐습니다.

CH02~CH03에서 사내 시스템의 기반을 다졌습니다. 직원, 연차, 매출 데이터를 API로 만들고, 사내 문서의 표준을 정했습니다. CH04에서 문서를 벡터로 바꾸는 기술을 배웠고, CH05에서 드디어 질문하면 답해주는 RAG 엔진을 완성했습니다.

하지만 “연차 몇 개? 규정은?”이라는 복합 질문에는 답하지 못했습니다. DB와 문서를 동시에 볼 수 없었으니까요. CH06에서 에이전트를 만들어 이 문제를 해결했고, CH07에서는 캐시와 모니터링으로 운영 안정성을 갖췄습니다.

CH08부터 본격적인 튜닝이 시작됐습니다. “영동한 문서를 가져온다”는 문제를 청킹 최적화, 리랭킹, 하이브리드 검색으로 해결했습니다. CH09에서 질문 자체를 재구성하고 답변에 근거를 붙이는 기술을 추가했습니다. 그리고 이번 CH10에서 이미지까지 이해하는 사서를 만들고, 성적표로 품질을 수치화합니다.

돌이켜보면 이 책은 하나의 질문에서 출발했습니다. “**시가 우리 회사 문서를 알게 하려면 어떻게 해야 하지?**” 그 질문의 답이 RAG이고, 10개 챕터가 그 답을 점진적으로 완성해가는 과정이었습니다.

환각을 보고 → 문서를 넣고 → 검색하고 → 답변하고 → 통합하고 → 안정화하고 → 튜닝하고 → 측정하고. 이 흐름 자체가 실무에서 AI 시스템을 만드는 과정과 같습니다.

이제 실습으로 PDF 이미지 파싱과 품질 평가를 구현해보겠습니다.

2.1 용어 정리

이야기 속 표현	진짜 이름	정의
사서의 확대경	광학 문자 인식(OCR, Optical Character Recognition)	이미지에서 문자를 인식하는 기술. EasyOCR 등의 엔진이 이미지 속 글자 위치와 내용을 추출한다
사서의 눈	비전 LLM(Vision LLM)	이미지를 입력으로 받아 내용을 이해하고 설명하는 멀티모달 (Multimodal) 대형 언어 모델. Qwen2.5-VL, LLaVA, MiniCPM-V(로컬), GPT-4o, Gemini Pro Vision(클라우드) 등이 대표적
확대경+눈 자동 전환	하이브리드 이미지 처리 (Hybrid Image Processing)	OCR을 먼저 시도하고, 텍스트가 부족하면 비전 LLM으로 자동 전환하는 전략. 속도와 품질을 동시에 확보한다
사서의 성적표 — 정확도	정확도(Precision@k)	검색된 상위 k개 문서 중 실제 관련 문서의 비율. k=3일 때 3개 중 2개가 정답이면 Precision@3 = 0.67
사서의 성적표 — 재현율	재현율(Recall@k)	전체 정답 문서 중 상위 k개 안에 포함된 비율. 정답 4개 중 2개를 찾았으면 Recall@3 = 0.50
사서의 성적표 — MRR	MRR(Mean Reciprocal Rank)	첫 번째 정답 문서가 검색 결과 몇 번째에 등장하는지의 역수. 1위에 정답이 있으면 1.0, 3위에 있으면 0.33
사서의 성적표 — 환각률	환각률(Hallucination Rate)	답변에서 출처 문서에 근거하지 않은 내용의 비율. 0에 가까울수록 좋다
OCR 파싱	EasyOCR	pip만으로 설치 가능한 오픈소스 OCR 라이브러리. PyTorch 기반이며 한국어를 포함해 80개 이상 언어를 지원한다

이야기 속 표현	진짜 이름	정의
비전 LLM 파싱	Qwen2.5-VL (Qwen 2.5 Vision-Language)	Ollama에서 실행 가능한 오픈소스 비전 LLM. 한국어를 공식 지원하며, 이미지를 base64로 인코딩하여 전달하면 구조화된 설명을 생성한다

2.2 실습 환경 구축

기본 환경(Python 3.12, Ollama, Docker)이 없다면 **부록(환경 설정)** 을 먼저 참고하세요.

```
cd ex10
cp .env.example .env
python3.12 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
docker compose up -d      # PostgreSQL 실행 (실습 4에서 필요)

# 비전 LLM 모델 (실습 1, 2에서 사용)
ollama pull qwen2.5vl:7b

# 텍스트 LLM 모델 (실습 3 환각률 측정에서 사용)
ollama pull llama3.1:8b

pip install -r requirements.txt
```

이전 챕터 Docker 종료: CH07의 Docker가 실행 중이라면 `cd ex07 && docker compose down` 으로 먼저 종료하세요.

OCR 엔진 **EasyOCR** 은 `requirements.txt`에 포함되어 있어 별도 시스템 패키지 설치가 필요 없습니다. 비전 LLM은 컴퓨터 사양이 부족하면 `.env`에서 `VISION_PROVIDER=openai`로 전환할 수 있습니다.

패키지	역할
PyMuPDF	PDF → 이미지 렌더링 + 텍스트 추출
easyocr	OCR 파싱 (한국어+영어)
chromadb	벡터DB 저장
rank-bm25	BM25 키워드 검색 (하이브리드 검색)
sentence-transformers	Cross-Encoder 리랭킹
httpx	Ollama API 호출 (실습 2, 3)

2.3 파일 계층 구조

```

ex10/
├── tuning/                                <- 실험 코드
│   ├── step1_document_parser/           [실습 1] OCR vs 비전 LLM 파싱 비교
│   ├── step2_hybrid_parser/             [실습 2] 하이브리드 이미지 처리 (OCR+Vision 자동 선택)
│   └── step3_eval_framework/            [실습 3] Precision@k, Recall@k, 환각률 평가
├── src/                                  <- CH05~09 기술 통합 + 실습 4 코드
│   ├── capture.py                       [실습 4] 새로 작성 - PDF 캡처
│   ├── evidence.py                      [실습 4] 새로 작성 - 근거 이미지 URL 변환
│   ├── agent_config.py                  [참고] 에이전트 설정 (CH06)
│   ├── tools/search_documents.py        [설명] 검색 파이프라인 (CH08~09)
│   └── ...
├── app/                                  [참고] FastAPI 웹 앱 (완성본)
├── data/                                  [참고] 사내 문서 원본 + 평가 질문 세트
├── docker-compose.yml                   [참고] PostgreSQL (ex07 동일)
└── run.py                               [참고] 서버 실행 진입점

```

실습 1~3은 **tuning/**에서 실험하고, 실습 4는 **src/**에 **새 파일을 직접 만들어** 작성합니다. 기존 파일(**agent_config.py**, **tools/** 등)은 CH05~09에서 만든 코드로 수정하지 않습니다.

2.4 실습 순서

1. OCR vs 비전 LLM 비교
2. 하이브리드 처리 구현
3. 평가 프레임워크
4. 완성 웹 UI

OCR과 비전 LLM을 비교하고(실습 1), 둘을 자동 선택하는 하이브리드 처리를 구현합니다(실습 2). 평가 프레임워크로 기준선을 잡고(실습 3), 마지막으로 CH08~09 기술이 통합된 웹 UI를 직접 만들면서 성능 변화를 확인합니다(실습 4).

2.5 실습 1: 문서 파싱 전략 비교 (tuning/step1_document_parser/)

이야기 파트에서 정보보안서약서 스캔본이 pypdf로 파싱되지 않는 문제를 경험했습니다. 스캔 PDF에서 텍스트를 뽑는 방법은 크게 두 가지입니다 — OCR(광학 문자 인식)과 비전 LLM. 같은 스캔 PDF를 두 방식으로 파싱하고 결과를 비교합니다.

파라미터	설명	권장값
--step	실험 단계 (1~1: OCR, 1~2: 비전 LLM)	—
--pdf_path	파싱할 PDF 경로	data/docs/hr/HR_정보보안서약서.pdf
--dpi	이미지 렌더링 해상도	150

파라미터	설명	권장값
<code>--timeout</code>	비전 LLM 타임아웃 초 (1-2 전용)	200

실험 1-1: OCR 파싱 — PyMuPDF + EasyOCR

OCR(Optical Character Recognition)은 이미지에서 글자를 인식하는 기술입니다. 스캔본처럼 텍스트 레이어가 없는 PDF도 읽을 수 있습니다. **EasyOCR**은 PyTorch 기반의 오픈소스 OCR 라이브러리로, 한국어를 포함해 80개 이상의 언어를 지원합니다.

`ex10/tuning/step1_document_parser/parser.py`의 `parse_pdf_ocr()` 함수가 핵심입니다. PDF 페이지를 이미지로 바꾸고, EasyOCR에게 “이 이미지에서 글자를 찾아줘”라고 말합니다.

```
def parse_pdf_ocr(pdf_path, dpi=150):
    reader = easyocr.Reader(["ko", "en"], gpu=False)
    doc = fitz.open(str(pdf_path))
    page_texts = []
    for page_num in range(len(doc)):
        page = doc[page_num]
        pix = page.get_pixmap(dpi=dpi)          # 페이지 → 이미지 렌더링
        img = Image.open(io.BytesIO(pix.tobytes("png")))
        img_array = np.array(img)

        ocr_results = reader.readtext(img_array, detail=0) # OCR
        page_texts.append("\n".join(ocr_results))
    return {"text": "\n\n".join(page_texts)}
```

`Reader(["ko", "en"])` — 한국어와 영어를 동시에 인식합니다. `gpu=False`는 CPU만으로 실행하는 옵션입니다. `detail=0`은 텍스트만 반환하고 좌표는 생략합니다.

```
python -m tuning.step1_document_parser --step 1-1
```

[이미지: 실험 1-1: OCR 파싱 결과] 그림 10-2: OCR 파싱 결과. 스캔본에서 텍스트를 추출했지만 구조(표, 항목 번호)가 뭉개진다.

OCR은 글자를 찾아내지만, 두 가지 한계가 있습니다. 첫째, 문서의 **구조**를 모릅니다. 표의 셀 경계를 인식하지 못하고, 조항 번호와 본문이 한 줄로 이어집니다. 결재란의 도장이나 서명도 무시합니다. 둘째, 글자 자체가 **깨지는** 경우가 많습니다. 스캔 품질이 낮거나 한국 어처럼 획이 복잡한 문자에서는 “서약서”가 “서사서”으로, “스캔본”이 “스스년”으로 인식되기도 합니다. ICCV 2025에서 발표된 연구에 따르면, 최고 성능의 OCR 엔진도 원본 텍스트 대비 7.5% 이상의 정확도 차이가 발생합니다. 결국 OCR만으로는 “읽을 수 있는 문서”가 되지 않습니다. 다른 PDF로도 직접 확인해보세요.

다른 PDF로 실험

```
python -m tuning.step1_document_parser --step 1-1 --pdf_path data/docs/ops/OPS_신규서비스_런칭전략.pdf
```

```
# DPI를 높여서 실험 — 글자 인식이 올라가는지 확인
python -m tuning.step1_document_parser --step 1-1 --dpi 200
```

실험 1-2: 비전 LLM 파싱 — PyMuPDF + Qwen2.5-VL

비전 LLM은 여러 종류가 있습니다. 로컬에서 돌릴 수 있는 모델과 클라우드 API 모델로 나뉩니다.

모델	실행 환경	한국어	비고
Qwen2.5-VL 7B	Ollama (로컬)	O	이 책에서 사용. 한국어 공식 지원
LLaVA 7B	Ollama (로컬)	X	영어만. 한국어 문서 인식 불가
MiniCPM-V	Ollama (로컬)	△	한국어 일부 지원. 정확도 낮음
GPT-4o / GPT-4o-mini	OpenAI API	O	빠르고 정확. API 비용 발생
Gemini Pro Vision	Google API	O	한국어 우수. API 비용 발생

로컬 모델의 속도는 하드웨어에 따라 크게 달라집니다. 같은 Qwen2.5-VL 7B라도 RAM 16GB 환경에서 25초, 8GB 환경에서는 수 분이 걸릴 수 있습니다. 또한 Ollama에 다른 모델이 VRAM에 올라가 있으면 메모리 경합으로 속도가 급격히 느려지니, 비전 실험 전에 **ollama ps** 로 확인하세요.

이 책에서는 Ollama로 로컬 실행이 가능한 **Qwen2.5-VL 7B** 를 사용합니다. API 비용이 없고, 사내 문서를 외부 서버로 보내지 않아도 됩니다. 무엇보다 **한국어를 공식 지원**하는 로컬 비전 모델이라 한국어 사내 문서를 정확하게 읽어냅니다.

내 컴퓨터 사양이 부족하다면? Qwen2.5-VL 7B 모델은 RAM 10GB 이상을 권장합니다. 다른 모델이 VRAM에 올라가 있으면 속도가 급격히 느려지니, 비전 실험 전에 **ollama ps**로 확인하세요. 모델 로딩이 너무 느리거나 메모리 부족 오류가 나면, **.env** 파일에서 **VISION_PROVIDER=openai**로 바꿔 GPT-4o-mini 비전 API를 사용할 수 있습니다.

같은 파일의 **parse_pdf_vllm()** 함수입니다. OCR과 마찬가지로 PDF 페이지를 이미지로 바꾸지만, 이번에는 비전 LLM에게 “이 이미지를 분석해줘”라고 말합니다. 글자만 찾는 OCR과 달리, 비전 LLM은 문서의 구조, 표, 도장까지 이해합니다.

```
def parse_pdf_vllm(pdf_path, dpi=150):
    doc = fitz.open(str(pdf_path))
    page_texts = []
    for page_num in range(len(doc)):
        page = doc[page_num]
        pix = page.get_pixmap(dpi=dpi) # 페이지 → 이미지 렌더링
        img_path = f"_vllm_page_{page_num + 1}.png"
        pix.save(img_path)

        caption = _call_vision_llm(img_path) # 비전 LLM에게 이미지 분석 요청
        page_texts.append(caption)
```

```
Path(img_path).unlink() # 임시 이미지 삭제
return {"text": "\n\n".join(page_texts)}
```

`get_pixmap(dpi=150)` — PDF 페이지를 통째로 이미지로 렌더링합니다. DPI(Dots Per Inch)는 1인치당 점의 개수, 즉 해상도입니다. 150이면 A4 한 페이지가 약 1240 × 1754 픽셀(Pixel)이 됩니다. 72로 낮추면 빠르지만 글자가 흐릿하고, 300으로 높이면 선명하지만 이미지 용량이 커져서 비전 LLM 처리 시간이 늘어납니다. 150은 표의 셀 구분이나 도장까지 식별할 수 있는 적절한 균형점입니다. `_call_vision_llm()`이 비전 LLM(기본 Ollama Qwen2.5-VL, 또는 OpenAI GPT-4o-mini)에 이미지를 베이스64(base64) 인코딩(Encoding)해서 보냅니다.

프롬프트(Prompt)가 중요합니다 — “마크다운 형식으로 출력하세요”가 없으면 비전 LLM이 자유 형식으로 답하기 때문에, 후속 청킹과 검색에 불리합니다. Qwen2.5-VL은 한국어를 지원하므로 프롬프트도 한국어로 작성합니다.

```
_VISION_PROMPT = (
    "이 문서 이미지를 분석하세요. "
    "텍스트, 표, 도장, 서명, 구조적 요소를 빠짐없이 추출하세요. "
    "결과는 마크다운 형식으로 출력하세요. "
    "표는 마크다운 테이블 문법을 사용하세요. "
)
```

타임아웃(Timeout)은 기본 600초로 넉넉하게 잡아두었습니다. 비전 LLM의 처리 시간은 컴퓨터 사양에 따라 크게 달라집니다 — 빠른 머신에서는 25초면 끝나지만, CPU만으로 돌리면 5분 이상 걸릴 수도 있습니다. 타임아웃 오류가 나면 `--timeout 900` 처럼 더 늘려주면 됩니다.

```
python -m tuning.step1_document_parser --step 1-2
```

```
# 타임아웃 늘리기 (첫 실행 시 모델 로딩이 오래 걸리면)
```

```
python -m tuning.step1_document_parser --step 1-2 --timeout 900
```

[이미지: 실험 1-2: 비전 LLM 파싱 + 비교표] 그림 10-3: 같은 스캔 PDF를 OCR과 비전 LLM으로 파싱한 결과. OCR은 텍스트만, 비전 LLM은 구조화된 마크다운을 추출했다.

캡처 결과를 보면, 비전 LLM이 문서번호(2026-HR-SEC-002), 조항(제4조 생성형 AI 활용 지침), 금지 사항까지 마크다운 구조로 정리해냈습니다. OCR이 같은 내용을 평문으로 뽑아낸 것과 비교해보세요.

항목	OCR (EasyOCR)	비전 LLM (Qwen2.5-VL)
속도	약 13~50초	약 25~180초
텍스트 추출	평문 (구조 없음)	마크다운 (구조화)
표 인식	셀 경계 무시	마크다운 테이블
도장/서명	무시	시각 요소 설명
후속 검색 품질	보통	높음

다른 PDF와 DPI를 바꿔서 직접 비교해보세요.

```
# 다른 PDF로 실험 — 문서마다 차이가 다르다
python -m tuning.step1_document_parser --step 1-2 --pdf_path data/docs/ops/OPS_신규서비스_원청전략.pdf

# DPI를 높여서 실험 (정밀도↑ 속도↓) — 150 vs 200, 추출량이 달라지는지 확인
python -m tuning.step1_document_parser --step 1-2 --dpi 200

# 두 전략을 한번에 비교
python -m tuning.step1_document_parser --step all
```

전략 선택 가이드

상황	추천 전략
빠르게 텍스트만 필요	OCR (EasyOCR) — 빠르지만 구조 없음
구조/표/서명까지 필요	비전 LLM — 느리지만 마크다운 구조화
대량 문서 처리	OCR로 1차 필터링 후, 중요 문서만 비전 LLM
실무 최적 조합	OCR + 비전 LLM 하이브리드

2.6 실습 2: 하이브리드 이미지 처리 (tuning/step2_hybrid_parser/)

실습 1에서 OCR과 비전 LLM을 비교했습니다. OCR은 빠르지만 구조를 모르고, 비전 LLM은 구조까지 파악하지만 느립니다. 실무에서는 둘 중 하나만 고르는 것이 아니라 **자동으로 선택**하게 만듭니다.

이 실습에서는 자동 선택의 두 가지 방식을 직접 실험합니다. 첫 번째 방식의 한계를 확인하고, 두 번째 방식으로 개선하는 흐름입니다.

파라미터	설명	권장값
<code>--step</code>	실험 단계 (2-1, 2-2, all)	—
<code>--pdf</code>	파싱할 PDF 경로	<code>data/docs/hr/HR_정보보안서약서.pdf</code>
<code>--threshold</code>	OCR 판정 기준 글자 수	50

실습 2-1: OCR 글자 수 기반 하이브리드

가장 직관적인 방법부터 시작합니다. OCR로 먼저 시도해서 텍스트가 충분히 나오면 그대로 쓰고, 글자가 거의 없으면 비전 LLM으로 전환합니다. `ex10/tuning/step2_hybrid_parser/hybrid_parser.py` 의 `process_image_hybrid()` 함수가 핵심입니다.

```
def process_image_hybrid(page, dpi=150, threshold=None, vision_model=None):
    threshold = threshold or MIN_TEXT_LENGTH # 기본 50자

    # Step 1: OCR 먼저
    ocr_text = _ocr_page(page, dpi=dpi)
    ocr_len = len(ocr_text.strip())

    # Step 2: 판정 - 50자 이상이면 OCR 채택
    if ocr_len >= threshold:
        return {"strategy": "ocr", "text": ocr_text, "char_count": ocr_len}

    # Step 3: 부족하면 Vision LLM 전환
    vision_text = _vision_page(page, dpi=dpi, model=vision_model)
    return {"strategy": "vision", "text": vision_text or ocr_text}
```

핵심은 `MIN_TEXT_LENGTH`, 즉 **최소 글자 수**입니다. OCR이 뽑아낸 글자 수가 이 기준을 넘으면 OCR 결과를 채택하고, 넘지 못하면 비전 LLM으로 전환합니다. 기본값은 50자이며, `--threshold` 옵션이나 `.env`의 `MIN_TEXT_LENGTH`로 조절할 수 있습니다.

```
# 하이브리드 파싱 실행
python -m tuning.step2_hybrid_parser --step 2-1
```

[이미지: 실험 2-1: 하이브리드 파싱 결과]

결과 테이블에서 페이지별로 어떤 전략이 선택되었는지 확인합니다. 텍스트가 풍부한 페이지는 OCR, 스캔본이나 이미지 위주 페이지는 비전 LLM이 자동으로 선택됩니다.

```
# 판정 기준을 80자로 높여서 실험 - 비전 LLM 전환이 더 자주 일어남
python -m tuning.step2_hybrid_parser --step 2-1 --threshold 80
```

그런데 이 방식에는 **한정이 있습니다**. 스캔 품질이 나쁜 PDF에서 OCR이 755자를 뽑았다고 해봅시다. 글자 수로는 기준 50을 훌쩍 넘기니 OCR을 채택합니다. 하지만 실제 텍스트를 보면 “서사서”, “스스년”처럼 절반 이상이 깨진 글자입니다. 글자 수만으로는 **텍스트의 품질**까지 판단할 수 없다는 뜻입니다.

실무에서는 EasyOCR이 제공하는 **신뢰도(confidence) 점수**를 글자 수와 함께 활용하여 이런 경계 사례를 보완하기도 합니다. 하지만 더 근본적인 해결책이 있습니다.

실험 2-2: 텍스트 레이어 기반 하이브리드

OCR 결과의 글자 수를 세는 대신, PDF 자체에 텍스트 정보가 들어 있는지 확인하는 방법입니다.

PDF에는 두 종류가 있습니다. 워드나 한글로 작성한 **디지털 PDF**는 텍스트 레이어가 내장되어 있어서 복사·붙여넣기가 됩니다. 반면 스캐너로 찍은 **스캔본 PDF**는 이미지만 들어 있어서 텍스트 레이어가 없습니다. PyMuPDF의 `get_text()`로 이 차이를 바로 확인할 수 있습니다.

`ex10/tuning/step2_hybrid_parser/hybrid_parser.py`에 `process_image_textlayer()` 함수를 확인합니다.

```
def process_image_textlayer(page, dpi=150, vision_model=None):
    # Step 1: 텍스트 레이어 확인
    text_layer = page.get_text().strip()

    # Step 2: 텍스트가 있으면 디지털 PDF — 그대로 사용
    if text_layer:
        return {"strategy": "text_layer", "text": text_layer,
                "char_count": len(text_layer)}

    # Step 3: 텍스트가 없으면 스캔본 — Vision LLM 전환
    vision_text = _vision_page(page, dpi=dpi, model=vision_model)
    return {"strategy": "vision", "text": vision_text,
            "char_count": len(vision_text) if vision_text else 0}
```

2-1과 비교해봅시다. OCR을 돌릴 필요 자체가 없습니다. 디지털 PDF라면 텍스트 레이어를 바로 가져오니 OCR보다 빠르고 정확합니다. 스캔본이라면 텍스트 레이어가 비어 있으니 확실하게 비전 LLM으로 전환됩니다. 깨진 글자에 속을 일이 없습니다.

```
# 텍스트 레이어 기반 하이브리드 실행
python -m tuning.step2_hybrid_parser --step 2-2
```

[이미지: 실험 2-2: 텍스트 레이어 기반 하이브리드]

결과 테이블에서 "텍스트 레이어" 컬럼을 확인합니다. 디지털 PDF는 수천 자의 텍스트 레이어가 있어서 즉시 사용되고, 스캔본 PDF는 0자로 표시되며 비전 LLM이 처리합니다.

실무에서는 어떤 방식을 쓸까?

대부분의 실무 파이프라인은 텍스트 레이어 확인(방법 2)을 기본으로 사용합니다. Grab, 네이버 등의 대규모 문서 처리 시스템도 텍스트 레이어 유무로 1차 분기한 뒤, 스캔본만 별도 OCR/비전 파이프라인으로 보냅니다.

그런데 최근 트렌드는 OCR을 아예 건너뛰는 방향입니다. 페이지 전체를 이미지로 캡처해서 비전 LLM에 바로 넘기는 세 번째 방식이 빠르게 확산되고 있습니다. GPT-4o, Qwen2.5-VL 같은 최신 비전 LLM은 텍스트뿐 아니라 표, 도장, 서명, 레이아웃까지 한 번에 이해합니다. 실습 1에서 확인했듯 OCR은 글자가 깨지고 구조가 뒤흔드는 한계가 있습니다. 비전 LLM과 OCR을 결합하면 정확도가 크게 향상되며, OCR만 사용하는 것이 오히려 RAG 성능을 깎아먹을 수 있습니다.

정리하면 문서 처리 파이프라인의 흐름은 이렇게 진화하고 있습니다.

세대	방식	장단점
1세대	OCR만 사용	빠르지만 구조 손실, 깨진 글자
2세대	텍스트 레이어 + OCR/ 비전 LLM 분기	안정적, 현재 실무 표준
3세대	페이지 전체를 비전 LLM에 전달	가장 정확, 비용·속도 트레이드오프

이 실습에서는 2세대 방식까지 직접 구현했습니다. 3세대 방식은 실습 4의 최종 웹 UI에서 직접 확인합니다.

2.7 실습 3: RAG 평가 프레임워크 (tuning/step3_eval_framework/)

AI 비서가 “잘 찾고 있다”는 걸 어떻게 증명할까요? 팀장이 “검색 품질 어때?”라고 물었을 때 “괜찮은 것 같습니다”로는 부족합니다. 숫자로 된 성적표가 필요합니다.

이 실습에서는 세 가지 지표를 하나씩 만들어봅니다. 각 지표마다 “무엇을 측정하는지” 이해하고, 코드를 확인한 뒤, 직접 실행해서 결과를 봅니다.

파라미터	설명	권장값
<code>--step</code>	실험 단계 (2-1, 2-2, 2-3, compare, all)	—
<code>--k</code>	상위 몇 개 결과를 평가할지	3

시험을 보려면 정답지가 먼저 있어야 합니다. `data/test_questions.json`이 그 역할을 합니다.

`ex10/data/test_questions.json`

```
{
  "questions": [
    {
      "id": 1,
      "query": "연차 신청하려면 어떻게 해?",
      "relevant_sources": ["HR_취업규칙_v1.0"],
      "category": "비정형",
      "expected_answer": "AI HR 봇을 통해 스마트 휴가 승인 시스템으로 신청합니다."
    },
    {
      "id": 23,
      "query": "보안 규정이란 휴가 규정 둘 다 알려줘",
      "relevant_sources": ["SEC_보안규정_v1.0", "HR_취업규칙_v1.0"],
      "category": "복합",
      "expected_answer": "보안규정과 취업규칙 두 문서의 내용을 종합하여 안내합니다."
    }
  ]
}
```

필드	설명	예시
<code>query</code>	사용자가 실제로 물어볼 질문	“연차 신청하려면 어떻게 해?”
<code>relevant_sources</code>	이 질문의 정답이 들어 있는 문서 이름	<code>["HR_취업규칙_v1.0"]</code>
<code>expected_answer</code>	기대하는 답변 요약	“AI HR 봇으로 신청합니다.”

필드	설명	예시
category	질문 유형 — 비정형, 정형, 복합	“비정형”

핵심은 **relevant_sources**입니다. 검색기가 가져온 문서가 여기 적힌 문서와 일치하면 “맞혔다”, 다른 문서를 가져왔으면 “틀렸다”로 판정합니다. 이 파일에 총 37개 질문이 6개 문서에 걸쳐 들어있습니다. 이 중 7개는 의도적으로 어렵게 만든 **복합 질문**입니다. “재택근무 시 보안 규정은?” 처럼 정답이 두세 개 문서에 걸쳐 있어서, 재현율(Recall) 차이를 뚜렷하게 보여줍니다.

평가용 벡터DB는 청크 크기를 200자로 설정했습니다. 문서 6개가 약 36개 청크로 나뉘어, k=3으로 검색하면 정답을 못 찾는 질문이 생깁니다. 실제 서비스의 검색 품질 차이를 체험하기 위한 의도적인 설정입니다.

실험 3-1: 정확도(Precision@k) — “가져온 것 중 맞는 게 몇 개야?”

사서가 서가에서 책 3권을 가져왔습니다. 이 중 몇 권이 실제로 질문과 관련된 책일까요? 이것이 **정확도(Precision)**입니다.

`ex10/tuning/step3_eval_framework/metrics.py` 의 `calculate_precision_at_k()` 함수입니다.

```
def calculate_precision_at_k(retrieved_sources, relevant_sources, k):
    top_k = retrieved_sources[:k]
    relevant_set = set(relevant_sources)
    hits = sum(1 for src in top_k if any(rel in src for rel in relevant_set))
    return hits / k
```

3개를 가져왔는데 2개가 정답이면 Precision@3 = 2/3 = 0.67입니다.

```
python -m tuning.step3_eval_framework --step 2-1 --k 3
```

[이미지: 실험 3-1: Precision@k 측정]

평균 Precision@3이 0.64입니다. 3개를 가져오면 1개 정도는 엉뚱한 문서라는 뜻입니다. 이 숫자가 실습 4의 완성 웹 UI에서 CH08~09 기술을 적용한 뒤 얼마나 올라가는지가 핵심입니다.

실행 결과에 **MRR(Mean Reciprocal Rank)** 도 함께 표시됩니다. 첫 번째 정답이 1위에 나오면 1.0, 2위에 나오면 0.5, 3위에 나오면 0.33입니다. “사용자가 원하는 답을 찾으려고 몇 번째까지 스크롤해야 하는가?”를 숫자로 보여주는 지표입니다.

실험 3-2: 재현율(Recall@k) — “놓친 건 없어?”

정답 문서가 4개 있었는데, 사서가 그중 몇 개를 찾아왔을까요? 이것이 **재현율(Recall)**입니다.

```
def calculate_recall_at_k(retrieved_sources, relevant_sources, k):
    top_k = retrieved_sources[:k]
    relevant_set = set(relevant_sources)
```

```
hits = sum(1 for rel in relevant_set if any(rel in src for src in top_k))
return hits / len(relevant_set)
```

k를 늘리면 재현율은 올라가지만 정확도는 떨어질 수 있습니다. 이 **정확도-재현율 트레이드오프**가 평가의 핵심입니다.

```
python -m tuning.step3_eval_framework --step 2-2 --k 3
```

[이미지: 실험 3-2: Recall@3]

정답 문서가 1개인 질문은 k=3이면 대부분 찾아냅니다(R@3 = 1.00). 하지만 “채택근무 시 보안 규정은?”처럼 정답이 HR 규정 **과** 보안 규정 두 곳에 걸치는 복합 질문은 R@3이 낮아집니다. --k 값을 5, 10으로 바꿔가며 실험해보세요.

실험 3-3: 환각률(Hallucination Rate) — “지어낸 건 없어?”

검색은 잘 찾았는데, LLM이 답변할 때 출처에 없는 내용을 지어내면 어떡할까요? CH01에서 처음 만났던 그 문제를, 이제 숫자로 잡아냅니다.

측정 흐름은 이렇습니다.

1. 질문 + 검색된 컨텍스트를 Ollama LLM에 전달하여 답변 생성
2. 답변에서 핵심 단어(3글자 초과)를 추출
3. 핵심 단어가 컨텍스트에 얼마나 포함되어 있는지 매칭률 계산
4. 매칭률이 30% 미만이면 “환각”으로 판정

```
def estimate_hallucination_rate(answers, contexts):
    hallucination_count = 0
    for answer, context_docs in zip(answers, contexts):
        context_combined = " ".join(context_docs).lower()
        key_words = [w for w in answer.lower().split() if len(w) > 3]
        if key_words:
            context_words = set(context_combined.split())
            overlap = len([w for w in key_words if w in context_words]) / len(key_words)
            if overlap < 0.3:
                hallucination_count += 1
    return hallucination_count / len(answers) if answers else 0.0
```

```
python -m tuning.step3_eval_framework --step 2-3
```

이 실험은 Ollama LLM(llama3.1:8b)으로 37개 질문에 대한 답변을 생성합니다. 모델에 따라 1~3분 정도 소요됩니다.

[이미지: 실험 3-3: 환각률 측정]

환각률이 높게 나오는 것이 정상입니다. 아직 리랭킹도, 쿼리 재작성도 적용하지 않은 상태이니까요. 검색이 엉뚱한 문서를 가져오면 LLM도 “그렇듯하게” 지어낼 수밖에 없습니다. 이 숫자가 바로 **개선의 출발점**입니다.

2.8 실습 4: 문서 캡처와 근거 표시 (src/capture.py, src/evidence.py)

지금까지 만든 기술을 하나의 웹 UI로 통합합니다. 이 실습에서는 **src/** 디렉토리에 **새 파일 2개**를 직접 작성합니다.

파일	역할	태그
src/capture.py	문서 캡처 (PDF → PNG + 텍스트 추출)	[실습] 새로 작성
src/evidence.py	근거 이미지 URL 변환	[실습] 새로 작성
src/tools/search_documents.py	검색 파이프라인 (image_path 저장·반환)	[설명] CH08~09 확장
app/main.py	FastAPI 진입점 + 정적 파일 마운트	[참고] 완성본
app/chat_api.py	채팅 API (evidence.py 활용)	[참고] 완성본

실습 4-1: 문서 캡처 (src/capture.py)

PDF 페이지를 이미지로 저장하고 텍스트를 함께 추출하는 모듈입니다. 아래 코드를 **ex10/src/capture.py**에 단계별로 나누어 작성합니다.

1단계: 임포트 및 경로 설정

```

"""문서 캡처 — PDF 페이지를 이미지로 저장한다."""

from pathlib import Path

import fitz # PyMuPDF

BASE_DIR = Path(__file__).resolve().parent.parent
DOCS_DIR = BASE_DIR / "data" / "docs"
CAPTURED_DIR = BASE_DIR / "data" / "captured" / "pdf"

```

fitz는 PyMuPDF 라이브러리입니다. **CAPTURED_DIR**은 캡처된 이미지가 저장될 경로입니다.

2단계: 캡처 함수 작성

```

def capture_pdf_pages(pdf_path):
    """PDF를 페이지별 PNG로 캡처하고 텍스트를 추출한다."""
    CAPTURED_DIR.mkdir(parents=True, exist_ok=True)

```

```

doc = fitz.open(str(pdf_path))
results = []
for page_num in range(len(doc)):
    page = doc[page_num]
    # ① 페이지를 PNG 이미지로 렌더링
    pix = page.get_pixmap(dpi=200)
    img_path = CAPTURED_DIR / f"{pdf_path.stem}_page_{page_num + 1}.png"
    pix.save(str(img_path))
    # ② 텍스트 레이어도 함께 추출
    text = page.get_text()
    results.append({
        "page": page_num + 1,
        "image_path": str(img_path), # ← 이 경로가 근거 이미지의 핵심
        "text": text,
        "metadata": {"source": pdf_path.name, "image_path": str(img_path)},
    })
doc.close()
return results

```

텍스트를 추출하는 동시에 **페이지 이미지도 저장**합니다. 핵심은 `image_path`입니다. 이 경로가 어떻게 흘러가는지 `src/tools/search_documents.py`에서 확인합니다.

[설명] `search_documents.py` — 검색 파이프라인 전체 흐름

CH05에서 기본 검색을 만들고, CH08~09에서 기술을 하나씩 추가해온 파일입니다. 현재 검색 파이프라인의 전체 흐름은 다음과 같습니다.



검색 파이프라인 전체 흐름: 사용자 질문 → 약어 확장 → HyDE → 하이브리드 검색 → 리랭킹 → 결과

CH10에서 달라진 부분만 짚겠습니다.

① 벡터DB 저장 — `image_path`를 메타데이터에 포함

```

# search_documents.py - _build_vectorstore() 중 일부
for i, doc in enumerate(docs):
    meta = {"source": doc["source"], "page": doc.get("page", 1)}
    if doc.get("image_path"):
        meta["image_path"] = doc["image_path"] # ← 캡처 이미지 경로
    collection.add(
        ids=[f"doc_{i}"],
        documents=[doc["content"]],
        metadatas=[meta],
    )

```

`capture.py`가 반환한 `image_path`가 ChromaDB 메타데이터에 저장됩니다. 텍스트와 이미지 경로가 같은 레코드에 묶이는 셈입니다.

② 검색 결과 반환 — `image_path`를 꺼내 프론트엔드에 전달

```
# search_documents.py - search_documents() 중 일부
for doc in results:
    entry = {
        "content": doc["content"],
        "source": doc.get("source", "unknown"),
        "score": round(doc.get("rerank_score", doc.get("score", 0)), 4),
        "page": doc.get("page", ""),
    }
    if doc.get("image_path"):
        entry["image_path"] = doc["image_path"] # ← 근거 이미지 전달
    formatted.append(entry)
```

검색 결과에 `image_path`가 포함되어 있으면 응답에 함께 담깁니다. `chat_api.py`가 이 경로를 받아 `evidence.py`의 `resolve_image_url()`로 웹 URL로 변환하고, 프론트엔드에 근거 이미지로 표시합니다.

실습 4-2: 근거 이미지 URL 변환 (src/evidence.py)

캡처된 이미지의 절대 경로를 웹 브라우저에서 접근할 수 있는 URL로 변환하는 모듈입니다. 아래 코드를 `ex10/src/evidence.py`에 단계별로 나누어 작성합니다.

1단계: 임포트 및 경로 설정

```
"""근거 이미지 경로 변환 - 절대 경로를 웹 URL로 바꾼다."""

import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
CAPTURED_DIR = BASE_DIR / "data" / "captured"
```

2단계: 경로 변환 함수

```
def resolve_image_url(image_path):
    """절대 경로를 /captured/... 웹 URL로 변환한다."""
    captured_str = str(CAPTURED_DIR)
    if captured_str in image_path:
        relative = image_path.split("captured" + os.sep, 1)[-1]
        return f"/captured/{relative}"
    if image_path.startswith("/captured/"):
        return image_path
    return ""
```

`/Users/.../data/captured/pdf/HR_정보보안서약서_page_1.png` 같은 절대 경로가 `/captured/pdf/HR_정보보안서약서_page_1.png` 으로 변환됩니다. `app/chat_api.py`가 이 함수를 호출해서 채팅 응답의 `evidence_images` 리스트에 담습니다.

3단계: 이미지 목록 조회 함수

```
def list_captured_images():
    """캡처된 이미지 파일 목록과 웹 URL을 반환한다."""
    images = []
    if not CAPTURED_DIR.exists():
        return images

    for sub_dir in sorted(CAPTURED_DIR.iterdir()):
        if not sub_dir.is_dir():
            continue
        fmt = sub_dir.name.upper()
        for img in sorted(sub_dir.glob("*.png")):
            size_kb = img.stat().st_size / 1024
            web_url = resolve_image_url(str(img))
            images.append({
                "format": fmt,
                "filename": img.name,
                "size_kb": round(size_kb, 1),
                "web_url": web_url,
            })
    return images
```

실습 4-3: 웹 서버 기동

`app/main.py`에 이미 완성본이 들어 있습니다. 핵심 구조만 확인합니다.

```
# app/main.py - 핵심 부분
app = FastAPI(title="사내 AI 비서 - ex10")

# ① 정적 파일 마운트 - CSS/JS + 캡처 이미지
app.mount("/static", StaticFiles(directory="static"), name="static")
app.mount("/captured", StaticFiles(directory=_CAPTURED_DIR), name="captured")

# ② 라우터 등록
app.include_router(chat_router)
```

핵심은 `/captured` 마운트입니다. 실습 4-1에서 `data/captured/` 디렉토리에 저장한 PNG 파일을, 실습 4-2에서 `/captured/...` URL로 변환했고, 이 마운트 한 줄로 브라우저가 실제 이미지에 접근할 수 있게 됩니다.

데이터가 흐르는 경로

캡처(4-1) → 벡터DB 저장(`image_path` 메타데이터) → 검색 → URL 변환(4-2) → API 응답 → 브라우저 표시(4-3)

나머지 인프라 코드(에이전트 설정, 라우터, 캐시, 모니터링)는 CH05~07에서 만든 것과 동일합니다. `src/` 디렉토리에 이미 들어 있으므로 별도로 작성하지 않습니다.

```
python run.py
```

브라우저에서 <http://localhost:8000>에 접속하여 질문을 던져보세요. 답변 아래에 원본 PDF 페이지 이미지가 근거로 표시됩니다. 썸네일을 클릭하면 원본 크기로 확대됩니다.

예시 질문 – “취업규칙에서 연차 관련 조항이 뭐야?” – “정보보안서약서에 어떤 내용이 있어?” – “신규서비스 런칭 전략 알려줘”

[이미지: 웹 UI에서 질문을 던지면 답변 아래에 원본 PDF 페이지 이미지가 근거로 표시된다]

실습 4-4: 성능 비교 — 실습 3의 성적표와 대조

실습 3에서 측정한 기준선과 비교합니다. CH08~09 기술(약어 확장, 하이브리드 검색, 리랭킹)이 `src/tools/search_documents.py`에 통합되어 있으므로, 웹 UI를 기동한 상태에서 평가 프레임워크를 다시 돌리면 됩니다.

```
python -m tuning.step3_eval_framework --step compare
```

[CAPTURE NEEDED]

적용 기술	출처	정확도 기여	비용
약어/동义词 확장	CH09	용어 불일치 해소	없음
하이브리드 검색	CH08	키워드+의미 균형	없음
리랭킹	CH08	순위 정밀도 향상	Cross-Encoder 추론
HyDE	CH09	스타일 갭 해소	+1 LLM 호출

각 기술의 상세한 동작 원리는 CH08~09에서 다루었습니다. 여기서는 **통합 결과**를 숫자로 확인하는 것이 목적입니다.

2.9 더 알아보기

RAGAS는 뭔가요? — RAGAS(Retrieval Augmented Generation Assessment)는 RAG 시스템을 자동으로 평가하는 프레임워크입니다. 충실도(Faithfulness), 문맥 관련성(Context Relevancy), 답변 관련성(Answer Relevancy) 등 더 정교한 지표(Metric)를 제공합니다. 이 책에서는 평가의 기본 개념(정확도, 재현율, 환각률)에 집중하고, RAGAS 같은 고급 프레임워크는 다루지 않습니다. `step3_eval_framework`에 RAGAS 연동 코드가 준비되어 있으니, 관심 있다면 `USE_RAGAS=true` 환경 변수를 설정하고 시도해보세요.

테스트 케이스(Test Case)를 어떻게 만들어야 하나요? — 실제 사용자가 자주 하는 질문 20~30개를 모으세요. 각 질문에 대해 “이 질문의 정답은 이 문서에 있다”를 표시합니다. `data/test_questions.json`이 바로 이 평가용 질문 파일입니다. 처음 만들 때 수작업이 필요하지만, 한 번 만들면 튜닝할 때마다 재사용할 수 있습니다.

정확도와 재현율 중 뭐가 더 중요하나요? — 사내 AI 비서에서는 정확도가 더 중요합니다. 사용자는 상위 3개 결과만 봅니다. 3개 중 2개가 엉뚱한 문서이면 신뢰를 잃습니다. 재현율은 놓친 문서가 있어도 상위 결과가 정확하면 사용자 경험에 큰 영향이 없습니다.

OCR 결과를 마크다운으로 구조화할 수는 없나요? — OCR은 글자만 뽑기 때문에 구조(제목, 조항, 표)가 없습니다. 두 가지 방법이 있습니다. 첫째, OCR로 텍스트를 뽑은 뒤 텍스트 LLM에게 “마크다운으로 정리해줘”라고 요청하는 2단계 방식입니다. 둘째, 정규식으로 “제N조”를 제목으로, “1.”을 리스트로 바꾸는 규칙 기반 후처리입니다.

하이브리드 처리의 최소 글자 수는 어떻게 정하나요? — 실습 2-1에서 50자를 기본값으로 사용했습니다. OCR이 노이즈만 뽑는 스캔본은 보통 10~20자 이하이고, 텍스트가 포함된 PDF는 수백 자 이상이라 50자면 충분히 구분됩니다. 다만 실습 2-1에서 확인했듯 깨진 글자가 많으면 글자 수만으로는 한계가 있으므로, 실무에서는 실습 2-2의 텍스트 레이어 확인 방식을 권장합니다.

2.10 이것만은 기억하세요.

- **측정해야 개선할 수 있습니다.** Precision@k, Recall@k, MRR, Hallucination Rate — 이 네 가지 숫자가 RAG 시스템의 성적표입니다. 느낌이 아니라 숫자로 말하세요.
- **이미지도 문서입니다.** PDF 속 표, 차트, 조직도는 텍스트로 변환하면 정보가 손실됩니다. 하이브리드 처리로 OCR과 비전 LLM을 자동 선택하세요.
- **하이브리드가 답입니다.** 검색도 하이브리드(BM25+벡터), 이미지 처리도 하이브리드(OCR+Vision). 하나의 기술에 의존하지 말고 서로 보완하게 만드세요.
- **ex10이 끝이 아닙니다.** 성적표가 있으니 이제 어디를 개선해야 하는지 보입니다. 정확도가 낮으면 리랭킹을 강화하고, 환각률이 높으면 프롬프트를 조정하세요.
- **CH01에서 ex10까지, 이 여정의 핵심은 하나입니다.** “직접 만들어봐야 이해한다.” 환각을 보고, 문서를 넣고, 검색하고, 답변하고, 통합하고, 안정화하고, 튜닝하고, 측정하고 — 이 흐름을 한 번 경험한 여러분은, 이제 어떤 RAG 시스템이든 만들 준비가 되었습니다. 에필로그에서 이 여정을 마무리합니다.

에필로그: 당신만의 AI 비서를

ConnectHR은 완성됐습니다.

LLM이 우리 회사 문서를 모른다는 걸 깨달은 날부터, 파싱하고 청킹하고 임베딩하고, 에이전트를 붙이고 튜닝하고 평가하기까지. 꽤 긴 여정이었습니다.

하지만 ConnectHR은 예시입니다.

인사팀 문서를 넣었지만, 넣어야 할 문서는 회사마다 다릅니다. 연차 규정이 아니라 제품 매뉴얼일 수도 있고, 법무 계약서일 수도 있고, 고객 응대 스크립트일 수도 있습니다.

에이전트가 조회하는 건 직원 DB였지만, 어떤 곳은 재고 DB일 것이고, 어떤 곳은 주문 DB일 겁니다.

ConnectHR의 구조는 그대로입니다. 파싱 → 청킹 → 임베딩 → 검색 → 답변. 그 흐름은 어디서든 같습니다. 달라지는 건 넣는 문서와 연결하는 DB뿐입니다.

이 책에서 배운 것들을 생각해보면:

- 환각을 체험하면서 RAG가 왜 필요한지 이해했습니다.
 - 파싱과 청킹을 해보면서 “문서를 넣는다”는 게 무슨 뜻인지 알게 됐습니다.
 - 검색이 엉뚱한 결과를 가져올 때, LLM 탓이 아니라 Retriever를 먼저 의심하게 됐습니다.
 - 질문 하나가 바뀌면 답변이 달라진다는 것도 봤습니다.
 - 숫자로 측정하기 전까지는 “더 좋아진 것 같다”는 느낌뿐이라는 것도 알게 됐습니다.
-

이제 ConnectHR을 당신 회사의 이름으로 바꿔보세요.

문서를 바꾸고, DB를 바꾸고, 팀이 자주 묻는 질문에 맞춰 튜닝해보세요. 그 과정에서 이 책에서 본 것과 똑같은 시행착오를 겪을 겁니다. “왜 이 문서는 못 찾지?”, “왜 이 질문엔 엉뚱한 답을 하지?” 그럴 때 이 책을 다시 펼쳐보세요.

직접 만들어봐야 이해한다고 했습니다.

ConnectHR을 만들었으니, 이제 당신만의 AI 비서를 만들 차례입니다.

부록

A. 환경 설정

Python 설치

이 책의 모든 예제는 Python 3.12 를 기준으로 작성했습니다. 3.10~3.12에서 동작하며, 3.13 이상에서는 일부 패키지 호환성 문제가 있을 수 있습니다.

macOS

```
# Homebrew로 설치  
brew install python@3.12
```

Windows

공식 사이트(<https://www.python.org/downloads/>)에서 Python 3.12를 다운로드합니다. 설치 시 “Add Python to PATH” 체크 박스를 반드시 선택하세요.

가상환경 설정

예제마다 패키지 버전이 다를 수 있으므로, 반드시 가상환경을 만들어서 진행하세요.

```
# 가상환경 생성  
python3 -m venv .venv  
  
# 활성화 (macOS/Linux)  
source .venv/bin/activate  
  
# 활성화 (Windows)  
.venv\Scripts\activate  
  
# 패키지 설치  
pip install -r requirements.txt
```

가상환경이 활성화되면 터미널 프롬프트 앞에 (.venv) 가 표시됩니다.

자주 만나는 오류

python 명령어가 안 될 때

macOS/Linux에서는 **python** 대신 **python3** 를 사용해야 할 수 있습니다.

```
# python이 안 되면
python3 --version
python3 -m venv .venv
```

pip install 에서 권한 오류

가상환경 없이 시스템 Python에 설치하려고 하면 권한 오류가 발생합니다. 가상환경을 먼저 활성화하세요.

```
# 이렇게 하면 안 됩니다
pip install langchain # PermissionError 또는 externally-managed-environment

# 이렇게 하세요
source .venv/bin/activate # 먼저 가상환경 활성화
pip install -r requirements.txt
```

pip 대신 pip3

pip 명령이 안 되면 **pip3** 를 사용하세요. 가상환경 안에서는 둘 다 동일합니다.

psycpg2-binary 설치 실패 (macOS Apple Silicon)

M1/M2/M3 Mac에서 psycpg2-binary 설치가 실패할 수 있습니다.

```
# libpq 먼저 설치
brew install libpq
pip install psycpg2-binary
```

LLM 설정

이 책은 두 가지 LLM 백엔드를 지원합니다.

옵션	장점	환경 변수
Ollama (로컬)	무료, 오프라인 가능	OLLAMA_BASE_URL, OLLAMA_MODEL
OpenAI (클라우드)	품질 우수, 설치 불필요	OPENAI_API_KEY, OPENAI_MODEL

CH04~06에서는 **deepseek-r1:8b**를 사용합니다. CH07부터 에이전트의 툴콜링(Tool Calling)이 필요해서 **llama3.1:8b**로 전환합니다. DeepSeek-R1은 툴콜링을 지원하지 않기 때문입니다.

```
ollama pull deepseek-r1:8b # CH04~06
ollama pull llama3.1:8b # CH07~
```

비전 LLM 설정 (CH10)

CH10에서는 스캔 PDF를 처리하기 위해 OCR(EasyOCR)과 비전 LLM을 사용합니다.

```
# EasyOCR - requirements.txt에 포함, 별도 설치 불필요
# pip install easyocr (이미 requirements.txt에 포함)

# 비전 LLM (실험 1~2)
ollama pull qwen2.5vl:7b
```

EasyOCR은 `pip install`만으로 설치되므로 별도 시스템 패키지가 필요 없습니다. 컴퓨터 사양이 부족하면 `.env`에서 `VISION_PROVIDER=openai`로 전환하여 GPT-4o-mini 비전 API를 사용할 수 있습니다.

.env 파일 예시

각 예제 폴더의 `.env.example` 을 `.env` 로 복사한 뒤 값을 채워넣으세요.

```
# LLM 설정
LLM_PROVIDER=ollama
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=deepseek-r1:8b      # CH04~06
# OLLAMA_MODEL=llama3.1:8b      # CH07~ (툴콜링 필요)

# OpenAI 사용 시
# LLM_PROVIDER=openai
# OPENAI_API_KEY=sk-...
# OPENAI_MODEL=gpt-4o-mini

# PostgreSQL (ex02 이후)
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=rag_db
POSTGRES_USER=rag_user
POSTGRES_PASSWORD=rag_password

# 벡터DB + 임베딩
CHROMA_PERSIST_DIR=./data/chroma_db
EMBEDDING_MODEL=jhgan/ko-sroberta-multitask

# 비전 LLM (ex10)
VISION_MODEL=qwen2.5vl:7b
VISION_PROVIDER=ollama
# VISION_PROVIDER=openai # 로컬 사양 부족 시
```

Docker (PostgreSQL)

ex02 이후 예제는 PostgreSQL이 필요합니다. Docker Compose로 실행합니다.

```
docker compose up -d
```

PostgreSQL 16 Alpine 이미지를 사용하며, `data/schema.sql` 이 자동으로 초기화됩니다.

핵심 패키지 요약

패키지	버전	용도
<code>langchain</code>	0.3.x	RAG 파이프라인, 에이전트
<code>chromadb</code>	1.5.x	벡터 데이터베이스
<code>fastapi</code>	0.115.x	API 서버
<code>sentence-transformers</code>	3.3.x	한국어 임베딩 모델
<code>psycopg2-binary</code>	2.9.x	PostgreSQL 연결
<code>pypdf</code>	4.3.x	PDF 파싱
<code>python-docx</code>	1.1.x	DOCX 파싱
<code>openpyxl</code>	3.1.x	XLSX 파싱
<code>rank-bm25</code>	0.2.x	하이브리드 검색 (CH08)
<code>easyocr</code>	1.7.x	OCR (CH10) — pip만으로 설치

각 예제 폴더의 `requirements.txt` 로 한 번에 설치할 수 있습니다.

```
pip install -r requirements.txt
```

C. 더 알고 싶다면

RAG와 LLM 응용에 관심이 생겼다면, 아래 주제를 찾아보세요.

- **Graph RAG** — 단순 링크 검색이 아니라 지식 그래프로 문서 관계를 표현하는 방식
- **멀티모달 RAG** — 텍스트뿐 아니라 이미지, 표, 차트를 함께 다루는 RAG
- **Agentic RAG** — 에이전트가 검색 전략을 스스로 결정하는 방식

ConnectHR에서 한 단계 더 나아가는 방향들입니다.
